

BitZ: proofs and commitments in arbitrary rings through binary fields

Remco Bloemen¹, Albert Garreta², Marcin Kostrzewa³, Shreyas Londhe⁴, and John Wu²

¹World Foundation

²Nethermind Research

³Reilabs

⁴Ivy Research

September 18, 2026

Abstract

We introduce BitZ, a hash-based Polynomial Commitment Scheme (PCS) for committing to multilinear polynomials $\mathbf{f}$ with coefficients in an arbitrary finitely generated ring S , e.g. a finite field $\mathbb{F}$, the integers $\mathbb{Z}$, a cyclotomic ring, etc. Moreover, given another arbitrary ring R and a ring homomorphism $\psi : S \rightarrow R$, BitZ then proves evaluation claims over R for the polynomial $\psi(\mathbf{f})$. BitZ’s costs depend almost exclusively on the number of bits in the coefficients of $\mathbf{f}$, and not on S , R or ψ . Moreover, BitZ provides range checks (or more generally, bit-size checks) essentially for free.

BitZ can thus be used as a PCS in essentially any proof system. We do so to build a SNARK, called BitZ-SNARK, for integer polynomial constraints, following the fingerprinting technique of Campanelli and Hall-Andersen, where one commits over $\mathbb{Z}$ and proves the constraints over a random prime field $\mathbb{F}_q$, i.e. BitZ is deployed with $S = \mathbb{Z}$, $R = \mathbb{F}_q$, and ψ reduction modulo q . BitZ applies equally to other ring-based proof systems, or field-based ones.

Our approach decomposes the witness into a string of bits and commits to it over a binary field $\mathbb{F}_{2^\nu}$, in packed form. Then, if necessary, transforms the evaluation claim over R into one over $\mathbb{Z}$ via a randomized process. Finally, using the tensor structure of “eq” vectors, it decomposes the claim into square root many inner-product claims over $\mathbb{Z}$, and proves each of these in the exponent of a generator of $\mathbb{F}_{2^\nu}^*$ via the GKR protocol for batched grand products. We design specific optimizations that exploit that the factors across all the grand products have low entropy (i.e. they take a “small” amount of different values).

We implement BitZ-SNARK and use it to prove, among others, SHA-256 hashing followed by ECDSA signature verification; RSA modular exponentiation and Poseidon hashing; and integer multiplication, consistently obtaining better performance than prior approaches on most tasks. As an example, we prove a SHA-256 hash of a 8 kB (2^7 compressions) message followed by an ECDSA signature verification with 68 ms and 4.0 ms prover and verifier time, respectively, and with a proof of 92 kB, single-threaded on a MacBook Air M5 24GB (CPU-only), at 100 bits of round-by-round knowledge soundness. With 10 threads the times are 26.0 ms and 3.6 ms.

Contents

1 Introduction

3

1.1	Our contributions	3
1.2	BitZ in a nutshell	6
1.3	Related work	10
1.4	Acknowledgements	11
2	Technical overview	11
2.1	The prime field case	12
2.2	Generalization to pairs of arbitrary finitely generated rings	15
2.3	GKR for low-entropy batched grand products	18
2.4	Virtual $\mathbb{F}_2$ -linear transforms, NP-complete multi-domain linear relations, and hybrid proof systems	21
2.5	Round-by-round knowledge soundness	23
2.6	Further work and open problems	26
3	Preliminaries	26
3.1	Algebra	26
3.2	Linear codes and relations	27
3.3	Interactive Oracle Reductions (IOR) of proximity	29
4	BitZ IOP for LinBitsRings	30
4.1	The core LinBitsRings relation	31
4.2	Tensor decomposition and products of constrained codes	32
4.3	The BitZ IOP for the core LinBitsRings relation	33
4.4	Soundness and completeness	38
4.5	The general LinBitsRings relation	42
4.6	The BitZ IOP for the general LinBitsRings relation	44
4.7	Instantiation and achieving 100 or 128 bits of soundness	47
5	An end-to-end BitZ-based SNARK over any finitely generated ring	52
6	A GKR protocol for low entropy batched grand products	54
6.1	Prover optimization via lookup tables	58
7	Experiments	64
7.1	Raw performance of the BitZ PCS for the core LinBitsRings relation	66
7.2	RSA accumulation and Poseidon hashing	67
7.3	SHA-256 hashing followed by ECDSA signature verification	67
7.4	Modular multiplications and bit operations (mod 2^{32} multiplication and SHA-256 hashing)	68
7.5	Integer multiplication	71
7.6	Comparison with Fields-witch	74
	References	75
A	Deferred proofs	80
A.1	Completeness and soundness of Construction 4.13	80
A.2	Soundness of Out Of Domain Sampling (OOD) in Construction 4.4	81

1 Introduction

A Succinct Non-interactive Argument of Knowledge (SNARK) for a relation $\mathcal{R}$ is a non-interactive protocol that enables a prover to convince a verifier that, for a given public input $\mathbf{x}$, they know a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, and where communication and verifier runtime are sublinear (i.e. the scheme is succinct).

In the vast majority of scenarios, $\mathcal{R}$ is defined over an algebraic object R , most commonly a finite field, and $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ if and only if $(\mathbf{x}, \mathbf{w})$ consists of elements that satisfy some polynomial equations over R , called constraints. The choice of R has a crucial impact on the design and performance of the proof system, for two reasons:

1. **(Non-)Nativity of the computation.** Usually the computations one wants to prove are not native to R , so one has to perform a more or less complex emulation of the computation into polynomial equations over R . This can affect the size of the input-witness pair $\mathbf{x}, \mathbf{w}$ significantly [KPS18, BFK⁺24, OKMZ24, GWV⁺25, DP23], which is the main cost driver in SNARK costs. Additionally, the emulation step can be technically involved (thus requiring experts to perform it) and prone to security errors [CET⁺24, WSC⁺24, Imm23, 0xP] (thus requiring extensive security audits).
2. **Proof system efficiency.** The efficiency and design of the prover and verifier algorithms are affected by the choice of R : for example, modern CPUs are more performant for operations in certain small fields (e.g. M31 [HLN23, HLP24], Babybear [BG23], Goldilocks [Pol22]) than in arbitrary fields, especially if these are larger. Importantly, another class of fields where CPUs operate exceptionally fast are *binary fields*, i.e. fields of order a power of two.

Recently, two independent lines of research have received increased attention:

- Designing proof systems for relations expressed over the ring of integers $R = \mathbb{Z}$, and over other rings that are not fields [CHA24, GWV⁺25, AGK⁺26, SSPP26, CXNB26, GNSV23, SV22, BBMHS22, HMZ25, WZD25]. The motivation behind this is to address Item 1 above: in many practical scenarios, the computation to be proved is most succinctly and easily expressed over such rings, see [GWV⁺25, AGK⁺26, CXNB26].
- Designing proof systems for $R = \mathbb{F}_{2^\nu}$ a binary field, for example [BSBHR18b, BSCKL22, DP24, Sou24, BRW26]. The reason for this is twofold: first, some computations like classic hashes (SHA-256, Blake, etc.) are easier to express over $\mathbb{F}_{2^\nu}$ than over other structures such as prime fields (however, these hash functions remain relatively un-native to $\mathbb{F}_{2^\nu}$ since arithmetic modulo 2^{32} or 2^{64} remains relatively foreign to them). Second, binary fields are highly performant on modern CPU's, and further enable using a series protocol techniques that greatly benefit performance, most importantly *packed* polynomial commitment schemes [DP23, DP24].

1.1 Our contributions

In this work we bring the two aforementioned lines of research together. Concretely, we present BitZ, a hash-based Polynomial Commitment Scheme (PCS) that can commit to a witness vector $\mathbf{f} \in S^{\ell^1}$ over an arbitrary finitely generated ring S . For example, S can be any finite field $\mathbb{F}$, the ring of integers $\mathbb{Z}$, a cyclotomic ring (for lattice-based cryptography), the integers modulo 2^{32} , etc.

¹With the entries of $\mathbf{f}$ belonging to a publicly specified finite subset of S .

BitZ can then prove linear constraints on $\mathbf{f}$, such as Multilinear Extension (MLE) evaluation claims, over S , of the form

$$\langle \mathbf{f}, \mathbf{v} \rangle = \mu \quad \text{in } S$$

for some public $\mathbf{v} \in S^\ell$, $\mu \in S$. More generally, given another finitely generated ring R and a ring homomorphism $\psi : S \rightarrow R$, BitZ can prove claims of the form

$$\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu \quad \text{in } R$$

for public $\mathbf{v} \in R^\ell$, $\mu \in R$. For example, S could be the ring of integers, R could be a prime field $\mathbb{F}_q$, and ψ could be reduction modulo q . See below for an example of where such setting is useful.

We implement BitZ² and use it, among others (see below), to build a SNARK for proving R1CS constraints over the integers, following the fingerprinting paradigm from Campanelli and Hall-Andersen [CHA24]. Concretely, we commit to witnesses over the integers (setting $S = \mathbb{Z}$); then the verifier samples a random prime q ; then the R1CS is projected to $\mathbb{F}_q$ through the morphism π_q consisting in reduction modulo q ; then a PIOP is run on the resulting R1CS, yielding a linear claim (an MLE claim) on the projected witness $\pi_q(\mathbf{f})$ of the form $\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu$; finally, this claim is proved with BitZ. The commitment is made before q is sampled, which is required for the soundness of the fingerprinting technique. The PIOP may output its claim over a field extension of $\mathbb{F}_q$, which BitZ can prove (Section 2.2). An example end-to-end SNARK is described in Section 5.

We also build a SNARK, called BitZ-SNARK, that can prove constraints simultaneously over the integers and binary fields, and which uses BitZ as PCS. This SNARK also supports *virtualization* of $\mathbb{F}_2$ addition (XOR) between integer witness, in the sense that it can commit to two vectors of integer bits and prove MLE evaluation or inner product claims of the XOR between them. We discuss this further in Section 1.2.1.

We emphasize that BitZ can also be used to prove more exotic constraints, such as lattice-type operations, by using as constraint system and PIOP those of, for example, Zinc+ [AGK⁺26], see Remark 5.2. Overall, our BitZ-based SNARKs support highly expressible constraints that allow to arithmetize many computations of interest with little technical complexity, and with small circuits.

The table below shows highlight benchmark results of our implementation of BitZ-SNARK. We compare against the most performant prior SNARKs we could find for each task at hand.

Note that multiplication modulo 2^n , say $x \cdot y = z \pmod{2^n}$, can be arithmetized over the integers as the constraint $x \cdot y = z + 2^n \cdot w$ with w being a new witness element. We discuss proving SHA-256 below.

²Our implementation is available at <https://github.com/albert-garreta/BitZ-pcs>.

Computation	Scheme	Prover (ms)		Verifier (ms)		Proof (KB)	Peak mem. (GB)
		1 thr	10 thr	1 thr	10 thr		
SHA-256 + ECDSA (8 KB message) (2^7 compressions)	BitZ-SNARK, rate 1/2	68.4	25.7	4.14	3.82	136	0.790
	BitZ-SNARK, rate 1/8	67.9	26.0	4.02	3.61	92	0.793
	Binius64 [Bin26], rate 1/2	168	63.0	12.7	6.37	394	0.941
	Binius64, rate 1/8	219	74.0	12.7	6.29	289	1.03
	Zinc+	4707	970	319	115	2432	2.03
RSA and Poseidon [CXNB26]	BitZ-SNARK, rate 1/8	287	122	7.9	10.1	268	–
	Limber [CXNB26]	1048	527	38.3	37.5	5769	–
	Zinc+ [AGK ⁺ 26]	1876	523	16.1	12.1	1272	–
2^{20} mult. mod 2^{32} and 2^{12} SHA-256	BitZ-SNARK, rate 1/2	1132	287	51.1	15.7	303	5.30
	BitZ-SNARK, rate 1/8	1456	360	50.6	15.9	176	5.48
	Binius64, rate 1/2	1995	617	87.5	27.3	569	12.7
	Binius64, rate 1/8	2417	710	85.8	26.0	387	12.7
2^{19} multiplications of 32-bit integers	BitZ-SNARK, rate 1/2	380	108	4.35	3.07	189	0.488
	BitZ-SNARK, rate 1/8	484	126	4.22	3.00	116	0.544
	Binius64, rate 1/2	863	277	20.4	9.46	472	3.67
	Binius64, rate 1/8	1052	319	19.9	8.99	330	3.93
	Limber	4418	1348	192	147	12193	3.66
	Zinc+	2847	757	31.5	14.4	1791	1.23

Table 1: Performance of BitZ on different computations of practical interest that are costly to arithmetize over finite fields. For each such computation, we also benchmark the performance of prior state of the art protocols. All experiments were run on a MacBook Air M5 24 GB, CPU-only, targeting 100 bits of security (except for the RSA and Poseidon experiment, which targets 114 bits). See Section 7 for details on these and more experiments.

BitZ presents the following properties:

1. **Pay-per-bit and algebraic independence.** Its performance depends essentially exclusively on the bit-size of the witness $\mathbf{f}$. In particular, the choice of ring S (and R and ψ if it applies) has a small impact on its performance.
2. **Composability and universality.** Because of its flexibility in its choice of rings, and because of the point above, BitZ can be used as a PCS in essentially any proof system. This includes standard proof systems based on finite fields, or more exotic ones working over different rings, e.g. the integers [CHA24, CXNB26, GWV⁺25, AGK⁺26, SSPP26], polynomial rings [AGK⁺26], lattices [BS23, NS24, KLOT25, NOZ26], etc.

For example, a proof system using a PCS that is not plausibly post-quantum secure, could replace its PCS with BitZ (which is plausibly post-quantum secure), regardless of the proof system’s underlying field.

3. **“Free” range checks.** BitZ provides range checks “for free”. For example, when committing to integer witnesses, BitZ exposes a bit-size parameter B . Then, BitZ can be used to commit to integers of that bit-size, and it automatically guarantees that a malicious prover cannot commit to larger integers. This allows to omit many range checks that a standard proof system would need to enforce.

1.2 BitZ in a nutshell

BitZ proves the aforementioned claim $\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu$ over R as follows. Assume first that R is a finite prime field $\mathbb{F} = \mathbb{F}_q$. We discuss the general case later. We describe first an initial, mostly insufficient, attempt.

1. **Bitification.** First, it transforms the claim into a claim of the form $\langle \psi(\mathbf{bits}(\mathbf{f})), \mathbf{u} \rangle = \mu$ over R , for some $\mathbf{u}$, and where $\mathbf{bits}(\mathbf{f})$ denotes a vector with entries in $\{0, 1\} \subseteq \mathbb{Z}^3$. Intuitively, $\mathbf{bits}(\mathbf{f})$ is simply the result of bit-decomposing the entries of $\mathbf{f}$ in a natural way. This decomposition is linear, and so an inner product on $\mathbf{f}$ equals an inner product on $\mathbf{bits}(\mathbf{f})$, for a different weight vector. For example, if $S = \mathbb{Z}$ and $\mathbf{f}$ is filled with B bit nonnegative integers, then $\mathbf{u} = \mathbf{v} \otimes (1, 2, \dots, 2^{B-1})$.
We note that the scheme publicly specifies a finite subset of S where the entries of $\mathbf{f}$ belong
2. **Commitment.** The prover commits to $\mathbf{bits}(\mathbf{f})$ over a binary field $\mathbb{F}_{2^\nu}$ (by looking at integers bits as elements from $\mathbb{F}_2$), using a packed commitment scheme such as WHIR [ACFY25] over $\mathbb{F}_{2^\nu}$, with the ring-switch techniques from [DP23, DP24]. Here *packed commitment* means that every ν bits are grouped together into a single field element of $\mathbb{F}_{2^\nu}$, and that we encode and Merklize the vector of length $|\mathbf{bits}(\mathbf{f})|/\nu$ over $\mathbb{F}_{2^\nu}$. *Ring-switching* allows to prove linear claims for $\mathbf{bits}(\mathbf{f})$ using only the commitment to the packed vector.
3. **Lift to integers.** Assuming $\mathbb{F}$ is a prime field of size q and $\psi : \mathbb{Z} \rightarrow \mathbb{F}$ is reduction modulo q , the prover computes and sends $\mu' \in \mathbb{Z}$ such that $\langle \pi_q^{-1}(\mathbf{u}), \mathbf{bits}(\mathbf{f}) \rangle = \mu'$ in $\mathbb{Z}$.
4. **Proof in the exponent of $\mathbb{F}_{2^\nu}$.** Let g be a generator of the multiplicative subgroup $\mathbb{F}_{2^\nu}^*$. Inspired by [Bin26], we have prover and verifier prove that $g^{\langle \pi_q^{-1}(\mathbf{u}), \mathbf{bits}(\mathbf{f}) \rangle} = g^{\mu'}$ in $\mathbb{F}_{2^\nu}$. Precisely, we have

$$g^{\langle \pi_q^{-1}(\mathbf{u}), \mathbf{bits}(\mathbf{f}) \rangle} = \prod_i \left((g^{\pi_q^{-1}(\mathbf{u}_i)} - 1) \cdot \mathbf{bits}(\mathbf{f})_i + 1 \right) = g^{\mu'} \quad \text{in } \mathbb{F}_{2^\nu}, \quad (1)$$

which is a polynomial expression over $\mathbb{F}_{2^\nu}$ on the vector $\mathbf{bits}(\mathbf{f})$ that we committed over $\mathbb{F}_{2^\nu}$. This is a standard setting where there is no “wrong-field” arithmetic, hence *in principle* one can use any binary-field based proof system over $\mathbb{F}_{2^\nu}$ to prove (1). In Section 2.3.1 and below in the “key point”, we compare our method to using and proving (1) with other approaches [Bin26, Sou26].

Remark 1.1. Proving constraints “in the exponent” is a well-known idea that goes back at least to [Bin26, Sou26]. One of our main contributions is 1) in what context this technique is used, namely inside the PCS to prove MLE evaluation claims (or inner product claims) of the witness, as opposed to proving the initial arithmetized computation, see Section 2.3.1 for more details; 2) the optimizations we design for such a specific “proof in the exponent” setting, see Sections 2.3 and 6; and 3) extending it so it can be used to prove constraints over arbitrary rings, and not just, say, integers or *prime* fields with small values (to avoid overflowing the multiplicative order of g).

There are several issues with the attempt above:

- **How to lift from general rings?** First, R may not be a prime field, in which case there is no natural lift to integers in Step 3; rather, there is a lift to integer polynomials $\mathbb{Z}[X_1, \dots, X_k]$.
- **How to lift large values?** Second, even if $R = \mathbb{F}_q$ is a prime field, $q = |\mathbb{F}|$ may be large and the linear expression $\langle \pi_q^{-1}(\mathbf{u}), \mathbf{bits}(\mathbf{f}) \rangle$ may overflow the order of $g \in \mathbb{F}_{2^\nu}^*$, which is $2^\nu - 1$. In this case, proving $\langle \pi_q^{-1}(\mathbf{u}), \mathbf{bits}(\mathbf{f}) \rangle = \mu'$ “in the exponent of g ” is unsound.

³ $\psi(\mathbf{bits}(\mathbf{f}))$ is the natural interpretation of integer bits as the 0 or 1 element of R .

- **Expensive grand product.** Third, even if neither of these problems hold, (1) is, in principle, an expensive constraint to prove: indeed, it is a length $|\mathbf{bits}(\mathbf{f})|$ grand product of, essentially, arbitrary factors from $\mathbb{F}_{2^\nu}$ (typically ≈ 128 bit elements). Grand products can be expensive, and furthermore the fact that the factors are large prevents us from using known sumcheck optimizations such as univariate skips [Gru24] and others from [BRW26, DT24, BDDT25].
- **Non-succinct verifier.** Fourth, when proving (1), typically one ends up with an MLE evaluation claim on the vector $(g^{\pi_q^{-1}(\mathbf{u})_i})_i$ (and another on $\mathbf{bits}(\mathbf{f})$, which is proved with any PCS over $\mathbb{F}_{2^\nu}$). If the verifier checks the claim itself, then it pays a linear cost. If otherwise the prover has to prove this claim, then it is essentially back to square one.

We solve these issues as follows:

Issues 1 and 2: Inspired by the fingerprinting techniques from [CHA24, AGK⁺26] (which we generalize to arbitrary finitely generated rings in this work), these are solved by lifting the inner product claim to $\mathbb{Z}[X_1, \dots, X_k]$ (after observing that all finitely generated rings are a projection of a ring such as $\mathbb{Z}[X_1, \dots, X_k]$), and then projecting the resulting claim to a new claim over a random prime field. Then we proceed as in Steps 3 and 4 with this new claim over $\mathbb{F}_{q'}$.

128 bits of security using $\mathbb{F}_{2^{128}}$, arbitrary rings R , and arbitrary sizes of the entries of $\mathbf{v}$. We manage to instantiate $\mathbb{BitZ}$ with 128 bits of security, using $\mathbb{F}_{2^\nu} = \mathbb{F}_{2^{128}}$, while having essentially no limitation on the size of the ring R we start with, see Section 4.7.

Issues 3 and 4: batched low entropy grand products. We split (1) into $\approx \sqrt{|\mathbf{bits}(\mathbf{f})|}$ grand products of the same form as (1), so that, even though in total there are $|\mathbf{bits}(\mathbf{f})|$ factors among all of them, these factors can only take $\sqrt{|\mathbf{bits}(\mathbf{f})|}$ values. More concretely, we assume $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ tensor-decomposes into $\approx \sqrt{|\mathbf{bits}(\mathbf{f})|}$ sized factors. This is typically the case since $\mathbf{v}$ is usually an “eq” vector, and $\mathbf{u} = L^\top \cdot \mathbf{v}$ for a highly structured matrix L ⁴. Then, assuming the notation of Step 3 above, we aim to prove in the exponent $\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \mathbf{bits}(\mathbf{f})_{*j} \rangle = \mu'_j$ in $\mathbb{Z}$ for all $j \in [|\mathbf{u}^{(2)}|]$, such that $\sum_j \pi_q(\mu'_j) \cdot \mathbf{u}_j^{(2)} = \mu$, where we index $\mathbf{bits}(\mathbf{f})$ with indices $i \in [|\mathbf{u}^{(1)}|], j \in [|\mathbf{u}^{(2)}|]$. We set up a grand product for each of these claims, obtaining

$$g^{\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \mathbf{bits}(\mathbf{f})_{*j} \rangle} = \prod_i \left((g^{\pi_q^{-1}(\mathbf{u}_i^{(1)})} - 1) \cdot \mathbf{bits}(\mathbf{f})_{ij} + 1 \right) = g^{\mu'_j} \quad \text{for all } j. \quad (2)$$

There are $|\mathbf{bits}(\mathbf{f})|$ factors in total, but only $\approx \sqrt{|\mathbf{bits}(\mathbf{f})|}$ different values these factors can take.

We say (2) is a *low entropy batched grand product*. To prove (2) we use a batched GKR protocol with the following optimization, among others: because of the low-entropy phenomenon, we build several small lookup tables of size $O(\sqrt{|\mathbf{bits}(\mathbf{f})|})$ (for example, of a few MB for $|\mathbf{u}^{(1)}| = 2^{16}$, see Table 11) which are used during several of the most expensive sumcheck rounds of the GKR protocol. Concretely, the last sumcheck (the most expensive one) uses lookup tables to compute the first three messages of the prover; the second last sumcheck does the same for the first two prover messages (with two tables); and the third last sumcheck does the same for the first prover message (with one table).

⁴For example, in our previous example, $\mathbf{u} = \mathbf{v} \otimes (1, 2, \dots, 2^{B-1})$. If $\mathbf{v}$ is an “eq” vector, then $\mathbf{u}$ admits highly customizable tensor decompositions.

A key point: The key is that these square-root sized tables can be used in each of the grand products from (2), producing a large amortization, and allowing us to build several tables for different sumcheck rounds, while keeping them in cache, where table reads are cheaper than field multiplications.

The reason this is possible is that we only seek to prove witness MLE evaluations (or inner products) in the exponent. In this specific setting, we can decompose the initial claim into the subclaims $\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{f})_{*j} \rangle = \mu'_j$. The reason we only need to address this simple setting is that our scheme starts off in a ring R , with its *non-linear* constraints arithmetized over R . Then the constraints are reduced via a PIOP to a *linear* integer claim on the witness, which we then prove “in the exponent”.

Other proof systems using the “proof in the exponent” technique, e.g. [Bin26], aim to prove entire integer constraints in the exponent. Ultimately, this means one has to prove products of the form $y \cdot x$ in the exponent, for x, y two witness elements, while we only prove products $a \cdot x$, for a a value coming from a public vector admitting a square-root sized tensor decomposition. See Section 2.3.1 for more details.

Issue 4. The above solution also takes care of Issue 4: the batched GKR protocol outputs MLE evaluation claims on $(g^{\pi_q^{-1}(\mathbf{u}^{(1)})_i})_i$ (and $\text{bits}(\mathbf{f})$, which is dealt with by the PCS over $\mathbb{F}_{2^\nu}$). This is an $O(\sqrt{|\text{bits}(\mathbf{f})|})$ vector, and the verifier can check this claim by itself cheaply.

1.2.1 Using BitZ to prove constraints with hybrid arithmetics

In our implementation, we use BitZ as a PCS that can simultaneously prove R1CS constraints over integers and binary fields. We explain how next. We stress again that one can use BitZ to build all sorts of proof systems for all sorts of constraints. Nothing forces one to use R1CS-like constraints and integers and binary fields.

Virtual $\mathbb{F}_2$ -linear transforms, NP-complete dually linear relations, and one arithmetization of SHA-256. At the end of the steps of Section 2.1, we have a standard linear claim on $\text{bits}(\mathbf{f})$ in $\mathbb{F}_{2^\nu}$, say

$$\langle \text{bits}(\mathbf{f}), \mathbf{u}' \rangle = \eta \quad \text{in } \mathbb{F}_{2^\nu}. \quad (3)$$

Say we would like to add an extra constraint, over $\mathbb{F}_{2^\nu}$, on $\text{bits}(\mathbf{f})$ of the form $\text{bits}(\mathbf{f}) = M \cdot \mathbf{g}$ for some matrix M and vector $\mathbf{g}$ with entries in $\mathbb{F}_2$. Then, we could instead of committing to $\text{bits}(\mathbf{f})$ in Step 1 of Section 2.1, commit to $\mathbf{g}$. The claim (3) becomes

$$\langle \mathbf{g}, M^\top \cdot \mathbf{u}' \rangle = \eta \quad \text{in } \mathbb{F}_{2^\nu},$$

where $\top$ denotes transposition. This is a linear claim on $\mathbf{g}$, the vector we committed to. We can prove this claim with a standard PCS over $\mathbb{F}_{2^\nu}$ such as [ACFY25].

To keep the verifier succinct, we need to ensure that $M^\top \cdot \mathbf{u}'$ admits a succinct representation. In our applications, the vector $\mathbf{u}'$ admits such (see Step 2 in Section 2.1), and so one needs to ensure M carries some succinct structure, such as for example a tensor decomposition, so that $M^\top \cdot \mathbf{u}'$ does not become a dense vector [Albert: review paragraph](#).

This allows BitZ to prove polynomial constraints over R on witness vectors $\mathbf{f}$ whose bit-decomposition is a virtual $\mathbb{F}_2$ -linear combination of a committed vector $\mathbf{g} \in \mathbb{F}_2^{\ell'}$. For example, BitZ can efficiently

prove the following R1CS-like relation:

$$\mathcal{R}_{\text{R1CS-bits-virtual}}(\mathbb{Z}) := \left\{ \begin{array}{l} \mathbf{x} = (\mathbf{x}, A, B, C, M) \in \mathbb{F}_2^k \times (\mathbb{Z}^{m \times \ell})^3 \times \mathbb{F}_2^{\ell \times (k + \ell')} \\ \mathbf{w} = \mathbf{g} \in \{0, 1\}^{\ell'} \subseteq \mathbb{F}_2^{\ell'} \end{array} \middle| \begin{array}{l} (A \cdot \mathbf{f}) \circ (B \cdot \mathbf{f}) = C \cdot \mathbf{f} \text{ in } \mathbb{Z} \\ \text{bits}(\mathbf{f}) = (M \cdot \mathbf{g}) \text{ in } \mathbb{F}_{2^\nu} \end{array} \right\},$$

A particularly interesting subclass of this relation is that obtained by setting $A = B = \mathbf{0}$, in which case the resulting relation, which we call the **CM relation**, degenerates to linear constraints over $\mathbb{Z}$ and over $\mathbb{F}_2$, i.e. a “dually linear” constraint.

In [Lemma 2.4](#) we show that the CM relation is NP-complete [Albert: Or NP-hard?](#). Our arithmetization of SHA-256 + ECDSA signature verification ([Table 1](#)) is an instance of $\mathcal{R}_{\text{R1CS-bits-virtual}}(\mathbb{Z})$. We arithmetize SHA-256 as an instance of the CM relation, with $4.4\times$ smaller witnesses than Binius64 [[Bin26](#)] and Flock [[BRW26](#)] ($3.6\times$ smaller than the AI-optimized arithmetization), see [Table 2](#). We remark that despite this small arithmetization, when it comes to proving a large amount of classic hash compressions, we observe that binary field native proof systems such as Flock [[BRW26](#)] and Binius64 [[Bin26](#)] perform better (see [Section 7.4](#)). Because of this, in [Section 2.4.1](#) we describe how the BitZ-SNARK can be combined with such a binary-field SNARK, on a common commitment, to prove computations that mix constraints over the integers (or other rings) and over binary fields.

Polynomial constraints in integers and binary fields For computations mixing constraints over the integers (or other rings) with constraints over binary fields, we construct a SNARK that works, intuitively, as follows: the witness of both constraints is committed together over a binary field. Then we run BitZ-SNARK for the integer constraints, and a binary-field SNARK such as Binius64 [[Bin26](#)], both using the committed witness. Both of these reduce the initial constraints to an inner product claim of the witness over a binary field. Now we batch the two resulting inner product claims together and prove them over the binary field using a PCS such as WHIR [[ACFY25](#)] or Ligerito [[NA25](#)]. We provide more details in [Section 2.4.1](#).

We use this simple combination to prove SHA-256 compressions together with multiplications modulo 2^{32} (see [Table 1](#)).

SHA-256 arithmetization type	Witness bit-size per compression
CM	7144 bits
R1CS over $\mathbb{F}_{2^\nu}$	31744 bits
R1CS over $\mathbb{F}_{2^\nu}$, AI optimized	25470 bits

Table 2: Witness bit-size per SHA-256 compression as per two different arithmetizations. The CM row indicates the bit-size when arithmetizing SHA-256 via an instance of the CM relation (see (23)) using the methods from [Eq. \(23\)](#) and [Lemma 2.4](#), see <https://github.com/reilabs/freigen/blob/324935e/Freigen/Circuit/Examples/Sha256.lean>. The second row displays the witness bit-size in the arithmetization from Binius64 [[Bin26](#)], which is an R1CS constraint over a binary field. Flock’s [[BRW26](#)] arithmetization has 31400 bits per compression. The AI row refers to an optimized arithmetization found by an AI working against a Lean specification in the zkGolf competition <https://zk.golf/>.

1.3 Related work

Proof systems over integers and other rings. In [CHA24], Campanelli and Hall-Andersen propose proving polynomial constraints over integers, by projecting them onto a random prime field (sampled by the verifier), a technique which the authors call *fingerprinting*. Subsequent works have adopted this idea [GWV⁺25, SSPP26, AGK⁺26, CXNB26]. [CHA24] also proposes a hidden-order group based PCS suitable for their PIOP, similar to the ones in [BFS19, BHR⁺21]. The PCS is not implemented and so we do not compare against it.

The Zinc [GWV⁺25] work adopts *fingerprinting* and designs a hash-based PCS over the rationals $\mathbb{Q}$, suitable for the fingerprinting approach, providing the first relatively practical proof system over the integers. [GWV⁺25] addresses subtle technical issues that arise when aiming to commit to *integer* vectors with error correcting codes (namely, malicious provers can potentially use rational numbers that are very close to being integers of the target magnitude).

The Zinc+ [AGK⁺26] work extends [GWV⁺25] in two directions: 1) It proposes a constraint system that includes polynomial constraints simultaneously over many types of rings⁵ and fields, and provides a method for constructing PIOP’s for those. This method extends the fingerprinting approach by not only projecting $\mathbb{Z}$ onto a random prime field, but projecting all sorts of rings and fields onto a random prime field. 2) It improves the soundness analysis of the PCS in [GWV⁺25], plus it proposes a new linear code over $\mathbb{Z}$ or $\mathbb{Q}$, called Integer Pseudo Reed-Solomon (IPRS) code, obtained by “lifting” (in a suitable way) an FFT encoding algorithm over a small finite field, to the integers.

In [SSPP26] the authors propose Qaren, a compiler that takes any PCS over a finite field of sufficiently large characteristic and creates a PCS for integer polynomials (or, more precisely, rational polynomials close to the target integer polynomials, see the overview of [GWV⁺25] above). The authors instantiate their compiler to obtain the first transparent and fully succinct PCS over integers/rationals. The prior work [GWV⁺25, AGK⁺26] is not succinct on the bit-size of the integers being committed. No implementation or performance reports are provided in [SSPP26].

The concurrent work Limber [CXNB26] proposes a new compiler that transforms any PCS over fields of sufficiently large characteristic q (in this case, q is $\Omega(\lambda^2 \log^2 N)$, N being the witness size, while in [SSPP26] it is $\Omega(2^\lambda)$) into a PCS for integer polynomials. Additionally Limber compiles into strictly integer PCS, and does not present the issue that a malicious prover could use rationals that are “almost integers”. Limber is instantiated in practice with the Hyrax and the Brakedown PCS. In Section 7 we compare BitZ against Limber.

Compared to all prior work, BitZ proposes a new concrete PCS over rings, and a new paradigm for building such. As already demonstrated in the present work for $\mathbb{Z}$ and Zaratan, BitZ can be combined with the arithmetizations and PIOP’s from Zaratan, Zinc, and Zinc+, and any other scheme for constraints over rings, to build efficient SNARKs over finitely generated rings.

Proof systems over binary fields. Hash-based SNARKs over binary fields were pioneered by the line of work of Ben-Sasson et al. [BSBHR18a, BSBHR18b, BSCG⁺19, BSCKL22], but did not see widespread adoption until the introduction of *packed* polynomial commitment schemes by Diamond and Posen [DP23, DP24]; see also [Bin26]. Such schemes allow one to commit to a bit-valued witness over an extension field without incurring embedding overhead, while still providing

⁵In the terminology of [AGK⁺26], the constraints are over *polynomial rings with ideal membership constraints*, for certain types of ideals (generated by a monic and non-constant polynomial, or zero). However this is equivalent to saying the constraints are over different types of rings (within a certain class)

evaluation proofs for its original “uncompressed” representation.

At the time of writing, two further works push the performance envelope of binary proof systems. Bünz, Rothblum, and Wang [BRW26] combine [Bin26] with the batched RICS strategy inspired by [BSCG⁺19], achieving a striking proving throughput of several hundred thousand hashes per second on an M4 Max CPU. BinarySpartan [Set26], in turn, incorporates the optimizations of [Bin26]-Flock into Spartan, resulting in a proof system with again high hash-proving throughput.

Commitments in binary fields with openings in rings R of characteristic different than 2.

To our knowledge this primitive is studied only in Soukhanov’s note [Sou26] and in the present work. The note [Sou26] proposes a scheme, called *Fields-Witch* for committing over the binary field $\mathbb{F}_{2^{127}}$ and then executing a PCS opening over $\mathbb{F}_{2^{127}-1}$ (a prime field), by running the PCS in the exponent of a generator of $\mathbb{F}_{2^{127}}^*$. The approach, substantially different to ours, is limited to $R = \mathbb{F}_{2^{127}-1}$ (or analogous configurations where $2^n - 1$ is prime) and does not seem to be able to leverage our aforementioned “low-entropy” based optimizations. We discuss this further in Section 2.3.1 and compare BitZ with [Sou26] in Section 7.6.

1.4 Acknowledgements

This work was supported by the World Foundation.

We thank the engineering teams at World Foundation, Nethermind, and Ivy Research for their contributions towards implementing BitZ, concretely we thank Alexander Abdugafarov, Xander van der Goot, and Sina (Ali Parvizi).

We also thank Giacomo Fenzi, Ulrich Haböck, and Lev Soukhanov for insightful and helpful discussions.

Usage and non-usage of AI. *Implementation:* We crucially used AI in our implementation and experiments, in the way explained in Section 7.

Document preparation: This document was written with relatively limited assistance of AI. It was used to: identify typos and incoherences; make refactorings and stylistic improvements; and for cost accounting in Sections 4.3, 4.5 and 6. The soundness proof of the Out Of Domain (OOD) step (Section A.2) was initially drafted with AI, and edited by us subsequently (such proof is a simple adaptation of well-known arguments to our notation and context). The soundness proof in the technical overview Section 2.5 was initially drafted with AI and was elaborated upon by us subsequently.

2 Technical overview

In this section we describe the BitZ PCS, step by step (Sections 2.1 and 2.2), and argue its efficiency (Section 2.3) and soundness (Sections 2.5.1 and 2.5.2) at an intuitive level. Section 2.4 describes how to prove claims on virtual $\mathbb{F}_2$ -linear transforms of the committed vector. In Section 5 (in the appendix) we explain how to build a full SNARK for constraints over (multiple) rings using BitZ as a PCS.

2.1 The prime field case

We start with a simple setting. We let $\mathbb{F}_q$ be a prime field with q elements, and assume the prover claims it knows a vector $\mathbf{f} \in \mathbb{F}_q^{\ell_0}$ such that

$$\langle \mathbf{v}, \mathbf{f} \rangle = \mu \quad \text{in } \mathbb{F}_q, \quad (4)$$

where $\mathbf{v} \in \mathbb{F}_q^{\ell_0}$, $\mu \in \mathbb{F}_q$ are public.

This is a core setting, in that many finite-field based SNARKs require such primitive. However, on several occasions, they require $\mathbb{F}_q$ to be a field extension, instead of just a prime field. In [Section 2.2](#) we treat this important case, as well as others.

Additionally, the present setting is required when using the fingerprinting PIOP approach of [\[CHA24\]](#) for proving integer constraints. In that approach, the integer constraint is projected to a random prime field $\mathbb{F}_q$, and a PIOP is run over $\mathbb{F}_q$, outputting a claim of the form (4).

We next present a protocol (a simple instantiation of BitZ) for proving (4). We assume $\mathbf{v}$ admits a tensor decomposition

$$\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} \in \mathbb{F}_q^{\ell_1 \times \ell_2},$$

where we assume there is freedom to choose the lengths of $\mathbf{v}^{(1)}, \mathbf{v}^{(2)}$ (whose product is ℓ_0). This is typically the case since $\mathbf{v}$ is usually an “eq” vector, i.e. $\mathbf{v} = (\text{eq}(\mathbf{b}, \mathbf{r}))_{\mathbf{b} \in \{0,1\}^{\log \ell_0}}$ for some $\mathbf{r} \in \mathbb{F}_q^{\log \ell_0}$. Tensor decompositions play a crucial role in the efficiency of BitZ , as we discuss in [Section 2.1.1](#).

Step 1: Commitment in $\mathbb{F}_{2^\nu}$. Let $B := \lceil \log q \rceil$ and let $\ell := B \cdot \ell_0$. The prover creates a vector $\text{bits}(\mathbf{f}) \in \{0,1\}^\ell \subseteq \mathbb{Z}^\ell$ of integer bits, obtained by bit-decomposing each entry of $\mathbf{f}$ (a field element) into a vector of B bits, and concatenating all such B -length vectors.

Next, the prover looks at $\text{bits}(\mathbf{f})$ as a vector from $\mathbb{F}_2^\ell$, packs it as a vector from $\mathbb{F}_{2^\nu}^{\ell/\nu}$ (see below), and encodes the packed vector with a Reed-Solomon code $\mathcal{C}$ (or any other code) over $\mathbb{F}_{2^\nu}$, and sends a Merkle root of the resulting codeword to the verifier. To *pack* $\text{bits}(\mathbf{f})$, every consecutive ν bits $b_1, \dots, b_\nu$ of $\text{bits}(\mathbf{f})$ are joined together as a single field element $\sum_{i \in [\nu]} X^{i-1} \cdot b_i$ from $\mathbb{F}_{2^\nu}$ (one can use a different $\mathbb{F}_2$ -base than the monomial base $(X^i)_i$). We denote the encoded word by $f \in \mathbb{F}_{2^\nu}^m$, with m being the length of the code.

From here on, the prover and verifier set out to prove an instance of the following relation, which we call LinBitsRings , and which takes a more general form in [Section 2.2](#):

$$\mathcal{R}_{\text{LinBitsRings}}(\mathbb{F}_q) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{F}_q \times \mathbb{F}_q^{\ell_0}, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}_q^{\ell_0} \end{array} \right) \left| \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})), \\ \langle \mathbf{f}, \mathbf{v} \rangle = \mu \quad \text{in } \mathbb{F}_q \end{array} \right. \right\},$$

where, $\text{Enc}_{\mathcal{C}}$ denotes encoding with the code $\mathcal{C}$; Pack_ν is the packing operation; and π_2 is reduction modulo 2 (so that $\pi_2(\text{bits}(\mathbf{f}))$ means that we look at the bits in $\text{bits}(\mathbf{f})$ as elements from $\mathbb{F}_2$). When proving such relation, the verifier receives $\mathbf{x}$ and oracle access the $\mathbf{y}$, and the prover receives additionally $\mathbf{w}$.

If using a proximity parameter beyond the unique decoding radius, the prover and verifier perform an out of domain sample, see Round 0 of [Construction 4.4](#).

Step 2: Bitification of the linear claim. Observe that

$$\langle \mathbf{v}, \mathbf{f} \rangle = \sum_{i \in [\ell_0]} \sum_{j \in [B]} \mathbf{v}_i \cdot 2^{j-1} \cdot \pi_q(\text{bits}(\mathbf{f})_{ij}) = \langle \mathbf{u}, \pi_q(\text{bits}(\mathbf{f})) \rangle \quad \text{in } \mathbb{F}_q,$$

for $\mathbf{u} := \mathbf{v} \otimes (1, 2, \dots, 2^{B-1})$, where we index the entries of $\text{bits}(\mathbf{f})$ with tuples $(i, j) \in [\ell_0] \times [B]$. Accordingly, the claim that the prover knows $\mathbf{f}$ satisfying (4) is replaced by the claim that it knows a vector of bits $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ satisfying

$$\langle \mathbf{u}, \pi_q(\mathbf{g}) \rangle = \mu \text{ in } \mathbb{F}_q, \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{g}). \quad (5)$$

Abusing the notation, we denote $\mathbf{g}$ by $\text{bits}(\mathbf{f})$ (the bit vector used by the honest prover).

If $\mathbf{v}$ is an “eq vector” or a vector admitting flexible tensor decompositions, then $\mathbf{u}$ can be tensor decomposed as $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)} \in \mathbb{F}_q^{\ell_1 \cdot \ell_2}$ with $\ell_1 \approx \ell_2 \approx \sqrt{\ell}$ (our instantiation takes $\ell_1 \in [\ell^{0.5}, \ell^{0.6}]$, see [Section 4.7](#)).

Step 3 (Optional): Resampling of $\mathbb{F}_q$. For soundness, we need that $|\mathbb{F}_q| < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ (because we will prove a collection of integer linear claims “in the exponent” of a generator of $\mathbb{F}_{2^\nu}^*$). If this condition is not met, we replace $\mathbb{F}_q$ by a smaller prime field via a randomized procedure. For now, we assume the condition holds, and we explain this field resampling step below in [Remark 2.1](#).

Step 4: Transformation of (5) into a batch of low-magnitude integer claims. The prover organizes $\text{bits}(\mathbf{f})$ into a $\ell_1 \times \ell_2$ matrix with columns $\text{bits}(\mathbf{f})_{*j} \in \{0, 1\}^{\ell_1}$ ($j \in [\ell_2]$). Then it publishes $\mu_1, \dots, \mu_{\ell_2} \in \mathbb{Z}$ defined as $\mu_j = \langle \pi^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{f})_{*j} \rangle$ in $\mathbb{Z}$, for all $j \in [\ell_2]$. Then the verifier checks that $\mu_j \in [0..(q-1) \cdot \ell_1]$ for all $j \in [\ell_2]$, and that

$$\sum_{j \in [\ell_2]} \pi(\mu_j) \cdot \mathbf{u}_j^{(2)} = \mu \quad \text{in } \mathbb{F}_q,$$

and (5) is replaced by the claim that the prover knows a vector of bits $\text{bits}(\mathbf{f})$ such that

$$\langle \pi^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{f})_{*j} \rangle = \mu_j \text{ in } \mathbb{Z}, \text{ for all } j \in [\ell_2], \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})). \quad (6)$$

where π^{-1} is the canonical representation of elements from $\mathbb{F}_q$ as integers in the range $[0..q-1]$.

Step 5: Exponentiation in $\mathbb{F}_{2^\nu}$ and grand product in $\mathbb{F}_{2^\nu}$. Since we assumed $|\mathbb{F}_q| < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, and since both sides of (6) lie in $[0..(q-1) \cdot \ell_1]$ (recall that $\text{bits}(\mathbf{f})$ has bit entries, and that the verifier checked the range of each μ_j), (6) is true if and only if

$$\begin{aligned} g^{\mu_j} &= g^{\langle \pi^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{f})_{*j} \rangle} \\ &= g^{\sum_{i \in [\ell_1]} \pi^{-1}(\mathbf{u}_i^{(1)}) \cdot \text{bits}(\mathbf{f})_{ij}} = \prod_{i \in [\ell_1]} g^{\pi^{-1}(\mathbf{u}_i^{(1)}) \cdot \text{bits}(\mathbf{f})_{ij}} \\ &= \prod_{i \in [\ell_1]} \left(\left(g^{\pi^{-1}(\mathbf{u}_i^{(1)})} - 1 \right) \cdot \text{bits}(\mathbf{f})_{ij} + 1 \right) \quad \text{in } \mathbb{F}_{2^\nu} \quad \text{for all } j \in [\ell_2], \end{aligned} \quad (7)$$

where g is a generator of the multiplicative subgroup of $\mathbb{F}_{2^\nu}$, and in the last expression, $\text{bits}(\mathbf{f})_{ij}$ denotes the i, j -th entry (a bit) in $\text{bits}(\mathbf{f})$ seen as an element of $\mathbb{F}_{2^\nu}$.

The prover and verifier run a batched grand product PIOP over the binary field $\mathbb{F}_{2^\nu}$ for the claim (7) (see Section 2.3), obtaining as a result a new claim of the form

$$\langle \mathbf{z}, \text{bits}(\mathbf{f})_{*j} \rangle = c_j \text{ in } \mathbb{F}_{2^\nu} \text{ for all } j \in [\ell_2], \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})), \quad (8)$$

for some $\mathbf{z} \in \mathbb{F}_{2^\nu}^{\ell_1}$ and $c_j \in \mathbb{F}_{2^\nu}$. In general, these claims can be batched to

$$\langle \mathbf{z}', \text{bits}(\mathbf{f}) \rangle = c \text{ in } \mathbb{F}_{2^\nu}, \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})), \quad (9)$$

for $\mathbf{z}' \in \mathbb{F}_{2^\nu}^\ell$, $c \in \mathbb{F}_{2^\nu}$ with, for example, $\mathbf{z}' = (\alpha_1, \dots, \alpha_{\ell_2}) \otimes \mathbf{z}$ for $\alpha_1, \dots, \alpha_{\ell_2}$ verifier's randomness, and $c = \sum_{j \in [\ell_2]} \alpha_j \cdot c_j$. However, in most instantiations this step may not be necessary, especially if using an interleaved code to commit to $\text{bits}(\mathbf{f})$.

This is by far the most expensive step of $\mathbb{B}it\mathbb{Z}$. Our grand product PIOP is a simple GKR protocol for grand products, for which we design novel, to our knowledge, prover optimizations, see Section 2.3 below and Section 6. These optimizations exploit the “low entropy” of the factors $(g^{\pi^{-1}(\mathbf{u}_i^{(1)})} - 1) \cdot \text{bits}(\mathbf{f})_{ij} + 1$ across $(i, j) \in [\ell_1] \times [\ell_2]$, in the sense that across $\ell = \ell_1 \cdot \ell_2$ factors, there are only $\ell_1 + 1 = O(\sqrt{\ell})$ possible values taken by these factors. This, among others, is why it is important to have the vector $\mathbf{u}$ admit a tensor split into $\approx$ square-root sized factors.

Step 6: Proving linear claims on $\text{bits}(\mathbf{f})$ over $\mathbb{F}_{2^\nu}$. At this point, since $\text{bits}(\mathbf{f})$ was committed over $\mathbb{F}_{2^\nu}$ (in packed form), one can prove (8) (or the batched version of it) via standard methods such as in [DP23, DP24, BRW26]. Concretely, we use the so-called ring switch from [DP23, DP24] to turn (8) into linear claims about the packed binary field elements we encoded, and then use WHIR or Ligerito [ACFY25, NA25] over $\mathbb{F}_{2^\nu}$ to prove these claims.

In Section 4.7 we discuss how to efficiently instantiate $\mathbb{B}it\mathbb{Z}$ with $\mathbb{F}_{2^\nu} = \mathbb{F}_{2^{128}}$, and attain 128 bits of security.

Remark 2.1 (Description of Step 3). Assume the condition $|\mathbb{F}_q| < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ does not hold. Then, in Step 3, the prover sends an integer $\mu' \in \mathbb{Z}$ such that

$$\langle \pi^{-1}(\mathbf{u}^{(1)}) \otimes \pi^{-1}(\mathbf{u}^{(2)}), \text{bits}(\mathbf{f}) \rangle = \mu' \quad \text{in } \mathbb{Z},$$

The verifier checks that $\pi(\mu') = \mu$, and responds with a random prime q' from a public set $\mathcal{P}$ of primes, all smaller than $(|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. Then, using the fingerprinting technique [CHA24], the prover and verifier project the above claim to the field $\mathbb{F}_{q'}$, so that it becomes

$$\langle \pi_{q'} \circ \pi^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{q'} \circ \pi^{-1}(\mathbf{u}^{(2)}), \pi_{q'}(\text{bits}(\mathbf{f})) \rangle = \pi_{q'}(\mu') \text{ in } \mathbb{F}_{q'}, \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})). \quad (10)$$

Then the prover and verifier proceed to Step 4 after updating $\mathbb{F}_q \leftarrow \mathbb{F}_{q'}$, $\pi \leftarrow \pi_{q'}$, $\mu \leftarrow \pi_{q'}(\mu')$, and $\mathbf{u} \leftarrow \pi_{q'} \circ \pi^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{q'} \circ \pi^{-1}(\mathbf{u}^{(2)})$, where $\pi_{q'}$ denotes reduction modulo q' .

To ensure soundness of the projection step, the verifier checks that $\mu' \in [0.. \ell \cdot (q - 1)^2]$ (see Section 2.5.2).

Remark 2.2 (Succinctness of the verifier and the proof size). Throughout the above construction, the verifier handles only vectors of size ℓ_1 and ℓ_2 , hence, by setting ℓ_1 and ℓ_2 approximately of size $\sqrt{\ell}$, we manage to keep the verifier and proof size succinct. We leave as open problem (Section 2.6) making these asymptotically polylogarithmic, instead of square root sized.

2.1.1 Why the tensor split

The claim $\langle \mathbf{u}, \pi_q(\text{bits}(\mathbf{f})) \rangle = \mu$ (cf. (5)) over $\mathbb{F}_q$ is split, in Step 4, into ℓ_2 claims over $\mathbb{Z}$ of the form $\langle \pi^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{f})_{*j} \rangle = \mu_j$ for all $j \in [\ell_2]$ (cf. (6)). This pays off in three ways:

- All ℓ_2 grand products in (7) share the same ℓ_1 public values $g^{\pi^{-1}(\mathbf{u}_i^{(1)})}$. In total, there are ℓ factors, but at most $\ell_1 + 1 \approx \sqrt{\ell}$ distinct values. This is crucial for performance when proving the grand products (see Section 2.3).
- The verifier only handles vectors of length $\ell_1 \approx \sqrt{\ell}$ and $\ell_2 \approx \sqrt{\ell}$, instead of vectors of length ℓ , allowing for succinctness.
- Each μ_j belongs to the range $[0..(q-1) \cdot \ell_1]$ instead of $[0..(q-1) \cdot \ell]$. Hence prime fields of size up to $(|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, instead of $(|\mathbb{F}_{2^\nu}| - 1)/\ell$, fit in the exponent of g without overflow. This difference is important in practice when seeking to instantiate BitZ at 128 bits of security while using $\mathbb{F}_{2^{128}}$, and not paying too much on grinding, cf. Section 4.7.

Table 5 in the appendix records the concrete costs of this protocol, stated for the general setting of Section 2.2

2.2 Generalization to pairs of arbitrary finitely generated rings

Let S be a finitely generated ring. For example, S could be a finite field extension, or the ring of integers $\mathbb{Z}$, or a cyclotomic ring.

That S is finitely generated means that there exist elements $1, \eta_1, \dots, \eta_k \in S$ such that every element $s \in S$ can be written as a polynomial expression $s = \sum_{\mathbf{a}} s_{\mathbf{a}} \cdot \eta_1^{a_1} \cdots \eta_k^{a_k}$ in $\eta_1, \dots, \eta_k$, for some elements $s_{\mathbf{a}} \in [0.. \text{char}(S) - 1]_S$ (for some integers $s_{\mathbf{a}}$ if $\text{char}(S) = 0$), where $\text{char}(S)$ is the characteristic of S ⁶. For example, if S is a field extension of size q^e with $e > 1$, then $k_S = 1$ and $\eta_1 = X$ (where we look at elements from the extension field as polynomials). If S is a prime field or the integers $\mathbb{Z}$, then $k_S = 0$ and the only generator is 1.

Let R be another finitely generated ring, and let

$$\psi : S \rightarrow R$$

be a ring homomorphism. For example, we could simply have $S = R$ be a finite field, and ψ be the identity (this is the setting in the previous Section 2.1). Or, in the fingerprinting approach to proving integer constraints, one would take $S = \mathbb{Z}$, R a prime field, and ψ reduction modulo q . In [AGK⁺26], S is a polynomial ring, R is a prime field, and ψ is a certain projection.

Assume now that the prover claims knowledge of a vector $\mathbf{f} \in S^{\ell_0}$ such that

$$\langle \mathbf{v}, \psi(\mathbf{f}) \rangle = \mu \quad \text{in } R, \tag{11}$$

where $\mathbf{v} \in R^{\ell_0}$, $\mu \in R$ are public.

Size bounds. For soundness reasons⁷, when S is infinite, we cannot allow a prover to use arbitrarily large witnesses, and we restrict the set of allowable witnesses to a publicly specified subset of S (this is standard practice, see e.g. [CHA24, GWV⁺25, SSPP26, CXNB26, AGK⁺26]).

⁶The characteristic of a ring with multiplicative unity 1 is the minimum n such that 1 added together n times equals 0; or $\text{char}(S) = 0$ if such n does not exist.

⁷There are also formalization difficulties when allowing arbitrary witness elements in an infinite ring. For example, no polynomial time extractor can output an exponentially sized witness.

Let $B, D > 0$ be public size parameters. Intuitively, we want the prover to prove knowledge of a witness that can be written as polynomial expressions on $\eta_1, \dots, \eta_{k_S}$ of total degree less than D , with coefficients in $[0..2^B - 1]_S$, where $[0..2^B - 1]_S$ is the set of elements from S that can be obtained by adding 1 together less than 2^B times (and zero). We denote the set of such elements by $S_{\leq B}^{< D}$. Formally,

$$S_{\leq B}^{< D} := \left\{ \sum_{\substack{\mathbf{a} \in \mathbb{Z}_{\geq 0}^k, \\ a_1 + \dots + a_k < D}} s_{\mathbf{a}} \cdot \eta_1^{a_1} \cdots \eta_k^{a_k} \mid s_{\mathbf{a}} \in [0..2^B - 1]_S \right\}$$

If S is finite to begin with, we can simply take B, D so that $S_{\leq B}^{< D} = S$, or we may prefer to restrict witnesses to a subset of S anyway.

Step 1: Commitment in $\mathbb{F}_{2^\nu}$. As before, the prover creates a vector $\text{bits}(\mathbf{f}) \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ of integer bits, for some ℓ , with $\text{bits}(\mathbf{f})$ being linearly related to $\mathbf{f}$ in a manner we explain next. Given $s \in S_{\leq B}^{< D}$ with $s = \sum_{\mathbf{a}} s_{\mathbf{a}} \cdot \eta_1^{a_1} \cdots \eta_k^{a_k}$, where the sum runs over the set I of tuples $\mathbf{a} \in \mathbb{Z}_{\geq 0}^k$ with $a_1 + \dots + a_k < D$, write

$$s = \sum_{\mathbf{a} \in I} \sum_{i \in [B]} s_{\mathbf{a}, i} \cdot 2^{i-1} \cdot \eta_1^{a_1} \cdots \eta_k^{a_k}, \quad \text{for } s_{\mathbf{a}, i} \in \{0, 1\}.$$

Then define $\text{bits}(s) = (s_{\mathbf{a}, i})_{i \in [B], \mathbf{a} \in I}$, where, abusing the notation, we look at 0 and 1 indistinctly as integers. Then $\text{bits}(\mathbf{f})$ consists in applying bits entry-wise to $\mathbf{f}$. Note that

$$\mathbf{f} = L \cdot \text{bits}(\mathbf{f}) \tag{12}$$

for certain matrix $L \in S^{\ell_0 \times \ell}$. Concretely, $L = \text{Id}_{\ell_0} \otimes (1, 2, \dots, 2^{B-1}) \otimes (\eta_1^{a_1} \cdots \eta_k^{a_k})_{\mathbf{a} \in I}$. Note that by definition of $S_{\leq B}^{< D}$ and L ,

$$L \cdot \mathbf{g} \in S_{\leq B}^{< D} \quad \text{for all } \mathbf{g} \in \{0, 1\}^{\ell_0}. \tag{13}$$

This is required to guarantee that extractors output vectors with entries in $S_{\leq B}^{< D}$.

We remark that this is not the only way one can transform $\mathbf{f}$ into a vector of bits. Especially if $\mathbf{f}$ contains elements with heterogeneous bit-sizes, a different bit-representation strategy is preferable, to avoid $\text{bits}(\mathbf{f})$ being unnecessarily large, see [Remark 4.11](#) for more comments on this bitification step. In any case, one always needs to make sure that a closure property such as (13) is satisfied.

Next, the prover commits to $\text{bits}(\mathbf{f})$ in packed form exactly as in Step 1 of [Section 2.1](#). From here on, the prover and verifier set out to prove an instance of the following relation, which we call [LinBitsRings](#), and which takes a more general form in [Section 4.5](#):

$$\mathcal{R}_{\text{LinBitsRings}}(S, R, \psi) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in R \times R^{\ell_0}, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in (S_{\leq B}^{< D})^{\ell_0} \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})), \\ \langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu \quad \text{in } R \end{array} \right\}.$$

Remark 2.3 (Integrated range checks). Note that $\mathcal{R}_{\text{LinBitsRings}}(S, R, \psi)$ includes, in its definition, a restriction of $\mathbf{f}$ into a finite subset of S , namely $S_{\leq B}^{\leq D}$. This means that a proof for $\mathcal{R}_{\text{LinBitsRings}}(S, R, \psi)$ ensures that $\mathbf{f}$ belongs to such a subset. Thus the $\mathbb{B}\text{it}\mathbb{Z}$ scheme described below also proves such membership constraint. When S is, say, a finite field or the integers, membership to $S_{\leq B}^{\leq D}$ is simply a range check (the $< D$ is vacuous in this case).

Moreover, it is possible to add other heterogeneous subset membership checks by using a non-standard bitification matrix, see [Remark 4.11](#). One, however, needs to make sure that L admits sufficient tensor structure, for efficiency purposes.

The cost of these membership checks comes for free once one is willing to commit separately to the bits comprising the witness. See [Section 2.5.2](#) for an explanation of how the extractor outputs witnesses satisfying the membership check.

Step 2: Bitification of the linear claim. Similarly as in Step 2 of [Section 2.1](#), the claim (11) is replaced by a claim of knowledge of a bit vector $\text{bits}(\mathbf{f})$ such that

$$\langle \mathbf{u}, \psi(\text{bits}(\mathbf{f})) \rangle = \mu \text{ in } R, \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})). \quad (14)$$

for a new $\mathbf{u} \in R^\ell$. This transformation is done using the linear expression from (12). Concretely, we have

$$\begin{aligned} \langle \mathbf{v}, \psi(\mathbf{f}) \rangle &= \langle \mathbf{v}, \psi(L \cdot \text{bits}(\mathbf{f})) \rangle = \langle \psi(L)^\top \cdot \mathbf{v}, \psi(\text{bits}(\mathbf{f})) \rangle = \langle \mathbf{u}, \psi(\text{bits}(\mathbf{f})) \rangle, \\ \mathbf{u} &:= \psi(L)^\top \cdot \mathbf{v}. \end{aligned}$$

As before, it is important that L is such that the new vector $\mathbf{u}$ still admits a tensor decomposition $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ with, say, $\mathbf{u}^{(1)} \in R^{\ell_1}$ and $\mathbf{u}^{(2)} \in R^{\ell_2}$ and $\ell_1 \approx \sqrt{\ell}$, $\ell_2 \approx \sqrt{\ell}$. This is the case for the example L given above. For such matrix L we have

$$\mathbf{u} = \mathbf{v} \otimes (1, 2, \dots, 2^{B-1}) \otimes (\psi(\eta_1^{a_1} \cdots \eta_k^{a_k}))_{\mathbf{a} \in I},$$

which admits flexible tensor decompositions if $\mathbf{v}$ does.

Step 3 (Optional): Replacing R by a random “small” field $\mathbb{F}_q$. Our next goal is to lift the claim (14), i.e. $\langle \mathbf{u}, \psi(\text{bits}(\mathbf{f})) \rangle = \mu$ over R , to a claim “in the exponent” of a generator g of $\mathbb{F}_{2^\nu}^*$. This time, however, there is a priori no natural lifting procedure, since, in intuitive terms, the exponent of g can only accommodate integers or integers modulo $|\mathbb{F}_{2^\nu}| - 1$ (the order of g), while $\langle \mathbf{u}, \psi(\text{bits}(\mathbf{f})) \rangle$ is a sum of elements from an arbitrary ring R .

Because of this, the prover and verifier undergo a randomization step where the ring R is replaced by a random “small” prime field $\mathbb{F}_q$. This step can be skipped if:

- $R = \mathbb{F}_q$ is a prime field with $q < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, i.e. R is a “small” field,
 - $R = \mathbb{Z}$ and all entries of $\mathbf{u}^{(1)}$ are nonnegative and smaller than $(|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ (cf. [Remark 4.5](#)),
- since in that case there is already a natural and sound lift of the claim $\langle \mathbf{u}, \psi(\text{bits}(\mathbf{f})) \rangle = \mu$ to the exponent of g . The bound $(|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ ensures that the integer claims of Step 4 of [Section 2.1](#) cannot overflow $|\mathbb{F}_{2^\nu}| - 1$, even if the claim is false (see [Section 2.5.1](#)).

The step proceeds as follows: suppose R is generated by $1, \gamma_1, \dots, \gamma_{k'}$. The prover computes and sends $\mu' \in \mathbb{Z}[X_1, \dots, X_{k'}]$ (the ring of polynomials with integer coefficients and on variables $X_1, \dots, X_{k'}$) such that

$$\langle \pi_{\text{canon}}^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{u}^{(2)}), \text{bits}(\mathbf{f}) \rangle = \mu' \quad \text{in } \mathbb{Z}[X_1, \dots, X_{k'}]. \quad (15)$$

where $\pi_{\text{canon}} : \mathbb{Z}[X_1, \dots, X_{k'}] \rightarrow R$ is a canonical ring homomorphism sending X_i to γ_i for all $i \in [k']$, and sending $1 \in \mathbb{Z}$ to the multiplicative identity of R (also denoted 1). The inverse π_{canon}^{-1} is a natural inverse of π_{canon} (there are several ways to choose it), see [Example 4.2](#).

The verifier checks that μ' has total degree at most $D_{\mathbb{Z}}$ and coefficients of absolute value at most $2^{B_{\mathbb{Z}}}$, where $D_{\mathbb{Z}}$ and $2^{B_{\mathbb{Z}}}$ are the largest total degree and the largest absolute value of a coefficient attainable by the left-hand side of (15), among all bit vectors $\text{bits}(\mathbf{f})$ (see [Section 2.5.2](#), and Round 1 of [Construction 4.4](#) for concrete expressions). Then the verifier checks that $\pi_{\text{canon}}(\mu') = \mu$.

Now the idea is that the claim (15) implies the claim (14), since $\pi_{\text{canon}}(\mu') = \mu$. Hence, the prover and the verifier proceed to certify (15). The constraint (15) is a polynomial constraint (in fact, a linear constraint) over the ring $\mathbb{Z}[X_1, \dots, X_{k'}]$. We now generalize the fingerprinting technique for polynomial rings from Zinc+ [\[AGK⁺26\]](#) (which in turn generalizes that of [\[CHA24\]](#)), so as to project (15) to a random “small” prime field.

Concretely, the verifier samples a random prime q and a random morphism $\pi_{\text{rand}} : \mathbb{Z}[X_1, \dots, X_{k'}] \rightarrow \mathbb{F}_q$. The protocol specifies a public set $\mathcal{P}$ of samplable primes, and for each $q \in \mathcal{P}$ a public set of samplable morphisms $\mathcal{M}_q$. We require that all samplable primes q satisfy $q < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. In our instantiation we let $\mathcal{M}_q$ be the set of homomorphisms of the form $\pi(h(X_1, \dots, X_{k'})) = \pi_q(h)(\alpha_1, \dots, \alpha_{k'})$.

Now, the claim (15) is replaced by the claim

$$\begin{aligned} \langle \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{u}^{(2)}), \pi_{\text{rand}}(\text{bits}(\mathbf{f})) \rangle &= \pi_{\text{rand}}(\mu') \text{ in } \mathbb{F}_q, \\ f &= \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})). \end{aligned} \quad (16)$$

(Note that since $\text{bits}(\mathbf{f})$ is full of bits, $\pi_{\text{rand}}(\text{bits}(\mathbf{f}))$ is simply an interpretation of these bits as the elements 0, 1 of $\mathbb{F}_q$).

Now the prover and verifier set out to prove (16). Recall that the prover has also sent a commitment to $\text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f}))$. Hence we are exactly in the same setting as in Step 4 of [Section 2.1](#). Accordingly, at this point, the prover and verifier apply Steps 4, 5 and 6 of [Section 2.1](#), taking

$$\begin{aligned} \mathbf{u}^{(1)} &\leftarrow \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{u}^{(1)}) \in \mathbb{F}_q^{\ell_1}, \\ \mathbf{u}^{(2)} &\leftarrow \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{u}^{(2)}) \in \mathbb{F}_q^{\ell_2}, \\ \mu &\leftarrow \pi_{\text{rand}}(\mu') \in \mathbb{F}_q. \end{aligned}$$

2.3 GKR for low-entropy batched grand products

In Step 5 of $\mathbb{B}\text{it}\mathbb{Z}$ ([Section 2.1](#)) the prover must prove that

$$g^{\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{f})_{*j} \rangle} = \prod_{i \in [\ell_1]} \left((g^{\pi_q^{-1}(\mathbf{u}_i^{(1)})} - 1) \cdot \text{bits}(\mathbf{f})_{ij} + 1 \right) = g^{\mu_j} \quad \text{in } \mathbb{F}_{2^\nu} \quad \text{for all } j \in [\ell_2]. \quad (17)$$

As we mentioned, this is by far the most expensive step in $\mathbb{B}\text{it}\mathbb{Z}$. We next explain how we prove (17) in an efficient manner. We make the crucial assumption that both ℓ_1 and ℓ_2 are $O(\sqrt{\ell})$ (see [Section 4.7](#) for exact parameter instantiations). This assumption is relatively simple to attain, as we have discussed throughout [Sections 1](#) and [2](#). At a protocol level, we simply prove (17) with a

standard GKR protocol for grand products [Tha13]. Concretely, let $\ell_1 = 2^{r_1}$ and $\ell_2 = 2^{r_2}$. We set up multilinear polynomials $L_0, L_1, \dots, L_{r_1}$ on $r_2 + i$ variables for $i \in [0..r_1]$, respectively, such that

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = \left(g^{\pi^{-1}(\mathbf{u}_{\text{int}(\mathbf{b})}^{(1)})} - 1 \right) \cdot \text{bits}(\mathbf{f})_{\text{int}(\mathbf{b}), \text{int}(\mathbf{c})} + 1 \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2},$$

and, for $i = 0, \dots, r_1 - 1$

$$L_i(\mathbf{b}, \mathbf{c}) = L_{i+1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{i+1}(\mathbf{b}, 1, \mathbf{c}) \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^i \times \{0, 1\}^{r_2}. \quad (18)$$

We then use a series of sumchecks to certify Eq. (18) for all $i = 0, \dots, r_1 - 1$, and $L_0(\mathbf{c}) = g^{\mu_{\text{int}(\mathbf{c})}}$ for all $\mathbf{c} \in \{0, 1\}^{r_2}$.

We emphasize again that, at a protocol level, this GKR scheme is completely standard. It is described in full detail in Section 6.

At the end of the GKR grand product, the prover and verifier are left with MLE evaluation claims of the form:

$$\widetilde{g^{\pi_q^{-1}(\mathbf{u}^{(1)})}}(\mathbf{r}) = a, \quad \widetilde{\text{bits}(\mathbf{f})_{*j}}(\mathbf{r}) = a_j \quad \text{for all } j \in [\ell_2] \quad (19)$$

for some $\mathbf{r} \in \mathbb{F}_{2^\nu}^{r_1}$, where $\widetilde{}$ denotes MLE, and $g^{\pi_q^{-1}(\mathbf{u}^{(1)})}$ is the vector with entries $g^{\pi_q^{-1}(\mathbf{u}_i^{(1)})}$.

Since $\mathbf{u}$ is public, the verifier checks the first claim of (19) on its own in time $O(\sqrt{\ell})$, leveraging its tensor decomposition. This is one among several reasons why the tensor split of $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ and splitting the original claim into a batch of ℓ_2 claims pay off (see Section 2.1.1).

We design novel (to our knowledge [Albert: check](#)) prover optimizations for the GKR grand product (without altering the protocol itself), see Section 6 for details. Namely, we exploit the following property, which is enabled, again, by the tensor split of $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ and splitting the original claim into a batch of ℓ_2 claims:

Low entropy grand products: There are at most $\ell_1 = O(\sqrt{\ell})$ values that $L_{r_1}(\mathbf{b}, \mathbf{c})$ can take on the hypercube of size $\ell_1 \cdot \ell_2 = \ell$.

This property trickles down to different stages of the GKR protocol. For example, the above property implies that $L_{r_1-1}(\mathbf{b}', \mathbf{c})$ as well can take only an $O(\sqrt{\ell})$ amount of values (with worse constants now) across all $(\mathbf{b}', \mathbf{c}) \in \{0, 1\}^{r_1-1} \times \{0, 1\}^{r_2}$. Similarly, the coefficients of the first messages sent by the prover at the last stages of the GKR protocol (which involve the most expensive sumchecks), also benefit from a low entropy property.

This allows the prover to compute small lookup tables that are kept in cache and reused across all values of $\mathbf{c} \in [\ell_2]$. We build different lookup tables, which used at different stages of the GKR protocol. For example, in the last sumcheck of GKR (the most expensive one, involving $r_1 - 1 + r_2$ variables) the prover uses 4 lookup tables to compute its first message, 5 more tables to compute its second message, and one last table to compute its third message (see Table 11). Similarly, we build lookup tables for the first rounds of the second and third last sumchecks of the GKR protocol. This configuration was found through an optimization process that was run with AI.

2.3.1 Efficiency and comparison to other “prove-in-the-exponent” usages

We next discuss other existing approaches to proving foreign field statements in the exponent of a generator of $\mathbb{F}_{2^\nu}^*$ (the multiplicative subgroup of a binary field), in the context of hash-based SNARKs. Notice that elliptic curve-based PCS’s usually perform commitments and opening proofs in the exponent of a generator of a large elliptic curve subgroup.

Binius64 For example, [Bin26] multiplies pairs of 64-bit integer witness elements in the exponent of g . This is substantially different from our setting where, when binary fields come into play, we are only interested in proving an integer claim about a linear combination of elements of the form “(public values) · (witness elements)”, i.e. $\langle \mathbf{v}, \mathbf{w} \rangle = \mu$ in $\mathbb{Z}$, with the public values $\mathbf{v}$ admitting a tensor split $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}$ (also BitZ can prove multiplication of all sorts of ring elements, not just 64-bit integers). This simpler setting is what allows us to use the low entropy optimizations designed in this work. Crucially, **the reason why we only need to address this setting is because we start off with constraints and a PIOP over integers/fields/rings (Steps 2 to 4 inclusive), and only ever move to work on binary fields when we need to prove an MLE evaluation of the witness (a linear claim) over a “small” prime field (Step 5).**

[Bin26] on the other hand multiplies 64-bit integer witness in the exponent directly. More precisely, [Bin26] proves the identity $g^{\mathbf{w}_{1i} \cdot \mathbf{w}_{2i}} = g^{2^{64} \cdot \mathbf{w}_{3i}} \cdot g^{\mathbf{w}_{4i}}$ for $\mathbf{w}_1, \dots, \mathbf{w}_4$ vectors of 64-bit integers (together with a small additional constraint to address possible overflow). This is done through a combination of LogUp* [Sou25], a GKR grand product (for relatively small products of 128 factors), and the Frobenius automorphism trick. It does not seem possible to follow a similar optimization strategy to ours.

We emphasize again that BitZ supports proving operations of all kinds of ring elements (small and large integers, field elements, lattice ring elements, extension field elements), as opposed to [Bin26] where multiplication is limited to 64-bit integers.

In Section 7 we compare the performance of BitZ and Binius64 when multiplying integers.

The note [Sou26]. In the note [Sou26], Soukhanov proposes a scheme for committing to a vector of bits $\mathbf{w}$ over the binary field $\mathbb{F}_{2^{127}}$, and proving MLE evaluation claims over the prime field $\mathbb{F}_{2^{127}-1}$, for a vector of elements from $\mathbb{F}_{2^{127}-1}$ obtained by packing bits from $\mathbf{w}$ together. To do so, at a high level, the MLE evaluation claims are proved “in the exponent” in the binary field $\mathbb{F}_{2^{127}}$, leveraging that the multiplicative subgroup of $\mathbb{F}_{2^{127}}$ has prime order $|\mathbb{F}_{2^{127}}^*| = 2^{127} - 1$. The proposed approach is substantially different from ours: [Sou26] essentially takes a polylogarithmic PCS over the prime field $\mathbb{F}_{2^{127}-1}$, such as Basefold [ZCF23], WHIR [ACFY25], etc., and executes it “in the exponent of $\mathbb{F}_{2^{127}}$ ”. To do so, it relies on the lookup argument LogUp* [Sou25], grand product arguments (for small products), the Frobenius trick, and additional commitments, for each of the $\log \ell$ rounds.

In its execution, [Sou26] and Binius64 [Bin26] seem to us to be more similar than any of these two to BitZ. Indeed, they both use, mostly, the same collection of subprotocols, in a similar manner. In Table 20 we compare Section 7.6 with BitZ. In Section 7 we perform extensive comparisons between BitZ and Binius64.

Conceptually, [Sou26] shares with BitZ the aim of committing to bits in a binary field, while proving MLE claims of those bits in some other structure R . [Sou26] is limited to $R = \mathbb{F}_{2^{127}-1}$, while BitZ can handle arbitrary finitely generated rings. Even with this greater generality capacity, we observe in Section 7.6 that BitZ performs better than Fields-witch on the latter’s native operation, i.e. proving MLE openings over the prime field $\mathbb{F}_{2^{127}-1}$.

2.4 Virtual $\mathbb{F}_2$ -linear transforms, NP-complete multi-domain linear relations, and hybrid proof systems

Observe that, at the end of the day, Step 5 of [Section 2.1](#) outputs a linear claim of the form⁸

$$\langle \mathbf{z}, \text{bits}(\mathbf{w}) \rangle = c \quad \text{in } \mathbb{F}_{2^\nu}, \quad (20)$$

for some $\mathbf{z} \in \mathbb{F}_{2^\nu}^\ell$ and $c \in \mathbb{F}_{2^\nu}$. This is a standard linear claim over $\mathbb{F}_{2^\nu}$ for the vector $\text{bits}(\mathbf{w})$ which we committed to previously (in this subsection we denote the witness by $\mathbf{w}$, to free the letter $\mathbf{f}$).

Now, say that we know that $\text{bits}(\mathbf{w})$ is a linear transformation of a vector $\mathbf{f} \in \mathbb{F}_2^{\ell'} \subseteq \mathbb{F}_{2^\nu}^{\ell'}$, so that

$$\text{bits}(\mathbf{w}) = M \cdot \mathbf{f} \quad \text{in } \mathbb{F}_{2^\nu},$$

for some matrix $M \in \mathbb{F}_2^{\ell \times \ell'}$. Then

$$\langle \mathbf{z}, \text{bits}(\mathbf{w}) \rangle = \langle M^\top \cdot \mathbf{z}, \mathbf{f} \rangle = c \quad \text{in } \mathbb{F}_{2^\nu}. \quad (21)$$

Hence, in this scenario, one can commit to $\mathbf{f}$ instead of $\text{bits}(\mathbf{w})$ (again, in packed form if necessary), replace the claim (20) by (21), and prove the latter in Step 6 of [Section 2.1](#).

Thus, $\mathbb{B}\text{it}\mathbb{Z}$ can be expanded so as to prove claims where we are given a public vector $\mathbf{x} = \mathbf{x} \in R^k$, and the prover claims it knows $\mathbf{w} = \mathbf{f} \in \mathbb{F}_2^{\ell'}$ such that

$$\begin{cases} \mathbf{p}_i(\mathbf{x}, \mathbf{w}) = 0 & \text{in } R & \text{for all } i \in [m], \\ \text{bits}(\mathbf{w}) = \pi_2^{-1}(M \cdot \mathbf{f}) \end{cases} \quad (22)$$

where $\mathbf{p} = (\mathbf{p}_i)_{i \in [m]}$ is a collection of polynomials over R ; and π_2 is reduction modulo 2.

A particularly interesting instantiation of this relation is the following R1CS-like relation over the integers, where $\mathbf{w}$ is a vector of bits (hence $\mathbf{w} = \text{bits}(\mathbf{w})$) and $\mathbf{w} = \pi_2^{-1}(M \cdot \mathbf{f})$:

$$\begin{aligned} \mathcal{R}_{\text{R1CS-bits-virtual}}(\mathbb{Z}) \\ := \left\{ \begin{array}{l} \mathbf{x} = (\mathbf{x}, A, B, C, M) \in \mathbb{F}_2^k \times (\mathbb{Z}^{m \times \ell})^3 \times \mathbb{F}_2^{\ell \times (k + \ell')} \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^{\ell'} \subseteq \mathbb{F}_2^{\ell'} \end{array} \middle| \begin{array}{l} (A \cdot \mathbf{w}) \circ (B \cdot \mathbf{w}) = C \cdot \mathbf{w} \text{ in } \mathbb{Z} \\ \mathbf{w} = \pi_2^{-1}(M \cdot (\mathbf{x}, \mathbf{f})) \end{array} \right\} \end{aligned}$$

An interesting subset of $\mathcal{R}_{\text{R1CS-bits-virtual}}(\mathbb{Z})$ is that where A and B are zero matrices. We call the resulting relation the CM relation because of the matrices C and M :

$$\mathcal{R}_{\text{CM}}(\mathbb{Z}) := \left\{ \begin{array}{l} \mathbf{x} = (\mathbf{x}, C, M) \in \mathbb{F}_2^k \times \mathbb{Z}^{m \times \ell} \times \mathbb{F}_2^{\ell \times (k + \ell')} \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^{\ell'} \subseteq \mathbb{F}_2^{\ell'} \end{array} \middle| \begin{array}{l} C \cdot \mathbf{w} = 0 \text{ in } \mathbb{Z} \\ \mathbf{w} = \pi_2^{-1}(M \cdot (\mathbf{x}, \mathbf{f})) \end{array} \right\} \quad (23)$$

In a sense, the CM relation places only linear constraints over its witness (with some constraints happening in $\mathbb{F}_{2^\nu}$ and others in $\mathbb{Z}$). Yet, the CM relation is NP-complete, as shown below in [Lemma 2.4](#).

Indeed, **we arithmetize SHA-256 as a CM instance**. Our arithmetization leads to witnesses of 7144 bits per compression, $4.4\times$ smaller than those of Binius64 [\[Bin26\]](#) and Flock [\[BRW26\]](#) (see [Table 2](#)).

⁸We start with the batched version of the claim, but the ideas in this section can be adapted to (8) as well.

Lemma 2.4 (The $\mathcal{R}_{\text{CM}}$ relation is NP-complete). *Any Boolean circuit can be arithmetized as an instance of $\mathcal{R}_{\text{CM}}$.*

Proof. Indeed, since any boolean circuit can be written as a system of equations using the operations XOR and AND, it suffices to prove that, given $x, y, z \in \{0, 1\}$, we can express both of the following constraints as an instance of $\mathcal{R}_{\text{CM}}$:

$$x \text{ XOR } y = z, \quad x \text{ AND } y = z.$$

Indeed, $x \text{ XOR } y = z$ if and only if $\pi_2^{-1}(\pi_2(x) + \pi_2(y)) = z$, where here $+$ means addition modulo 2. Further, $x \text{ AND } y = z$ if and only if

$$2 \cdot z = x + y - \pi_2^{-1}(\pi_2(x) + \pi_2(y)).$$

One can verify this last equivalence for example by directly inspecting that it holds for the four possible combinations of $x, y \in \{0, 1\}$. $\square$

2.4.1 BitZ for fully hybrid constraints over $\mathbb{Z}$ and $\mathbb{F}_{2^\nu}$.

Say one wants to prove a computation that involves both $\mathbb{F}_2$ operations and other more integer-like operations, like integer multiplication or multiplication mod 2^{32} . This is the case, for example, with Virtual Machines running RISC-V instructions. To prove these computations we use an arithmetization with constraints both in $\mathbb{Z}$ and in $\mathbb{F}_2$. We build a SNARK for such constraints as follows (and we use it in our benchmarks for proving SHA-256 compressions and multiplication modulo 2^{32}). Consider a witness $(\mathbf{f}_{\mathbb{Z}}, \mathbf{f}_{\mathbb{F}_2})$ split into an integer vector $\mathbf{f}_{\mathbb{Z}} \in \mathbb{Z}^{\ell_{\mathbb{Z}}}$ used in the integer constraints, and an $\mathbb{F}_2$ vector $\mathbf{f}_{\mathbb{F}_2} \in \mathbb{F}_2^{\ell_{\mathbb{F}_2}}$ used in the $\mathbb{F}_2$ constraints. We can also assume that the $\mathbb{F}_2$ constraints use some entries from $\mathbf{f}_{\mathbb{Z}}$ which are integer bits (looking at those integer bits as elements from $\mathbb{F}_2$), otherwise we would have two unrelated constraints. Accordingly we have $\mathbf{f}_{\mathbb{Z}} = (\mathbf{f}_{\mathbb{Z}}^{(1)}, \mathbf{f}_{\mathbb{Z}}^{(2)})$, where $\mathbf{f}_{\mathbb{Z}}^{(1)}$ is a vector of arbitrary integers, and $\mathbf{f}_{\mathbb{Z}}^{(2)}$ is a vector of bits.

We want to prove that $(\mathbf{x}_{\mathbb{Z}}, \mathbf{f}_{\mathbb{Z}}) \in \mathcal{R}_{\mathbb{Z}}$ and $(\mathbf{x}_{\mathbb{F}_2}, \pi_2(\mathbf{f}_{\mathbb{Z}}^{(2)}), \mathbf{f}_{\mathbb{F}_2}) \in \mathcal{R}_{\mathbb{F}_2}$, where $\mathcal{R}_{\mathbb{Z}}$ and $\mathcal{R}_{\mathbb{F}_2}$ are relations over $\mathbb{Z}$ and $\mathbb{F}_2$, respectively, and where $\pi_2(\mathbf{f}_{\mathbb{Z}}^{(2)})$ denotes the vector of bits $\mathbf{f}_{\mathbb{Z}}^{(2)}$ seen in $\mathbb{F}_2$. The overall proof system proceeds as follows:

1. Commit to the concatenation of $\text{bits}(\mathbf{f}_{\mathbb{Z}}) = (\text{bits}(\mathbf{f}_{\mathbb{Z}}^{(1)}, \mathbf{f}_{\mathbb{Z}}^{(2)}))$ and $\mathbf{f}_{\mathbb{F}_2}$ over a binary field $\mathbb{F}_{2^\nu}$ (in packed form).
2. Partially run the previously described BitZ-SNARK for the statement $(\mathbf{x}_{\mathbb{Z}}, \mathbf{f}_{\mathbb{Z}}) \in \mathcal{R}_{\mathbb{Z}}$. Run it until we have an inner product claim $\langle \mathbf{w}, \text{bits}(\mathbf{f}_{\mathbb{Z}}) \rangle = \eta$ over $\mathbb{F}_{2^\nu}$.
3. Then partially run any SNARK for binary fields such as [Bin26], for the claim $(\mathbf{x}_{\mathbb{F}_2}, \pi_2(\mathbf{f}_{\mathbb{Z}}^{(2)}), \mathbf{f}_{\mathbb{F}_2}) \in \mathcal{R}_{\mathbb{F}_2}$. Run it until we have an inner product claim $\langle \mathbf{w}', (\pi_2(\mathbf{f}_{\mathbb{Z}}^{(2)}), \mathbf{f}_{\mathbb{F}_2}) \rangle = \eta'$ over $\mathbb{F}_{2^\nu}$.
4. At this point we have two inner product claims over $\mathbb{F}_{2^\nu}$ for $\pi_2(\text{bits}(\mathbf{f}_{\mathbb{Z}}))$ and $(\pi_2(\mathbf{f}_{\mathbb{Z}}^{(2)}), \mathbf{f}_{\mathbb{F}_2})$, two vectors that we committed to. Now we prove both in a batched manner using WHIR [ACFY25] or any other similar PCS over $\mathbb{F}_{2^\nu}$.

If necessary the inner product claims can be reduced to MLE evaluation claims before, via a batched sumcheck.

We demonstrate the practicality of this approach by proving a large amount of chained SHA-256 compressions, together with a large amount of multiplications modulo 2^{32} . The results can be found in Table 1 and in Section 7.4.

2.5 Round-by-round knowledge soundness

We provide an overview of why the constructions from [Sections 2.1](#) and [2.2](#) are round-by-round knowledge sound.

2.5.1 Soundness of [Section 2.1](#)

In [Theorem 4.7](#) of [Section 4.4](#) we prove that the IOP Π of [Section 2.1](#) is perfectly complete and round-by-round knowledge sound. We do so by viewing Π as the composition of three Interactive Oracle Reductions (IORs, cf. [\[BCFW25\]](#) and [Section 3.3](#)) Π_1, Π_2, Π_3 . The first IOR Π_1 (Steps 2, 3, 4 and the first half of Step 5) reduces the initial claim, i.e. knowledge of a witness for $(\mathbf{x}, \mathbf{y}) = ((\mu, \mathbf{v}), f) \in \text{LANG}(\mathcal{R}_{\text{LinBitsRings}}(\mathbb{F}_q))$, to the grand product claim (7), which is a claim of knowledge of $\mathbf{g} \in \mathbb{F}_2^{\ell_1 \times \ell_2}$ with $f = \text{Enc}_C \circ \text{Pack}_\nu(\mathbf{g})$ and $\prod_{i \in [\ell_1]} ((g^{\pi_q^{-1}(\mathbf{u}_i^{(1)})} - 1) \cdot \mathbf{g}_{ij} + 1) = g^{\mu_j}$ for all $j \in [\ell_2]$ (an instance of the relation $\mathcal{R}_{\text{PESAT}}$ of [Definition 3.1](#), see [Section 4.2](#)). The second IOR Π_2 (second half of Step 5), is a standard instantiation⁹ of the GKR protocol for these ℓ_2 grand products which reduces them to the linear claims over $\mathbb{F}_{2^\nu}$ for the witness vectors $\text{bits}(\mathbf{f})_{*j}$ (8) (the relation $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$ of [Section 4.2](#)). The third IOR Π_3 (Step 6) proves the latter claim with the ring-switching technique of [\[DP24\]](#) followed by WHIR or Ligerito [\[ACFY25, NA25\]](#), exactly as in Flock [\[BRW26\]](#), possibly with a batched sumcheck before that, to reduce the claims (8) into a single MLE evaluation of $\text{bits}(\mathbf{f})$ over $\mathbb{F}_{2^\nu}$. It is well known that Π_2 and Π_3 are perfectly complete and round-by-round knowledge sound¹⁰, and that the composition of round-by-round knowledge sound IORs is again round-by-round knowledge sound ([Lemma 3.7](#)). Hence in this overview we only discuss Π_1 .

Remark 2.5 (The oracle serving as a commitment). For simplicity, we assume below that WHIR is configured with a proximity parameter within the unique decoding radius. In intuitive terms, this means that when the prover sends the oracle f , it is committing to a unique bit vector $\text{bits}(\mathbf{f}) \in \{0, 1\}^\ell$ determined by f . One can alternatively configure WHIR up to the Johnson bound, as long as one performs an Out Of Domain sample in Step 1.

Round-by-round knowledge soundness of Π_1 . The (possibly malicious) prover and verifier start with the public input $(\mathbf{v}, \mu)$ and the oracle f . Assume first that Step 3 does not trigger, so that $|\mathbb{F}_q| < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. Later we will discuss the case where this is otherwise.

Step 2: After Step 2, the claim is that the prover knows $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that

$$\langle \mathbf{u}, \pi_q(\mathbf{g}) \rangle = \mu \quad \text{in } \mathbb{F}_q, \quad \text{and} \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{g}). \quad (24)$$

This step has no knowledge soundness loss, in the sense that there exists an efficient algorithm that given a valid witness $\mathbf{g}$ for (24), outputs a witness $\mathbf{f} \in \mathbb{F}_q^{\ell_0}$ such that $((\mathbf{v}, \mu), f, \mathbf{f}) \in \mathcal{R}_{\text{LinBitsRings}}(\mathbb{F}_q)$. Indeed, for any such $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ it suffices to take $\mathbf{f} = \sum_{j \in [B]} 2^{j-1} \cdot \pi_q(\mathbf{g}_{ij})$, where these operations are over the field $\mathbb{F}_q$.

⁹Our optimizations from [Construction 6.1](#) for the grand product GKR apply at the implementation level: they keep the transcript invariant, and so they require no soundness analysis.

¹⁰For Π_3 this holds in a relaxed form, in the sense that a malicious prover may convince the verifier with an oracle f which is not exactly $\text{Enc}_C \circ \text{Pack}_\nu(\mathbf{g})$ for a valid witness $\mathbf{g}$, but is *close* to it in Hamming distance. This is a well-studied technicality [\[BSBHR18b, BCFW25, ACFY25, CFW26\]](#), which we treat in [Sections 3.3](#) and [4.4](#) using the formalism of round-by-round knowledge soundness *with relaxation* from [\[CFW26\]](#). We do not discuss this further in this overview.

Step 4: At the end of Step 4 the claim has become that the prover knows $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that

$$\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \mathbf{g}_{*j} \rangle = \mu_j \quad \text{in } \mathbb{Z} \quad \text{for all } j \in [\ell_2], \quad \text{and} \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{g}). \quad (25)$$

Again, we justify that given a valid witness $\mathbf{g}$ for (25), the exact same witness $\mathbf{g}$ satisfies (24), and hence this step also incurs no knowledge soundness loss. Indeed, if (25) holds and the verifier's checks pass, then

$$\langle \mathbf{u}, \pi_q(\mathbf{g}) \rangle = \sum_{j \in [\ell_2]} \pi_q(\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \mathbf{g}_{*j} \rangle) \cdot \mathbf{u}_j^{(2)} = \sum_{j \in [\ell_2]} \pi_q(\mu_j) \cdot \mathbf{u}_j^{(2)} = \mu,$$

Hence (24) holds for the same witness $\mathbf{g}$.

First half of Step 5: Finally, the first half of Step 5 replaces (25) by the grand product claim (7). If the latter holds for some $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$, then $\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \mathbf{g}_{*j} \rangle \equiv \mu_j \pmod{\text{ord}(g)}$, where $\text{ord}(g) = |\mathbb{F}_{2^\nu}| - 1$. Since $\mathbf{g}$ has bit entries and μ_j passed the range check of the verifier, both sides lie in $[0..(q-1) \cdot \ell_1]$. Further, $(q-1) \cdot \ell_1 < |\mathbb{F}_{2^\nu}| - 1$ by our assumption on q , so the congruence is an equality over $\mathbb{Z}$ and so (25) holds.

Hence, if Step 3 does not trigger, then Π_1 is a one-round IOR with round-by-round knowledge soundness error 0 (the verifier sends no message during this round, hence the round can be glued together with the next one in Section 4.4).

Step 3. Assume Step 3 triggers (Remark 2.1). Then the protocol has one extra round with a nonzero round-by-round knowledge soundness error. The prover sends $\mu' \in [0.. \ell \cdot (q-1)^2]$ with $\pi_q(\mu') = \mu$, the verifier samples a random prime $q' \in \mathcal{P}$.

In this case, at the end of Step 3, the claim (24) is replaced by a similar claim over a randomly sampled prime field $\mathbb{F}_{q'}$, namely the prover claims knowledge of $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $\langle \pi_{q'} \circ \pi_q^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{q'} \circ \pi_q^{-1}(\mathbf{u}^{(2)}), \pi_{q'}(\mathbf{g}) \rangle = \pi_{q'}(\mu')$ in $\mathbb{F}_{q'}$, and such that $f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{g})$. Let $\mathcal{E}$ be the event that this holds for some $\mathbf{g}$ but (24) does not. Because of Remark 2.5 and because the oracle f is the same in the claims (24) and (10), $\mathbf{g}$ is fixed before q' is sampled. Then the integer $h := \langle \pi_q^{-1}(\mathbf{u}^{(1)}) \otimes \pi_q^{-1}(\mathbf{u}^{(2)}), \mathbf{g} \rangle - \mu' \in \mathbb{Z}$ satisfies $\pi_{q'}(h) = \langle \mathbf{u}, \pi_{q'}(\mathbf{g}) \rangle - \mu' \neq 0$. Hence $h \neq 0$. Moreover, we have $|h| \leq 2 \cdot \ell \cdot (q-1)^2$, because $\mathbf{g}$ has bit entries and μ' passed the verifier's range check. Additionally, $\pi_{q'}(h) = 0$, and so q' divides h . Such a nonzero and bounded integer h has at most $\log(2\ell(q-1)^2)/\lfloor \log q_{\min} \rfloor$ prime divisors in a set of primes $\mathcal{P}$, where $q_{\min}$ is the smallest prime in $\mathcal{P}$. Hence the probability of $\mathcal{E}$ is at most

$$\kappa_0 := \frac{\log(2 \cdot \ell \cdot (q-1)^2)}{\lfloor \log q_{\min} \rfloor \cdot |\mathcal{P}|} \quad (26)$$

over the choice of q' .

After Step 3, we are exactly in the same scenario as at the beginning of Step 4 in the case that Step 3 did not trigger. Hence Π_1 is round-by-round knowledge sound with errors $(\kappa_0, 0)$ (or with error 0 if Step 3 is skipped).

Overall, Π has errors κ_0 followed by those of the GKR protocol and of the binary-field PCS (with prior possible batching sumcheck, followed by ring switch).

2.5.2 Soundness of Section 2.2

The full soundness statement of Section 2.2 can be found in Theorem 4.15, and is proved in Section A.1. We provide an overview of it next.

As in [Section 2.5.1](#), we view the protocol as a composition of three IORs Π_1, Π_2, Π_3 , with Π_2 and Π_3 being exactly the same as in [Section 2.5.1](#), i.e. Π_2 is the grand product GKR from Step 5, and Π_3 is the ring switch followed by a PCS such as WHIR or Ligerito. The IOR Π_1 consists of Steps 2 and 3 of [Section 2.2](#) followed by Step 4 and the first half of Step 5 of [Section 2.1](#).

After Step 3, the prover and verifier are exactly in the same setting as in Step 4 of [Section 2.1](#). Hence the analysis of the construction after Step 3 is exactly the same as in [Section 2.5.1](#). Because of this, below we only discuss Steps 1, 2 and 3.

Steps 1 and 2 (commitment and bitification). The (possibly malicious) prover and the verifier receive $\mathbf{x} = (\mathbf{v}, \mu) \in R^{\ell_0} \times R$ as input, and the prover sends an oracle $f \in \mathbb{F}_{2^\nu}^m$. The initial claim is that the prover knows $\mathbf{f} \in (S_{\leq B}^D)^{\ell_0}$ such that $(\mathbf{x}, f, \mathbf{f}) \in \mathcal{R}_{\text{LinBitsRings}}(S, R, \psi)$. After Step 2, the claim has been replaced by the claim that the prover knows $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that

$$\langle \mathbf{u}, \psi(\mathbf{g}) \rangle = \mu \quad \text{in } R, \quad \text{and} \quad f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{g}). \quad (27)$$

By definition, $\mathbf{u} = L^\top \cdot \mathbf{v}$. We argue that for any $\mathbf{g}$ satisfying (27) the vector $\mathbf{f} := L \cdot \mathbf{g}$ satisfies the initial claim, where L is the public fixed bitification matrix. Indeed, it is clear that $\langle \mathbf{v}, \mathbf{f} \rangle = \mu$. Moreover, we required when defining L that $L \cdot \mathbf{g}$ has all entries in $S_{\leq B}^D$ (as opposed to a different, possibly larger subset of S) whenever $\mathbf{g} \in \{0, 1\}^\ell$. Hence $\mathbf{f} \in (S_{\leq B}^D)^{\ell_0}$, as needed. This shows that Step 2 incurs no knowledge soundness loss.

Step 3. Let $\mathcal{E}$ be the event that the projected claim (16) holds over $\mathbb{F}_q$ for some $\mathbf{g} \in \{0, 1\}^\ell$ with $f = \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{g})$, but (27) does not hold. As in [Section 2.5.1](#), $\mathbf{g}$ is fixed by f before q, π_{rand} are sampled, see [Remark 2.5](#). Consider the polynomial

$$h := \langle \pi_{\text{canon}}^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{u}^{(2)}), \mathbf{g} \rangle - \mu' \in \mathbb{Z}[X_1, \dots, X_{k'}].$$

Since π_{canon} is a ring homomorphism with $\pi_{\text{canon}} \circ \pi_{\text{canon}}^{-1} = \text{id}$ and $\pi_{\text{canon}}(\mu') = \mu$, we have $\pi_{\text{canon}}(h) = \langle \mathbf{u}, \psi(\mathbf{g}) \rangle - \mu \neq 0$, and so $h \neq 0$. Since $\pi_{\text{canon}}^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{u}^{(2)})$ is public and $\mathbf{g}$ has bit entries, the total degree and the coefficient magnitudes of the polynomial $\langle \pi_{\text{canon}}^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{u}^{(2)}), \mathbf{g} \rangle$ are bounded by quantities that the verifier can compute. We let $D_{\mathbb{Z}}$ and $2^{B_{\mathbb{Z}}}$ be these quantities, respectively (concrete expressions are given in Round 1 of [Construction 4.4](#), cf. (32)). These are the bounds used by the verifier in Step 3 of [Section 2.2](#), which checks that μ' has total degree at most $D_{\mathbb{Z}}$ and coefficients of absolute value at most $2^{B_{\mathbb{Z}}}$. Hence h has total degree at most $D_{\mathbb{Z}}$ and coefficients of absolute value at most $2^{B_{\mathbb{Z}}+1}$.

Now, since π_{rand} is a ring homomorphism, the projected claim (16) states that $\pi_{\text{rand}}(h) = 0$. If, for example, $\mathcal{M}_q$ consists of the maps $h \mapsto \pi_q(h)(\alpha)$ for $\alpha \in \mathbb{F}_q^{k'}$ (as in our instantiation), then either q divides every coefficient of h , which as in [Section 2.5.1](#) happens with probability at most $(B_{\mathbb{Z}} + 1)/(\lfloor \log q_{\min} \rfloor \cdot |\mathcal{P}|)$, or $\pi_q(h)$ is a nonzero polynomial of total degree at most $D_{\mathbb{Z}}$, which vanishes at a random α . The latter occurs with probability at most $D_{\mathbb{Z}}/q_{\min}$ by the Schwartz–Zippel lemma. Overall we have

$$\Pr[\mathcal{E}] \leq \kappa_0 := \frac{D_{\mathbb{Z}}}{q_{\min}} + \frac{B_{\mathbb{Z}} + 1}{\lfloor \log q_{\min} \rfloor \cdot |\mathcal{P}|}, \quad (28)$$

cf. (26) for the case $k' = 0$, and [Lemma 4.3](#) for the concrete bounds $D_{\mathbb{Z}}, B_{\mathbb{Z}}$ of [Construction 4.4](#).

We conclude that Π_1 is round-by-round knowledge sound with errors κ_0 . The errors of the whole protocol construction from [Section 2.2](#) are κ_0 followed by those of Π_2 and Π_3 .

2.6 Further work and open problems

We list some ongoing work of ours and questions that we leave open:

- $\mathbb{BitZ}$ is not a zero-knowledge scheme as of now. Enabling zero-knowledge for it is ongoing work of ours.
- As mentioned throughout the paper, $\mathbb{BitZ}$ can be used in combination of the Zinc+ arithmetization system and PIOP in order to prove lattice computations. Ongoing work of ours is to integrate (in our implementation) the two schemes in order to prove Falcon signature verification.
- $\mathbb{BitZ}$'s verifier and proof sizes have asymptotic square root costs. Making them polylogarithmic is an intriguing problem which we leave open.
- $\mathbb{BitZ}$ works with finitely generated rings. This excludes the field or rational numbers. Ongoing work of ours is to extend $\mathbb{BitZ}$ to such field.

3 Preliminaries

We next present preliminary concepts and notation used throughout the paper.

General notation. Given an integer $n \geq 1$, we write $[n]$ to denote the set $\{1, \dots, n\}$. Given integers $m \leq n$, we let $[m..n]$ denote the set of integers $\{m, m+1, \dots, n\}$.

We use $\circ$ to denote composition of maps, so that for maps $g : S_1 \rightarrow S_2$ and $f : S_2 \rightarrow S_3$, we have $f \circ g : S_1 \rightarrow S_3$ defined as $f \circ g(x) = f(g(x))$.

Any map $f : S_1 \rightarrow S_2$ applies entry-wise to vectors from S_1^n , for any n , and similarly for matrices. The map f also applies coefficient-wise to polynomials with coefficients in S_1 . A multiplication of a vector or a matrix by a scalar means that we multiply each entry of the vector or matrix by that scalar.

We denote vectors with boldface letters, e.g. $\mathbf{u}, \mathbf{v}$. By $\mathbf{u}_i$ we denote the i -th position of $\mathbf{u}$. Sometimes we index the entries of a vector $\mathbf{u}$ with elements of a boolean hypercube $\{0, 1\}^t$.

We look at vectors both as row vectors and column vectors. When performing matrix-vector multiplications, it will always be clear which one we intend. Given a ring R , a matrix $M \in R^{n \times m}$ and vectors $\mathbf{u} \in R^m, \mathbf{v} \in R^n$, one has

$$\langle M \cdot \mathbf{u}, \mathbf{v} \rangle = \langle \mathbf{u}, M^\top \cdot \mathbf{v} \rangle, \quad (29)$$

where $M^\top$ denotes transposition and $\langle \cdot, \cdot \rangle$ denotes inner product.

3.1 Algebra

Concrete fields and rings We write $\mathbb{F}_q$ for the finite field with q elements, $\mathbb{Q}$ for the field of rational numbers, and $\mathbb{Z}$ for the ring of integers. For an integer $n \geq 1$, $\mathbb{Z}_n$ denotes the ring of integers modulo n , with the convention $\mathbb{Z}_0 := \mathbb{Z}$. When n is prime, $\mathbb{Z}_n = \mathbb{F}_n$. Let

$$\pi_n : \mathbb{Z} \rightarrow \mathbb{Z}_n$$

denote reduction modulo n . For $n \geq 1$, we fix the inverse $\pi_n^{-1} : \mathbb{Z}_n \rightarrow [0..n-1] \subseteq \mathbb{Z}$ mapping each element a in $\mathbb{Z}_n$ to the integer in the range $[0..n-1]$ representing a .

Rings in general All rings in this work are associative, commutative, and have a multiplicative identity. Given a ring R , we write $R[X_1, \dots, X_n]$ for the ring of polynomials with coefficients in R and variables $X_1, \dots, X_n$ (not necessarily multilinear).

The *characteristic* of a ring R , denoted $\text{char}(R)$, is the smallest $c \geq 1$, if it exists, such that $1 + \dots + 1 = 0$ in R . Otherwise, if $1 + \dots + 1 \neq 0$ for all c , the characteristic is zero. By $[0..n]_R$ we denote all elements in R of the form $1 + \dots + 1$ for $i = 0, \dots, n$. For every ring R we have a canonical embedding

$$\mathbb{Z}_{\text{char}(R)} \subseteq R$$

of the ring of integers modulo $\text{char}(R)$ into R . Namely, the set $\{n \cdot 1 \mid n \in \mathbb{Z}\} \subseteq R$ for 1 the identity element of R , forms the ring $\mathbb{Z}_{\text{char}(R)}$. Further, $\pi_{\text{char}(R)}$ induces a natural homomorphism (not necessarily surjective)

$$\pi_{\text{char}(R)} : \mathbb{Z} \rightarrow R \quad (30)$$

sending $n \in \mathbb{Z}$ to $n \cdot 1$ for 1 the identity of R .

Given a set or vector of integers $\mathbf{w} \in \mathbb{Z}^m$, we define

$$\mathbf{w}_R = \pi_{\text{char}(R)}(\mathbf{w}). \quad (31)$$

Finitely generated rings Further, all rings in this work are *finitely generated* as rings. Equivalently, every ring R in this work is isomorphic to a quotient

$$R = \mathbb{Z}[X_1, \dots, X_k]/I$$

of a polynomial ring over $\mathbb{Z}$ by an ideal I . This condition is satisfied by, for example:

1. All finite fields and the rings $\mathbb{Z}_n$;
2. $\mathbb{Z}$, and the polynomial rings $\mathbb{Z}[X]$ and $\mathbb{F}_q[X]$;
3. Arbitrary quotients of the previous rings, such as $\mathbb{Z}_n[X]/(h(X))$ for an arbitrary polynomial $h(X)$ and an arbitrary integer $n \geq 0$, which includes lattice rings.

It is not satisfied by, for example, $\mathbb{Q}$ and $\mathbb{Q}[X]$, which are not finitely generated as rings.

Given a finitely generated ring R with fixed generators $1, \eta_1, \dots, \eta_k$ (we always list 1 among the generators, in particular $k = 0$ for $\mathbb{Z}_n$) and bounds $B, D \geq 1$, we let $R_{\leq B}^{\leq D}$ denote the set of elements of R of the form

$$h = \sum_{\mathbf{a} \in \mathbb{Z}_{\geq 0}^k, a_1 + \dots + a_k < D} h_{\mathbf{a}} \cdot \eta_1^{a_1} \cdots \eta_k^{a_k}, \quad h_{\mathbf{a}} \in [0..2^B - 1]_R,$$

i.e. $R_{\leq B}^{\leq D}$ is formed by the elements of R that can be written as a polynomial of total degree less than D in $\eta_1, \dots, \eta_k$, with coefficients obtained by adding 1 together (in R) less than 2^B times.

3.2 Linear codes and relations

Linear codes Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code over $\mathbb{F}$ of dimension ℓ and blocklength m . Given two words w_1, w_2 of length n , the relative Hamming distance between w_1 and w_2 (the fraction of positions in which they differ) is denoted

$$\Delta(w_1, w_2).$$

Given $\delta \in (0, 1)$ and an integer $L \geq 1$, we say $\mathcal{C}$ is (δ, L) -*list-decodable* if for every word $w \in \mathbb{F}^m$ there are at most L codewords $c \in \mathcal{C}$ with $\Delta(w, c) < \delta$. Every code is $(\delta, 1)$ -list-decodable for $\delta \leq \Delta(\mathcal{C})/2$. The value $\Delta(\mathcal{C})/2$ is called the *unique decoding radius* of $\mathcal{C}$.

Relations A relation $\mathcal{R}$ consists of triples $(\mathbf{x}, \mathbf{y}; \mathbf{w})$, where $\mathbf{x}$ is the explicit instance, $\mathbf{y}$ the implicit instance or oracle, and $\mathbf{w}$ the witness. The corresponding language is $\text{LANG}(\mathcal{R}) := \{(\mathbf{x}, \mathbf{y}) \mid \exists \mathbf{w} \text{ such that } (\mathbf{x}, \mathbf{y}; \mathbf{w}) \in \mathcal{R}\}$.

Polynomial Equation Satisfiability (PESAT) over finite fields We recall a relation from [BCFW25] over finite fields, which captures popular constraints such as R1CS, CCS, AIR (cf. [BCFW25]), and lookup and permutation constraints (cf. [GWV⁺25]).

Fix a finite field $\mathbb{F}$, a witness length ℓ , an input length k , a number of rows n , a collection of n polynomials $\mathbf{p} = (\mathbf{p}_1, \dots, \mathbf{p}_n)$ on $k + \ell$ variables.

Definition 3.1. The PESAT relation with respect to these parameters is defined as

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}) = \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{F}^k, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}^\ell \end{array} \middle| \mathbf{p}_i(\mathbf{x}, \mathbf{f}) = 0 \text{ for all } i \in [n] \right\}$$

Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code of dimension ℓ' and blocklength m over $\mathbb{F}$. Let $W \subseteq \mathbb{F}^\ell$ be a subset and let $T : W \rightarrow \mathbb{F}^{\ell'}$ be an arbitrary map (looking ahead, for us T will be a packing map that packs elements from $\mathbb{F}_2$ into elements of a larger binary field, so that $W = \mathbb{F}_2^\ell \subseteq \mathbb{F}^\ell$). We define the *Encoded PESAT relation* as:

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}, \mathcal{C}, T) := \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{F}^k, \\ \mathbf{y} = f \in \mathbb{F}^{\ell'}, \\ \mathbf{w} = \mathbf{f} \in W \end{array} \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(T(\mathbf{f})) = f \\ \wedge \mathbf{p}_i(\mathbf{x}, \mathbf{f}) = 0 \text{ for all } i \in [n] \end{array} \right\}$$

We will drop the reference to T whenever it is understood from the context. When $n = 1$ and $\mathbf{p}_1(\mathbf{x}, \mathbf{f}) = \langle \mathbf{f}, \mathbf{v} \rangle - \mu$ where $\mathbf{x}$, the relation $\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}, \mathcal{C}, T)$ is called a *constrained code* [ACFY25]. We denote it

$$\mathcal{R}_{\text{Lin}}(\mathbb{F}, \mathcal{C}, T).$$

Definition 3.2 (Relaxation of $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$). Given $\delta \in (0, 1)$, the *relaxation* of $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$, denoted

$$\mathcal{R}_{\text{PESAT}}^\delta, \quad \mathcal{R}_{\text{Lin}}^\delta$$

are the relations defined exactly as $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$, respectively, except that the constraint $\text{Enc}_{\mathcal{C}}(T(\mathbf{f})) = f$ is replaced by

$$\Delta(\text{Enc}_{\mathcal{C}}(T(\mathbf{f})), f) < \delta.$$

Multilinear extensions. Given a field $\mathbb{F}$ and $\mathbf{u} \in \mathbb{F}^{2^r}$ indexed by $\{0, 1\}^r$, we denote by $\tilde{\mathbf{u}} \in \mathbb{F}[X_1, \dots, X_r]$ the unique multilinear polynomial with $\tilde{\mathbf{u}}(\mathbf{b}) = \mathbf{u}_{\mathbf{b}}$ for all $\mathbf{b} \in \{0, 1\}^r$. For $\alpha, \beta \in \mathbb{F}^r$ we let $\text{eq}(\alpha; \beta) := \prod_{i \in [r]} (\alpha_i \cdot \beta_i + (1 - \alpha_i) \cdot (1 - \beta_i))$, so that $\tilde{\mathbf{u}}(\alpha) = \sum_{\mathbf{b} \in \{0, 1\}^r} \mathbf{u}_{\mathbf{b}} \cdot \text{eq}(\mathbf{b}; \alpha)$. We write $\text{eq}(\mathbf{b}, \mathbf{c}; \alpha)$ for $\text{eq}((\mathbf{b}, \mathbf{c}); \alpha)$.

3.3 Interactive Oracle Reductions (IOR) of proximity

A public-coin *Interactive Oracle Reduction (IOR) of proximity* $\Pi = (\mathsf{P}, \mathsf{V})$ from a relation $\mathcal{R}$ to a relation $\mathcal{R}'$ is specified as follows. The prover P receives $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$. The verifier V receives $\mathbf{x}$ and query access to $\mathbf{y}$. They interact for k rounds. In round $j \in [k]$, the prover sends an oracle π_j and the verifier sends a uniformly random message $\rho_j \in \{0, 1\}^{r_j}$. After the k rounds, the verifier makes queries to $\mathbf{y}$ and to $\pi_1, \dots, \pi_k$. Then:

- V either rejects or outputs a new pair $(\mathbf{x}', \mathbf{y}')$, where $\mathbf{y}'$ is specified implicitly in terms of the oracles $\mathbf{y}$ and $\pi_1, \dots, \pi_k$.
- P outputs a witness $\mathbf{w}'$ such that $(\mathbf{x}', \mathbf{y}', \mathbf{w}') \in \mathcal{R}'$.

We write $\langle \mathsf{P}, \mathsf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})$ for the interaction of prover and verifier during the IOR.

Definition 3.3 (Completeness). Π has (*perfect*) *completeness* if for every $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$,

$$\Pr_{\rho_1, \dots, \rho_k} [(\mathbf{x}', \mathbf{y}'; \mathbf{w}') \in \mathcal{R}' \mid (\mathbf{x}', \mathbf{y}'; \mathbf{w}') \leftarrow \langle \mathsf{P}, \mathsf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})] = 1.$$

Remark 3.4. An IOP for a relation $\mathcal{R}$ is an IOR from $\mathcal{R}$ to the trivial relation $\mathcal{R}_{\text{triv}} = \{(\mathbf{x} = \perp, \mathbf{y} = \perp, \mathbf{w} = \perp)\}$.

We proceed to define round-by-round knowledge soundness of an IOR, following [BCFW25] in the formulation with relaxed relations of [CFW26].

Definition 3.5 (Knowledge state function with relaxation). Let $\Pi = (\mathsf{P}, \mathsf{V})$ be an IOR from a relation $\mathcal{R}$ to $\mathcal{R}'$. A *knowledge state function with relaxation* $(\widetilde{\mathcal{R}}, \widetilde{\mathcal{R}'})$ for Π is a (possibly inefficient) function KState that, on input $(\mathbf{x}, \mathbf{y})$, interaction transcript tr , and knowledge state witness $\mathbf{w}$, outputs a bit and has the following properties.

- **Empty transcript.** If $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}; \mathbf{w}) \in \widetilde{\mathcal{R}}$.
- **Prover moves.** If tr is a transcript where the prover is about to move and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$ for every prover message π .
- **Full transcript.** If $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if V on tr outputs $(\mathbf{x}', \mathbf{y}')$ such that $(\mathbf{x}', \mathbf{y}'; \mathbf{w}) \in \widetilde{\mathcal{R}'}$.

Definition 3.6 (Round-by-round knowledge soundness). A k -round IOR Π has *round-by-round knowledge soundness with relaxation* with errors $(\kappa_1, \dots, \kappa_k)$ if there exists a knowledge state function KState as in Definition 3.5 and a deterministic extractor $\mathcal{E}_{\text{rbr}}$ such that for every $(\mathbf{x}, \mathbf{y})$, every $j \in [k]$, and every transcript tr where the verifier is about to send ρ_j ,

$$\Pr_{\rho_j} \left[\exists \mathbf{w} \text{ such that } \begin{array}{l} \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_j, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_j, \mathbf{w}) = 1 \end{array} \right] \leq \kappa_j(\mathbf{x}, \mathbf{y}).$$

If $(\widetilde{\mathcal{R}}, \widetilde{\mathcal{R}'}) = (\mathcal{R}, \mathcal{R}')$ then we say the round-by-round knowledge soundness of Π is *without relaxation*.

Lemma 3.7 (Composition of IORs [CFW26, Theorem 4.5]). *Let Π_1 be a perfectly complete k_1 -round IOR from $\mathcal{R}_1$ to $\mathcal{R}_2$ with round-by-round knowledge soundness with relaxation $(\widetilde{\mathcal{R}}_1, \widetilde{\mathcal{R}}_2)$ and errors $\kappa_{11}, \dots, \kappa_{1k_1}$, and let Π_2 be a perfectly complete k_2 -round IOR from $\mathcal{R}_2$ to $\mathcal{R}_3$ with round-by-round knowledge soundness with relaxation $(\widetilde{\mathcal{R}}_2, \widetilde{\mathcal{R}}_3)$ and errors $\kappa_{21}, \dots, \kappa_{2k_2}$. Then the IOR from $\mathcal{R}_1$ to $\mathcal{R}_3$ that runs Π_1 and then Π_2 on the output of Π_1 is perfectly complete, has $k_1 + k_2$ rounds, and has round-by-round knowledge soundness with relaxation $(\widetilde{\mathcal{R}}_1, \widetilde{\mathcal{R}}_3)$ and errors $\kappa_{11}, \dots, \kappa_{1k_1}, \kappa_{21}, \dots, \kappa_{2k_2}$.*

Grinding. [Albert: Review](#) Grinding [Sta21] is a technique where, given an IOP Π for a relation $\mathcal{R}$ with relaxation $\tilde{\mathcal{R}}$ with round-by-round knowledge errors $\kappa_1, \dots, \kappa_k$, produces a new IOP Π' for $\mathcal{R}$ with round-by-round knowledge with relaxation $\tilde{\mathcal{R}}$ with errors

$$2^{-\alpha_1} \cdot \kappa_1, \dots, 2^{-\alpha_k} \cdot \kappa_k.$$

The new IOP Π' works exactly as Π , except that, at each round $i \in [k]$, the verifier samples a random string of α_i bits, called *grinding string*, which it sends alongside its message, and rejects unless all bits are 0.

Clearly Π' is not necessarily complete, even if Π is. The goal is to use the BCS compilation of IOPs, and not the IOPs themselves (i.e. the compilation where oracles are replaced by Merkle roots and where interactivensness is removed through the Fiat-Shamir transform). To this end, one chooses $\alpha_1, \dots, \alpha_k$ small enough that the prover of the BCS transform of Π' can *grind* transcripts until all of the grinding strings in the transcript are the zero string. This results in the prover having to compute per round $i \in [k]$, an average of 2^{α_i} hashes of its i -th message followed by a small string of bits (in practice, one has the prover send a small string of bits next to its message, so that during grinding, it does not need to produce fresh Merkle roots at each grinding iteration). In [Table 3](#), we provide example grinding times for different values of α_i .

The round-by-round knowledge errors of Π' are obtained by defining a state function which is exactly as the state function of Π , except that the function outputs zero if some of the aforementioned strings of bits are not the zero string. The extractor for Π' is defined as the extractor for Π , ignoring the additional strings of bits in the transcript.

Table 3: Cost of finding a 64-bit nonce such that $\text{BLAKE3}(\text{msg}, \text{nonce})$ has at least α leading zero bits, with msg of 256 bits. Single and multi-threaded (10 threads) on a MacBook Air M5 24 GB, CPU-only.

α	18	19	20	21	22	23	24	25
Mean, 1 thread (ms)	3.9	9.2	17.6	33.2	62.8	130.5	285.0	518.7
Median, 1 thread (ms)	2.9	6.0	12.4	25.2	44.3	92.7	213.2	379.5
Mean, 10 threads (ms)	0.8	1.7	3.0	5.5	10.3	21.3	47.9	90.2
Median, 10 threads (ms)	0.6	1.1	2.2	4.3	7.3	15.1	36.0	65.0

We next discuss the costs of $\mathbb{B}\text{it}\mathbb{Z}$, and how to instantiate it using $\mathbb{F}_{2^{128}}$ as binary field while achieving 100 or 128 bits of round-by-round knowledge soundness.

4 $\mathbb{B}\text{it}\mathbb{Z}$ IOP for LinBitsRings

In this section we describe the $\mathbb{B}\text{it}\mathbb{Z}$ IOP and the main relations it proves. We begin with a summary of the notation we use throughout the paper. Most of the notation will be introduced subsequently, but nevertheless we record it here for convenience of the reader.

ℓ	Bit-length of witness cf. Section 4.1
$\mathbf{f}$	Witness, either over $\{0, 1\} \subseteq \mathbb{Z}$, or $S_{\leq B}^{<D_S}$ (see below)
$\mathbf{v}, \mathbf{u}$	public vectors in inner product constraints, over R
R	Ring where constraints live, cf. Sections 4.1 and 4.5
ℓ_1, ℓ_2	Tensor decomposition dimensions, with $\ell_1 \cdot \ell_2 = \ell$
π_n	Reduction modulo n
$k + 1$	Number of generators of R
π_{canon}^R	Canonical morphism $\pi_{\text{canon}}^R : \mathbb{Z}[X_1, \dots, X_k] \rightarrow R$ (see Section 4.3)
$\mathbb{F}_{2^\nu}$	A binary field, typically with $\nu = 128$
$\mathcal{C}$	Linear code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν , and length m
$\mathcal{P}$	Set of primes
$\mathcal{M}_q$	Set of homomorphisms $\pi_{\text{rand}} : \mathbb{Z}[X_1, \dots, X_k] \rightarrow \mathbb{F}_q$ for $q \in \mathcal{P}$
ℓ_0	Witness length in the general LinBitsRings relation, cf. Section 4.5
S	Ring where witness lives, cf. Section 4.5
$S_{\leq B}^{<D_S}$	Elements of S expressible as polynomials of degree $< D_S$ in the generators of S with coefficients of bit-size $\leq B$, cf. Section 3.1 (In the core LinBitsRing relation, $S = \mathbb{Z}$, $B = 1$)
ψ	Ring homomorphism $\psi : S \rightarrow R$
d	Number of monomials $\gamma_1 = 1, \dots, \gamma_d$ of degree $< D_S$ in the generators of S (Section 4.5)
L	Bitification matrix, $L \in S^{\ell_0 \times \ell}$, cf. Definition 4.9

Table 4: Summary of notation used throughout the paper.

4.1 The core LinBitsRings relation

In this section we define the main linear relation we work with. We start by fixing notation:

- Let $\mathbb{F}_{2^\nu}$ be a field with 2^ν elements. Let $\beta_1, \dots, \beta_\nu \in \mathbb{F}_{2^\nu}$ be an $\mathbb{F}_2$ -basis of $\mathbb{F}_{2^\nu}$ seen as an $\mathbb{F}_2$ -vector space of dimension ν . Given a vector $\mathbf{u} \in \mathbb{F}_2^\ell$, with ν dividing ℓ , we define

$$\text{Pack}_\nu(\mathbf{u}) \in \mathbb{F}_{2^\nu}^{\ell/\nu}$$

as the vector whose j -th entry is

$$\text{Pack}_\nu(\mathbf{u})_j = \mathbf{u}_{(j-1) \cdot \nu + 1} \cdot \beta_1 + \dots + \mathbf{u}_{(j-1) \cdot \nu + \nu - 1} \cdot \beta_{\nu-1} + \mathbf{u}_{j \cdot \nu} \cdot \beta_\nu \in \mathbb{F}_{2^\nu}.$$

- Let $\ell \geq 1$ be a dimension parameter divisible by ν , $\mathcal{C}$ a code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and blocklength m .
- Let R be a finitely generated ring (typically R is $\mathbb{Z}$, a finite field, not necessarily a prime field, or a quotient of a polynomial ring $\mathbb{Z}[X]$ or $\mathbb{F}_q[X]$) and let $\pi : \mathbb{Z} \rightarrow R$ be the homomorphism mapping an integer a to $a \cdot 1 \in R$ (i.e. reduction modulo the characteristic of R).

We begin by defining a particular subclass of our target relation.

Definition 4.1. The *Core LinBitsRings* relation is defined as

$$\mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C}) := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \middle| \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f}) \\ \langle \pi(\mathbf{f}), \mathbf{v} \rangle = \mu \quad \text{in } R \end{array} \right\}$$

For $\delta \in (0, 1)$, we let $\mathcal{R}_{\text{LinBitsRings}}^\delta$ be the relation $\mathcal{R}_{\text{LinBitsRings}}$ after replacing the constraint $f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})$ by

$$\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta.$$

4.2 Tensor decomposition and products of constrained codes

As discussed in [Section 2.1](#), BitZ crucially demands and uses a tensor decomposition of $\mathbf{v}$. We next establish notation and basic definitions around it.

Let $(\mathbf{x}, \mathbf{y}, \mathbf{w}) = ((\mu, \mathbf{v}), f, \mathbf{f}) \in \mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C})$. We fix a tensor decomposition of the vector $\mathbf{v} \in R^\ell$ as

$$\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} = \left(\mathbf{v}_i^{(1)} \cdot \mathbf{v}_j^{(2)} \right)_{i \in [\ell_1], j \in [\ell_2]}$$

for some $\mathbf{v}^{(1)} \in R^{\ell_1}, \mathbf{v}^{(2)} \in R^{\ell_2}$, with $\ell_1 \cdot \ell_2 = \ell$. Note that the trivial tensor decomposition is admitted, i.e. we may effectively not decompose $\mathbf{v}$ by choosing $\ell_1 = \ell, \ell_2 = 1, \mathbf{v}^{(1)} = \mathbf{v}$, and $\mathbf{v}^{(2)} = (1)$.

We index the entries of $\mathbf{f}$ as $\mathbf{f} = (\mathbf{f}_{ij})_{i \in [\ell_1], j \in [\ell_2]}$. Note that

$$\langle \pi(\mathbf{f}), \mathbf{v} \rangle = \sum_{j \in [\ell_2]} \left(\sum_{i \in [\ell_1]} \pi(\mathbf{f}_{ij}) \cdot \mathbf{v}_i^{(1)} \right) \cdot \mathbf{v}_j^{(2)} = \sum_{j \in [\ell_2]} \langle \pi(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \cdot \mathbf{v}_j^{(2)},$$

where $\mathbf{f}_{*j} = (\mathbf{f}_{ij})_{i \in [\ell_1]}$. Consider the relation (cf. [Definition 3.1](#)) $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu)$ for ℓ_2 constraint polynomials Q_{BitZ} so that

$$\begin{aligned} & \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu) \\ &= \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{y}, \boldsymbol{\mu}) \in \mathbb{F}_{2^\nu}^{\ell_1} \times \mathbb{F}_{2^\nu}^{\ell_2}, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \mathbb{F}_2^{\ell_1 \times \ell_2} \end{array} \right) \middle| \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu(\mathbf{f}) \\ \bigwedge_{i=1}^{\ell_1} ((\mathbf{y}_i - 1) \cdot \mathbf{f}_{ij} + 1) = \boldsymbol{\mu}_j \text{ for all } j \in [\ell_2] \end{array} \right\}. \end{aligned}$$

Here Q_{BitZ} consists of the ℓ_2 polynomials $\prod_{i=1}^{\ell_1} ((\mathbf{y}_i - 1) \cdot \mathbf{f}_{ij} + 1) - \boldsymbol{\mu}_j, j \in [\ell_2]$.

We assume prover and verifier have access to an IOR Π_{red} that reduces $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$, where, recall, this latter relation has predicates of the form $\langle \mathbf{g}, \mathbf{z} \rangle = c$ with $\mathbf{g} \in \mathbb{F}_2^\ell, \mathbf{z}$ a vector over $\mathbb{F}_{2^\nu}, c \in \mathbb{F}_{2^\nu}$, and $g = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu(\mathbf{g})$. In our instantiation Π_{red} is an optimized GKR protocol for grand products (cf. [Section 6](#)), which reduces the initial grand product claims to MLE evaluation claims of $\mathbf{y}$ and $\mathbf{f}$ over $\mathbb{F}_{2^\nu}$.

We assume Π_{red} works so that the verifier never queries $\mathbf{y}$; and that, given an input

$$(\mathbf{x} = (\mathbf{y}, \boldsymbol{\mu}), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu),$$

Π_{red} outputs

$$((\mathbf{z}, c), f, \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$$

for some vector $\mathbf{z} \in \mathbb{F}_{2^\nu}^\ell$ and field element $c \in \mathbb{F}_{2^\nu}$. In particular, Π_{red} outputs the same oracle f and witness $\pi_2(\mathbf{f})$ as the oracle and witness it is given as inputs.

This is indeed the case with GKR: GKR reduces the initial circuit evaluation claim with a witness input, into an MLE evaluation claim of the same witness, without querying any oracle in-between. Consequently an IOR between the versions of these relations with oracle $\perp$ (such as [Construction 6.1](#), cf. [Lemma 6.2](#)) can be turned, in a straightforward manner, into an IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PESAT}}^\delta, \mathcal{R}_{\text{Lin}}^\delta)$ and the same errors, for every δ within the unique decoding radius of $\mathcal{C}$ (cf. [Lemma 6.2](#)). The input to the IOR now includes an oracle f , and this same oracle is output at the end of the IOR. The knowledge state function adds the condition $\Delta(f, \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu(\mathbf{w})) < \delta$.

4.3 The BitZ IOP for the core LinBitsRings relation

Algebraic set-up. Recall ([Section 3.1](#)) that all rings in this work are finitely generated as rings. We fix generators $1, \eta_1, \dots, \eta_k \in R$ and we let

$$\pi_{\text{canon}}^R : \mathbb{Z}[X_1, \dots, X_k] \rightarrow R$$

be the surjective ring homomorphism mapping each integer a to $a \cdot 1 \in R$, and X_i to η_i for $i \in [k]$. The choice of $\eta_1, \dots, \eta_k$ is far from unique, see [Example 4.2](#) below.

We also fix an inverse map $\pi_{\text{canon}}^{R^{-1}} : R \rightarrow \mathbb{Z}[X_1, \dots, X_k]$ for π_{canon}^R , so that $\pi_{\text{canon}}^R \circ \pi_{\text{canon}}^{R^{-1}} = \text{id}$. We refer to $\pi_{\text{canon}}^{R^{-1}}(a)$ as the *lift* of $a \in R$. As before, $\pi_{\text{canon}}^{R^{-1}}$ is not unique. In [Example 4.2](#) we provide example inverse maps.

Recall (see [Section 3.1](#)) that π_n denotes reduction modulo n and that π_n^{-1} maps each element from $\mathbb{Z}_n$ to its integer representative in $[0..n-1]$.

Example 4.2. We provide some examples of rings R and choices for projections π_{canon}^R and their inverses:

- **Prime fields.** If $R = \mathbb{F}_q$ is a prime field with q elements, then we can take $k = 0$, so that $\pi_{\text{canon}}^{\mathbb{F}_q} = \pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$ is simply reduction modulo q , with its inverse being π_q^{-1} (see [Section 3.1](#)).
- **Integers.** Similarly, for $R = \mathbb{Z}$ we can take $k = 0$, and then $\pi_{\text{canon}}^{\mathbb{Z}}$ and its lifts are the identity map on $\mathbb{Z}$.
- **Field extensions.** If R is a field extension of a prime field $\mathbb{F}_q$ of degree $d > 1$, then we can take $k = 1$ and $\eta_1 = x$, where x is a root of an irreducible polynomial $h(X_1) \in \mathbb{F}_q[X_1]$ of degree d (so that $R = \mathbb{F}_q(x)$). Then $\pi_{\text{canon}}^R : \mathbb{Z}[X_1] \rightarrow R$ consists in reducing coefficients modulo q , and then sending X_1 to x . Writing $a \in R$ as $a = \sum_{i=0}^{d-1} a_i \cdot x^i$ with $a_i \in \mathbb{F}_q$, one can then define a natural lift of a as $\sum_{i=0}^{d-1} \pi_q^{-1}(a_i) \cdot X_1^i$.
- **Univariate integer polynomials.** If $R = \mathbb{Z}[X]$, then we can take $k = 1$ and $\eta_1 = X$, and then $\pi_{\text{canon}}^{\mathbb{Z}[X]} : \mathbb{Z}[X_1] \rightarrow \mathbb{Z}[X]$ is an isomorphism, with lifts given by its inverse.
- **Lattices, Galois rings, etc.** More generally, if R is finitely generated as a $\mathbb{Z}_{\text{char}(R)}$ -module and it is free as a $\mathbb{Z}_{\text{char}(R)}$ -module, with $\mathbb{Z}_{\text{char}(R)}$ -module-basis $\beta_1 = 1, \beta_2, \dots, \beta_m$ (a condition satisfied by, e.g., all finite fields, $\mathbb{Z}$, and $\mathbb{Z}_n[X]/(h(X))$ for any monic $h(X)$ and any integer $n \geq 0$), then we may take $k = m - 1$ and $\eta_i = \beta_{i+1}$ for $i \in [k]$. Then given $a \in R$ and writing $a = \sum_{i \in [m]} a_i \cdot \beta_i$ with $a_i \in \mathbb{Z}_{\text{char}(R)}$, we can define the lift of a as $\pi_{\text{char}}^{-1}(a_1) + \sum_{i=2}^m \pi_{\text{char}}^{-1}(a_i) \cdot X_{i-1}$.

We assume without further comment that it is efficient to compute values $\pi_{\text{canon}}^R(h)$ for $h \in \mathbb{Z}[X_1, \dots, X_k]$, and to check whether $\pi_{\text{canon}}^R(h) = 0$. This is the case in all examples above.

The BitZ IOP. We provide an IOP for $\mathcal{R}_{\text{LinBitsRings}}$ next. We assume the notation from the previous two [Sections 4.1](#) and [4.2](#), and from the preliminary [Section 3.1](#), together with:

- Let Π_{iopp} be an IOP for $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$. In our instantiation, Π_{iopp} is WHIR or Ligerito [\[ACFY25, NA25\]](#) over binary fields as implemented in Flock [\[BRW26\]](#), with ring-switching [\[DP24\]](#) and a preliminary sumcheck over $\mathbb{F}_{2^\nu}$ that transforms arbitrary inner products to MLE evaluation claims. We set the proximity parameter to the Johnson bound when targeting 100 bits of round-by-round knowledge soundness, and to the unique decoding radius for higher security regimes.
- Let $\ell = \ell_1 \cdot \ell_2$ be dimension parameters, and R a ring with generators $1, \eta_1, \dots, \eta_k$, as in the previous sections. We let $\mathcal{P}$ be a set of primes with $\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. Given the tensor decomposition $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}$ from [Section 4.2](#), we define

$$\begin{aligned} \mathbf{u} &:= (\mathbf{v}^{(1)}, \mathbf{v}^{(2)}) \in R^{\ell_1 + \ell_2} \\ D &:= \max_{i \in [\mathbf{u}]} \deg(\pi_{\text{canon}}^R{}^{-1}(\mathbf{u}_i)), & N_R &:= \max_{i \in [\mathbf{u}]} \|\pi_{\text{canon}}^R{}^{-1}(\mathbf{u}_i)\|_\infty, \end{aligned} \quad (32)$$

where $\deg$ denotes total degree, and $\|\cdot\|_\infty$ denotes the maximum absolute value of the coefficients. We also let $T := \binom{D+k}{k}$ be the number of monomials in $X_1, \dots, X_k$ of total degree at most D . Its role is the following: if $f, g \in \mathbb{Z}[X_1, \dots, X_k]$ have total degree at most D , then $\|f \cdot g\|_\infty \leq T \cdot \|f\|_\infty \cdot \|g\|_\infty$, since each monomial of $f \cdot g$ is the sum of at most T pairs of monomials of f and g .

For example, $T = 1$ if $k = 0$; $T = D + 1$ if $k = 1$; and in general $T \leq (D + 1)^k$.

- For every prime $q \in \mathcal{P}$, let $\mathcal{M}_q$ be a set of ring homomorphisms

$$\pi_{\text{rand}} : \mathbb{Z}[X_1, \dots, X_k] \rightarrow \mathbb{F}_q,$$

and let $\mathcal{M} := (\mathcal{M}_q)_{q \in \mathcal{P}}$. The verifier will sample primes and morphisms from $\mathcal{P}$ and $\mathcal{M}_q$. We let $\theta(\mathcal{P}, \mathcal{M}, \ell)$ be the maximum probability that $\pi_{\text{rand}}(h) = 0$, over random $q \leftarrow \mathcal{P}$, $\pi_{\text{rand}} \leftarrow \mathcal{M}_q$, and where the maximum is taken over nonzero polynomials $h \in \mathbb{Z}[X_1, \dots, X_k]$ of total degree at most $2 \cdot D$, with $\|h\|_\infty \leq 2 \cdot T \cdot \ell \cdot N_R^2$, and such that $\pi_{\text{canon}}^R(h) \neq 0$. Precisely:

$$\theta(\mathcal{P}, \mathcal{M}, \ell) := \max_h \left\{ \Pr_{q \leftarrow \mathcal{P}, \pi_{\text{rand}} \leftarrow \mathcal{M}_q} [\pi_{\text{rand}}(h) = 0] \right\}, \quad (33)$$

where h ranges over the polynomials just described. See [Lemma 4.3](#) for an example. The quantities D , N_R , T and $\theta(\mathcal{P}, \mathcal{M}, \ell)$ depend on R and on the instance vector $\mathbf{v}$. We omit this dependence from the notation.

Lemma 4.3 (Example definition of $\mathcal{M}_q$ and the corresponding $\theta(\mathcal{P}, \mathcal{M}, \ell)$). *Let $\mathcal{P}$ be a set of primes. For each $q \in \mathcal{P}$ let $\mathcal{M}_q$ be the set of all maps $\pi : \mathbb{Z}[X_1, \dots, X_k] \rightarrow \mathbb{F}_q$ of the form*

$$\pi(h(X_1, \dots, X_k)) = \pi_q(h)(\alpha_1, \dots, \alpha_k), \quad \text{for } h \in \mathbb{Z}[X_1, \dots, X_k],$$

for some $\alpha_1, \dots, \alpha_k \in \mathbb{F}_q$, and where $\pi_q(h)$ is h after reducing its coefficients modulo q . Then

$$\theta(\mathcal{P}, \mathcal{M}, \ell) \leq \frac{2 \cdot D}{q_{\min}} + \frac{\log(2 \cdot T \cdot \ell \cdot N_R^2)}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor}.$$

where $q_{\min}$ is the smallest prime in $\mathcal{P}$.

Proof. We defer the proof to later in [Section 4.4](#). $\square$

Construction 4.4 (IOP for the core $\mathcal{R}_{\text{LinBitsRings}}$ relation). **Parameters.** $(\ell, \ell_1, \ell_2, \Pi_{\text{iopp}}, \Pi_{\text{red}}, \mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}})$ comprised of: a witness vector length ℓ ; tensor decomposition dimensions ℓ_1, ℓ_2 satisfying $\ell_1 \cdot \ell_2 = \ell$; the IOP Π_{iopp} described above, the IOR Π_{red} from [Section 4.2](#); and the sets $\mathcal{P}, \mathcal{M}_q$ from above.

Inputs. The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell$ and $\mathbf{y} = f \in \mathbb{F}_{2^\nu}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C})$. We assume the tensor split from the previous [Section 4.2](#), and adopt its notation.

Let $g \in \mathbb{F}_{2^\nu}$ be a fixed generator of the multiplicative group $\mathbb{F}_{2^\nu}^*$. Denote π_{canon}^R simply as π_{canon} .

1. **Round 0 (optional): Out Of Domain (OOD) sample.** Optionally, the verifier samples sends a random point of a certain shape, and the prover replies with the MLE evaluation of $\text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \pi_2(\text{bits}(\mathbf{f}))$ on the point. This step is skipped if the proximity parameter δ is set within the unique decoding radius of $\mathcal{C}$ (cf. [Section 3.2](#), and [Theorem 4.7](#)). Otherwise the step is executed (see [Section A.2](#)). In that case, the verifier samples and sends a random $\zeta \in \mathbb{F}_{2^\nu}$, and the prover responds with

$$y = \widetilde{\text{Pack}_\nu \circ \pi_2(\mathbf{f})}(\zeta), \quad \text{where } \zeta := (\zeta^{2^0}, \zeta^{2^1}, \dots, \zeta^{2^{\log(\ell/\nu)-1}}).$$

2. **Round 1 (optional): replacing R by a “small” prime field $\mathbb{F}_q$.** This step can be skipped (i.e. prover and verifier move to Round 2 directly) if R is a prime field $\mathbb{F}_p$ and $p < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$; in that case we set $q := p$ in Round 2 and Phase 3. Otherwise the step is executed.

The prover computes and sends $\mu' \in \mathbb{Z}[X_1, \dots, X_k]$ such that

$$\langle \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}), \mathbf{f} \rangle = \mu' \quad \text{in } \mathbb{Z}[X_1, \dots, X_k].$$

The verifier checks that μ' has total degree at most $2 \cdot D$ and $\|\mu'\|_\infty \leq T \cdot \ell \cdot N_R^2$, then it checks that $\pi_{\text{canon}}(\mu') = \mu$, and replies with a random prime $q \in \mathcal{P}$ and a random morphism $\pi_{\text{rand}} : \mathbb{Z}[X_1, \dots, X_k] \rightarrow \mathbb{F}_q$ from $\mathcal{M}_q$. The prover and verifier update:

$$\begin{aligned} \mathbf{v}^{(1)} &\leftarrow \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \in \mathbb{F}_q^{\ell_1}, \\ \mathbf{v}^{(2)} &\leftarrow \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}) \in \mathbb{F}_q^{\ell_2}, \\ \mu &\leftarrow \pi_{\text{rand}}(\mu') \in \mathbb{F}_q. \end{aligned}$$

The prover and verifier now set to prove the updated claim

$$\langle \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}, \pi_q(\mathbf{f}) \rangle = \mu \quad \text{over } \mathbb{F}_q. \tag{34}$$

Remark 4.5 (Skipping Round 1 when $R = \mathbb{Z}$ and $\|\mathbf{v}\|_\infty$ is “small”). This step can also be skipped when $R = \mathbb{Z}$ and all entries of $\mathbf{v}^{(1)}$ are nonnegative and less than $(|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. In that case the claim to prove at the start of Round 2 is $\langle \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}, \mathbf{f} \rangle = \mu$ over $\mathbb{Z}$. In this case the map π_q in Round 2 is simply replaced by the identity map, $\mathbb{F}_q$ is replaced by $\mathbb{Z}$, and q in the range checks of Round 2 denotes $\max_{i \in [\ell_1]} \mathbf{v}_i^{(1)} + 1$. In Phase 3, γ is $\mathbf{v}^{(1)}$.

3. **Round 2: transforming (34) to ℓ_2 integer constraints with bounded integers.**

- The prover computes and sends integers $\mu_1, \dots, \mu_{\ell_2} \in [0 \dots (q-1) \cdot \ell_1]$ such that, over $\mathbb{Z}$,

$$\langle \mathbf{f}_{*j}, \pi_q^{-1}(\mathbf{v}^{(1)}) \rangle = \mu_j \quad \text{for all } j \in [\ell_2]. \quad (35)$$

- The verifier checks that $\mu_j \in [0 \dots (q-1) \cdot \ell_1]$ for all $j \in [\ell_2]$, and that

$$\sum_{j \in [\ell_2]} \pi_q(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } \mathbb{F}_q. \quad (36)$$

Next, prover and verifier will focus on certifying that (35) holds for all $j \in [\ell_2]$.

4. **Phase 3: Transforming the claim into a PESAT claim over $\mathbb{F}_{2^\nu}$, and invoking the IOR Π_{red} .**

- Prover and verifier compute and define

$$\gamma = \pi_q^{-1}(\mathbf{v}^{(1)}) \in [0 \dots q-1]^{\ell_1} \subseteq \mathbb{Z}^{\ell_1},$$

and set to prove the following claim:

$$\prod_{i \in [\ell_1]} g^{\gamma_i \cdot \mathbf{f}_{ij}} = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2].$$

This claim is equivalent to

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2]. \quad (37)$$

Notice that

$$(\mathbf{x} = ((g^{\gamma_i})_{i \in [\ell_1]}, (g^{\mu_j})_{j \in [\ell_2]}), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{Bit}\mathbb{Z}}, \mathcal{C}, \text{Pack}_\nu). \quad (38)$$

Let $\boldsymbol{\mu} := (\mu_1, \dots, \mu_{\ell_2})$, $g^\gamma := (g^{\gamma_i})_{i \in [\ell_1]}$ and $g^\mu := (g^{\mu_j})_{j \in [\ell_2]}$.

- To prove (38), prover and verifier engage in the IOR Π_{red} ¹¹ with input

$$(\mathbf{x} = (g^\gamma, g^\mu), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})).$$

At the end of this IOR, they are left with the following claim to prove:

$$((\mathbf{z}, c), f, \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{u}, \mu) \in \mathbb{F}_{2^\nu}^\ell \times \mathbb{F}_{2^\nu}, \\ \mathbf{y} = g \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{g} \in \mathbb{F}_2^\ell \end{array} \right) \middle| \begin{array}{l} g = \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu(\mathbf{g}) \\ \langle \mathbf{u}, \mathbf{g} \rangle = \mu \end{array} \right\} \quad (39)$$

for some vector $\mathbf{z} \in \mathbb{F}_{2^\nu}^\ell$ and field element $c \in \mathbb{F}_{2^\nu}$ output during the IOR Π_{red} .

5. **Phase 4: Using an IOP over $\mathbb{F}_{2^\nu}$.** Prover and verifier engage in the IOP Π_{iopp} to certify the claim (39). If Round 0 (Out Of Domain sampling) triggered, they also use Π_{iopp} to certify the evaluation claim from Round 0.

In Table 5 we summarize the main costs of Construction 4.4.

¹¹Recall that, in our instantiation, Π_{red} is the GKR protocol applied to multiple grand products, cf. Section 6.

Costs ($k \leq 1$). In Table 5 we summarize the costs of Construction 4.4 beyond those of Π_{red} and Π_{iopp} (cf. Sections 4.7 and 6), for the specific case $k \leq 1$, corresponding to R being of the form $\mathbb{Z}_n$ or $\mathbb{Z}_n[X]/(h(X))$ for some $n \geq 0$ and some $h(X)$. In our instantiation Section 4.7. In our experiments we only use $k = 0$, but future work of ours on proving lattice computations requires $k = 1$. The number of rounds is between $t_{\text{red}} + t_{\text{iopp}}$ and $2 + t_{\text{red}} + t_{\text{iopp}}$, depending on whether Rounds 0 and 1 execute or not, where $t_{\text{red}}, t_{\text{iopp}}$ denote the number of rounds of $\Pi_{\text{iopp}}, \Pi_{\text{red}}$.

Rounds 0, 1 and 2 send only non-oracle messages, so the oracles and queries of Construction 4.4 are exclusively those of Π_{red} and Π_{iopp} . If Round 1 is skipped, q denotes $|R|$ when $R = \mathbb{F}_q$, and $\max_i \mathbf{v}_i^{(1)} + 1$ when $R = \mathbb{Z}$ (cf. Remark 4.5). The exponentiations of Phase 3, when computing $(g^{\gamma_i})_{i \in [\ell_1]}$ and $(g^{\mu_j})_{j \in [\ell_2]}$, take $\lceil \log(q \cdot \ell_1)/w \rceil \leq \nu/w$ multiplications each using a fixed-base table with w -bit windows ($w = 8$ in our implementation).

In our instantiation, we set $\ell_1 \approx \sqrt{\ell}$, $\ell_2 \approx \sqrt{\ell}$, making the verifier succinct. We assume implicitly that the instance is given to the verifier as $(\mu, \mathbf{v}^{(1)}, \mathbf{v}^{(2)})$, so that it does not read $\mathbf{v}$ in full.

When $k = 0$ (so that $R = \mathbb{Z}_n$ for some n), we have $D = 0$ and $T = 1$. When $k = 1$ and $R = \mathbb{Z}_n[X]/(h(X))$ for some monic $h(X)$, then $D \leq \deg(h(X)) - 1$ and $T \leq D + 1$, cf. (32) for a definition of these quantities. If $n > 0$, then $N_R \leq |\mathbb{Z}_n| = n$. Otherwise if $n = 0$, $N_R \leq \|(\mathbf{v}^{(1)}, \mathbf{v}^{(2)})\|_\infty$.

Round 0	Prover Verifier Proof (bits)	$O(\ell/\nu) \cdot \mathbb{F}_{2^\nu}$ – ν
Round 1	Prover Verifier Proof (bits)	$O(\ell \cdot (D + 1))$ additions of $\log(\ell_1 \cdot N_R)$ -bit integers $O(\ell_2 \cdot (D + 1)^2)$ additions of $\log(\ell \cdot N_R^2)$ -bit integers $O(\ell_2 \cdot (D + 1)^2)$ multiplications of $\log(\ell \cdot N_R^2)$ -bit integers $O((\ell_1 + \ell_2) \cdot (D + 1))$ reductions mod q of $\log(N_R)$ -bit integers $O((\ell_1 + \ell_2) \cdot (D + 1)) \cdot \mathbb{F}_q$ $O(D + 1)$ reductions mod $\text{char}(R)$ of $\log((D + 1) \cdot \ell \cdot N_R^2)$ -bit integers $O(D + 1) \cdot R$ $O((\ell_1 + \ell_2) \cdot (D + 1))$ reductions mod q of $\log(N_R)$ -bit integers $O((\ell_1 + \ell_2) \cdot (D + 1)) \cdot \mathbb{F}_q$ $(2D + 1) \cdot \log((D + 1) \cdot \ell \cdot N_R^2)$
Round 2	Prover Verifier Proof (bits)	$O(\ell)$ additions of $\log(q \cdot \ell_1)$ -bit integers $O(\ell_2)$ reductions mod q of $\log(q \cdot \ell_1)$ -bit integers $O(\ell_2) \cdot \mathbb{F}_q$ $\ell_2 \cdot \log(q \cdot \ell_1)$
Phase 3	Prover Verifier Proof (bits)	$O((\ell_1 + \ell_2) \cdot \nu/w) \cdot \mathbb{F}_{2^\nu}$ same as prover –

Table 5: Costs of Construction 4.4 when $k \leq 1$, in addition to those of Π_{red} and Π_{iopp} . By $N \cdot \mathbb{F}_{2^\nu}$, $N \cdot \mathbb{F}_q$, $N \cdot R$ we indicate N operations in the respective ring. Rounds 0 and 1 are optional.

We justify the costs listed in Round 1 of Table 5, which are arguably the most involved to derive. The first three prover costs in Round 1 come from the computation of μ' . The prover can compute μ' by first building, for each $j \in [\ell_2]$, the polynomial $h_j := \sum_{i \in [\ell_1]} \mathbf{f}_{ij} \cdot \pi_{\text{canon}}^{-1}(\mathbf{v}_i^{(1)})$, which requires

no multiplications since $\mathbf{f}$ has bit entries. The coefficients of these polynomials are bounded by $\ell_1 \cdot N_R$. Hence the h_j can be computed with $O(\ell \cdot (D + 1))$ additions of $\log(\ell_1 \cdot N_R)$ -bit integers. Now, to obtain μ' the prover computes $\sum_{j \in [\ell_2]} \pi_{\text{canon}}^{-1}(\mathbf{v}_j^{(2)}) \cdot h_j$. Each product in this sums costs at most $(D + 1)^2$ multiplications of $\log(\ell_1 \cdot N_R)$ -bit integers, and there are ℓ_2 of them, this yields the third cost in the prover's Round 1. The second cost correspond to adding together each of the terms in $\sum_{j \in [\ell_2]} \pi_{\text{canon}}^{-1}(\mathbf{v}_j^{(2)}) \cdot h_j$.

The last two cost of the prover and verifier in Round 1 are the costs of computing $\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(i)})$ for $i = 1, 2$. For both i , this is the cost of reducing $D + 1$ integers in the range $[0..N_R - 1]$ modulo a prime, and then computing a linear combination of the resulting elements.

The first two verifier costs in Round 1 correspond to computing $\pi_{\text{canon}}(\mu')$. For that, it first reduces the at most $2D + 1$ coefficients of μ' (which are integers of at most $\log(T \cdot \ell \cdot N_R^2)$ bits) modulo $\text{char}(R)$ and evaluates the resulting polynomial at η_1 in R using Horner's rule, and then compares the result with μ .

Remark 4.6 (Succinct verifier and proofs). By Table 5, choosing $\ell_1 = O(\sqrt{\ell})$ and $\ell_2 = O(\sqrt{\ell})$, the verifier and the proof size of Construction 4.4 have asymptotic cost $O(\sqrt{\ell})$ (where ℓ is the bit-length of the witness).

4.4 Soundness and completeness

For readability, we temporarily denote

$$\mathcal{R}_{\text{PESAT}} = \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu), \quad \mathcal{R}_{\text{Lin}} = \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu).$$

Theorem 4.7 (Completeness and round-by-round knowledge soundness of Construction 4.4). *Let $\delta \in (0, 1)$ be a proximity parameter within the unique decoding radius of $\mathcal{C}$ (cf. Section 3.2), and assume that Round 0 (OOD sampling) of Construction 4.4 is skipped. We treat the general case in Section A.2. Assume*

$$\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1.$$

Assume Π_{red} is a perfectly complete t_{red} -round IOR from $\mathcal{R}_{\text{PESAT}}$ to $\mathcal{R}_{\text{Lin}}$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PESAT}}^\delta, \mathcal{R}_{\text{Lin}}^\delta)$, with errors $(\kappa_{\text{red}1}, \dots, \kappa_{\text{red}t_{\text{red}}})$. Assume Π_{iopp} is a perfectly complete t_{iopp} -round IOP for $\mathcal{R}_{\text{Lin}}$ (i.e. an IOR from $\mathcal{R}_{\text{Lin}}$ to the trivial relation $\perp$) with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin}}^\delta, \perp)$ (see Section 3.3), and errors

$$\kappa_{\text{iopp}1}, \dots, \kappa_{\text{iopp}t_{\text{iopp}}}.$$

Then Construction 4.4 is a perfectly complete $(1 + t_{\text{red}} + t_{\text{iopp}})$ -round IOP for $\mathcal{R}_{\text{LinBitsRings}}$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{LinBitsRings}}^\delta, \perp)$, with errors

$$\kappa_0, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]},$$

where

$$\kappa_0 = \theta(\mathcal{P}, \mathcal{M}, \ell)$$

is the quantity defined in (33)¹². In particular, if the sets $\mathcal{M}_q$ are as in Lemma 4.3, then one can take

$$\kappa_0 = \frac{2 \cdot D}{q_{\min}} + \frac{\log(2 \cdot T \cdot \ell \cdot N_R^2)}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor},$$

¹²Note that $\theta(\mathcal{P}, \mathcal{M}, \ell)$ depends on the public input $\mathbf{x}$. In practice one can obtain a uniform bound on κ_0 by taking the maximum $\theta(\mathcal{P}, \mathcal{M}, \ell)$ across all public inputs that are admitted by the protocol.

where $q_{\min}$ is the smallest prime in $\mathcal{P}$ and D, T, N_R are as in (32). If Round 1 is not executed, the error term κ_0 and its round are absent, and [Construction 4.4](#) has $t_{\text{red}} + t_{\text{iopp}}$ rounds.

To prove [Lemma 4.3](#) we will use the following result:

Lemma 4.8. *Let $h \in \mathbb{Z}[X_1, \dots, X_m]$ be a nonzero polynomial of total degree $\leq d$ and with $\|h\|_\infty \leq M$ for some $M \geq 1$. Let $\mathcal{P}$ be a set of primes, and let $q_{\min}$ be the smallest prime in $\mathcal{P}$. Given a prime q and $\alpha \in \mathbb{F}_q^m$, write $\pi_{q,\alpha}(h) := \pi_q(h)(\alpha) \in \mathbb{F}_q$. Then*

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q^m}} [\pi_{q,\alpha}(h) = 0] \leq \frac{\log M}{\lfloor \log(q_{\min}) \rfloor \cdot |\mathcal{P}|} + \frac{d}{q_{\min}}.$$

Proof. Let $\mathcal{E}_{\text{prime}}$ be the event that all coefficients of h are multiples of q . Then, by the Schwartz–Zippel lemma,

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q^m}} [\pi_{q,\alpha}(h) = 0 \mid \neg \mathcal{E}_{\text{prime}}] \leq \frac{d}{q_{\min}}.$$

Now, every nonzero coefficient c of h is divisible by at most $\log M / \log q_{\min} \leq \log M / \lfloor \log(q_{\min}) \rfloor$ primes from $\mathcal{P}$, because the product of m' distinct such primes divides c , hence $q_{\min}^{m'} \leq |c| \leq M$. Hence, the probability that $\mathcal{E}_{\text{prime}}$ occurs is at most $(\log M / \lfloor \log(q_{\min}) \rfloor) / |\mathcal{P}|$, and the lemma follows. $\square$

Proof of Lemma 4.3. Let $h \in \mathbb{Z}[X_1, \dots, X_k]$ be a polynomial of total degree at most $2 \cdot D$, with $\|h\|_\infty \leq 2 \cdot T \cdot \ell \cdot N_R^2$, and with $\pi_{\text{canon}}^R(h) \neq 0$ (cf. the definition of $\theta(\mathcal{P}, \mathcal{M}, \ell)$). In particular h is nonzero, and $\|h\|_\infty \leq M := 2 \cdot T \cdot \ell \cdot N_R^2$. Since h is nonzero and has total degree at most $2 \cdot D$, [Lemma 4.8](#) (applied with $m = k$ variables and total degree $2 \cdot D$) yields

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q^k}} [\pi_q(h)(\alpha) = 0] \leq \frac{\log M}{\lfloor \log(q_{\min}) \rfloor \cdot |\mathcal{P}|} + \frac{2 \cdot D}{q_{\min}},$$

as needed. $\square$

Proof of Theorem 4.7. The completeness of [Construction 4.4](#) is clear from its description [Albert: Review](#). Recall that, by hypothesis, Round 0 (OOD sampling) is skipped; the case where Round 0 is executed is treated in [Section A.2](#).

We decompose [Construction 4.4](#) into three IORs Π_1, Π_2, Π_3 . The first one, Π_1 , is an IOR from $\mathcal{R}_{\text{LinBitsRings}}$ to $\mathcal{R}_{\text{PESAT}}$, and it consists of applying Rounds 1 and 2 of [Construction 4.4](#), and outputting

$$(\mathbf{x}' = (g^\gamma, g^\mu), \mathbf{y}' = f, \mathbf{w}' = \pi_2(\mathbf{f})),$$

as described in Phase 3 of [Construction 4.4](#). Formally, Π_1 is a 2-round IOR whose second verifier message is empty: the prover first sends μ' , the verifier replies with (q, π_{rand}) , and the prover then sends $\mu = (\mu_1, \dots, \mu_{\ell_2})$. The second IOR, Π_2 , is simply Π_{red} , reducing $\mathcal{R}_{\text{PESAT}}$ to $\mathcal{R}_{\text{Lin}}$. Finally, the third IOR, Π_3 , is simply the IOP Π_{iopp} for $\mathcal{R}_{\text{Lin}}$.

Below we show that Π_1 has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{LinBitsRings}}^\delta, \mathcal{R}_{\text{PESAT}}^\delta)$, with errors $(\kappa_0, 0)$, assuming Round 1 is not skipped. The second verifier message of Π_1 is empty, hence the last prover message of Π_1 can be sent together with the first prover message of Π_2 , and thus [Construction 4.4](#) has $1 + t_{\text{red}} + t_{\text{iopp}}$ rounds. The theorem then follows from the composition of IORs result (cf. [Lemma 3.7](#)).

At the end of the proof we discuss the case when Round 1 is skipped.

Completeness and soundness of Π_1 We describe a knowledge state function KState . Let $\mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell$ and $\mathbf{y} = f \in \mathbb{F}_2^m$. Let tr be a partial interaction transcript for Π_1 . We start by setting KState so that the first condition in [Definition 3.5](#) is met. Precisely, if $\text{tr} = \emptyset$, we set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{LinBitsRings}}^\delta$. Further, if $\text{tr} = (\mu')$ consists of a single prover message, we set $\text{KState}(\mathbf{x}, \mathbf{y}, \mu', \mathbf{w}) = \text{KState}(\mathbf{x}, \mathbf{y}, \emptyset, \mathbf{w})$, so that the second condition in [Definition 3.5](#) holds for the first prover message round.

After the verifier's message, tr is as follows:

$$\text{tr} = (\mu', (q, \pi_{\text{rand}})) \in \mathbb{Z}[X_1, \dots, X_k] \times (\mathcal{P} \times \mathcal{M}_q).$$

We define $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if the verifier's degree and range checks on μ' in Round 1 pass, $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$, $\mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in (\{0, 1\}^{\ell_1})^{\ell_2}$, and

$$\begin{aligned} \pi_{\text{canon}}(\mu') &= \mu, \\ \langle \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \otimes \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}), \pi_q(\mathbf{f}) \rangle &= \pi_{\text{rand}}(\mu') \quad \text{over } \mathbb{F}_q, \\ \Delta(f, \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) &< \delta. \end{aligned} \tag{40}$$

We continue with our definition of KState . Let $\text{tr} = (\mu', (q, \pi_{\text{rand}}), \mu)$ for a prover message $\mu = (\mu_1, \dots, \mu_{\ell_2}) \in [0..(q-1) \cdot \ell_1]^{\ell_2}$, or let tr be the full transcript obtained by appending the empty second verifier message to $(\mu', (q, \pi_{\text{rand}}), \mu)$. We set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if the verifier accepts on tr (i.e. its checks in Rounds 1 and 2 pass) and $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^\delta$, where $(\mathbf{x}', \mathbf{y}')$ is the output of the verifier, namely

$$\mathbf{x}' = (g^\gamma, g^\mu), \quad \mathbf{y}' = f, \quad \text{with } \gamma = \pi_q^{-1} \left(\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \right) \in [0..q-1]^{\ell_1} \subseteq \mathbb{Z}^{\ell_1}.$$

Thus KState satisfies the third condition in [Definition 3.5](#).

We define an extractor as $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = \mathbf{w}$. It remains to prove two statements: first, that

$$\Pr_{q, \pi_{\text{rand}}} \left[\exists \mathbf{w} \left| \begin{array}{l} \text{KState}(\mathbf{x}, \mathbf{y}, \mu', \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1 \end{array} \right. \right] \leq \kappa_0;$$

and second, that KState satisfies the second condition in [Definition 3.5](#) for transcripts where the prover is about to send μ (its second message).

We start with the first statement. Assume

$$\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1 \quad \text{and} \quad \text{KState}(\mathbf{x}, \mathbf{y}, \mu', \mathbf{w}) = \text{KState}(\mathbf{x}, \mathbf{y}, \mathbf{w}) = 0.$$

The latter occurs if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^\delta$. The former implies that $\mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^\ell$ satisfies (40); in particular $\Delta(f, \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta$, and hence, since $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^\delta$, necessarily

$$\langle \pi(\mathbf{f}), \mathbf{v} \rangle \neq \mu \quad \text{over } R. \tag{41}$$

Define the polynomial

$$h := \langle \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}), \mathbf{f} \rangle - \mu' \in \mathbb{Z}[X_1, \dots, X_k].$$

Since π_{canon} is a ring homomorphism satisfying $\pi_{\text{canon}} \circ \pi_{\text{canon}}^{-1} = \text{id}$ and restricts to π on $\mathbb{Z}$, the first condition in (40) gives

$$\pi_{\text{canon}}(h) = \langle \pi(\mathbf{f}), \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} \rangle - \pi_{\text{canon}}(\mu') = \langle \pi(\mathbf{f}), \mathbf{v} \rangle - \mu,$$

which is nonzero by (41). In particular $h \neq 0$. On the other hand, since π_{rand} is a ring homomorphism restricting to π_q on $\mathbb{Z}$, the second condition in (40) states precisely that $\pi_{\text{rand}}(h) = 0$. We claim that h has total degree at most $2 \cdot D$ and satisfies

$$\|h\|_{\infty} \leq 2 \cdot T \cdot \ell \cdot N_R^2.$$

Indeed, the lifts $\pi_{\text{canon}}^{-1}(\mathbf{v}_i^{(1)})$ and $\pi_{\text{canon}}^{-1}(\mathbf{v}_j^{(2)})$ have total degree at most D and coefficients of absolute value at most N_R , so each product $\pi_{\text{canon}}^{-1}(\mathbf{v}_i^{(1)}) \cdot \pi_{\text{canon}}^{-1}(\mathbf{v}_j^{(2)})$ has total degree at most $2 \cdot D$ and coefficients of absolute value at most $T \cdot N_R^2$ by the definition of T . Further, by the verifier's checks in Round 1,

$$\|\mu'\|_{\infty} \leq T \cdot \ell \cdot N_R^2$$

and the total degree of μ' is at most $2 \cdot D$. The claim follows now from the fact that the entries of $\mathbf{f}$ are bits and there are ℓ of them.

Since additionally $\pi_{\text{canon}}(h) \neq 0$, the polynomial h belongs to the class of polynomials over which $\theta(\mathcal{P}, \mathcal{M}, \ell)$ is defined. Finally, since δ is within the unique decoding radius of $\mathcal{C}$, the code $\mathcal{C}$ is $(\delta, 1)$ -list-decodable, and hence there is at most one $\mathbf{w}$ such that $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\mathbf{w})) < \delta$. Therefore [Albert: review](#)

$$\begin{aligned} & \Pr_{q, \pi_{\text{rand}}} [\exists \mathbf{w} \text{ such that } (\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^{\delta} \text{ and } \text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1] \\ & \leq \Pr_{q \leftarrow \mathcal{P}, \pi_{\text{rand}} \leftarrow \mathcal{M}_q} [\pi_{\text{rand}}(h) = 0] \leq \theta(\mathcal{P}, \mathcal{M}, \ell) = \kappa_0, \end{aligned}$$

where the last inequality holds by the definition of $\theta(\mathcal{P}, \mathcal{M}, \ell)$.

It remains to prove the second statement, namely: if $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 0$, then $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}}), \boldsymbol{\mu}), \mathbf{w}) = 0$ for every prover message $\boldsymbol{\mu} = (\mu_1, \dots, \mu_{\ell_2}) \in [0..(q-1) \cdot \ell_1]^{\ell_2}$. Suppose towards a contradiction that $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}}), \boldsymbol{\mu}), \mathbf{w}) = 1$, i.e. that the verifier accepts and $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^{\delta}$. Then $\mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^{\ell}$, $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\mathbf{f})) < \delta$, and, for all $j \in [\ell_2]$,

$$g^{\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle} = \prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j},$$

and hence

$$\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle = \mu_j \pmod{\text{ord}(g)},$$

where $\text{ord}(g)$ denotes the multiplicative order of g . Now, since $\gamma_i \in [0..q-1]$, $\mu_j \in [0..(q-1) \cdot \ell_1]$, and $\mathbf{f}_{*j}$ contains bits, we have $|\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle - \mu_j| \leq \ell_1 \cdot (q-1)$. Since g is a generator of $\mathbb{F}_{2^{\nu}}^*$ and $q \leq \max \mathcal{P} < (|\mathbb{F}_{2^{\nu}}| - 1)/\ell_1$, we have

$$\text{ord}(g) = |\mathbb{F}_{2^{\nu}}| - 1 > \ell_1 \cdot \max \mathcal{P} \geq \ell_1 \cdot q > \ell_1 \cdot (q-1),$$

and hence

$$\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle = \mu_j \quad \text{over } \mathbb{Z}, \quad \text{for all } j \in [\ell_2]. \quad (42)$$

Applying π_q to (42), and using that $\pi_q(\gamma) = \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)})$, we obtain

$$\langle \pi_q(\mathbf{f}_{*j}), \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \rangle = \pi_q(\mu_j) \quad \text{over } \mathbb{F}_q, \quad \text{for all } j \in [\ell_2].$$

Since the verifier accepts, its check (36) in Round 2 passes, i.e. $\sum_{j \in [\ell_2]} \pi_q(\mu_j) \cdot (\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}))_j = \pi_{\text{rand}}(\mu')$. Now

$$\begin{aligned} & \langle \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \otimes \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}), \pi_q(\mathbf{f}) \rangle \\ &= \sum_{j \in [\ell_2]} \langle \pi_q(\mathbf{f}_{*j}), \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \rangle \cdot \left(\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}) \right)_j \\ &= \sum_{j \in [\ell_2]} \pi_q(\mu_j) \cdot \left(\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}) \right)_j = \pi_{\text{rand}}(\mu'). \end{aligned}$$

Hence the second condition in (40) holds. The first condition in (40), i.e. $\pi_{\text{canon}}(\mu') = \mu$, holds because the verifier accepts, and the third one, i.e. $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\mathbf{f})) < \delta$, holds because $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^{\delta}$. Hence $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1$, a contradiction.

The case where Round 1 is skipped. *Albert: Review that I'm not missing any important argument* In this case, the messages μ' and (q, π_{rand}) and the error κ_0 disappear, and essentially the arguments above apply after replacing $\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}$ with the identity map on $\mathbb{F}_q$ or on $\mathbb{Z}$. In the latter case, π_q is also replaced by the identity map, cf. Remark 4.5. □

4.5 The general LinBitsRings relation

We now proceed to describe the full relation we seek to work with in this section. Compared to $\mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C})$, the witness $\mathbf{f}$ will now have entries in an arbitrary finitely generated ring S , instead of in $\{0, 1\} \subseteq \mathbb{Z}$. These entries will be restricted to a bounded subset of S , so that each witness can be expressed as a pre-specified linear expression (over S) of a string of bits.

We set the following additional notation:

- Let S be a ring, possibly different than R , let $\psi : S \rightarrow R$ be a ring homomorphism.
- Let $D_S \geq 1$ be a degree bound, let $1, \eta_1, \dots, \eta_{k_S}$ be a generating set of S (Section 3.1), and let $\gamma_1 = 1, \gamma_2, \dots, \gamma_d \in S$ be the distinct elements of the form

$$\eta_1^{a_1} \cdots \eta_{k_S}^{a_{k_S}}$$

with $a_1, \dots, a_{k_S} \geq 0$ and $a_1 + \dots + a_{k_S} < D_S$, i.e. the monomials of total degree less than D_S in the generators, so that d is the number of such monomials. Concretely $d \leq \binom{D_S - 1 + k_S}{k_S}$.

- Recall from Section 3.1 that $S_{\leq B}^{< D_S}$ consists of the sums $\sum_{i \in [d]} h_i \cdot \gamma_i$ with $h_i \in [0..2^B - 1]_S$, and that $\mathbf{w}_S$ denotes the projection of an integer vector $\mathbf{w}$ to S (see (31)).
- Let ℓ_0 be a vector witness length, so that our witnesses are $\mathbf{f} \in S^{\ell_0}$.

We will use the natural homomorphism $\pi_{\text{char}(S)} : \mathbb{Z} \rightarrow S$ consisting in reducing an element modulo the characteristic of S , see (30) in Section 3.1.

Definition 4.9 (Bitification of the witness). Let $\mathbf{f} \in (S_{\leq B}^{<D_S})^{\ell_0}$. Let ℓ be a vector length, called *bitification length*. Let $L \in S^{\ell_0 \times \ell}$ be a matrix, called *bitification matrix*. Let $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ be a vector filled with integer bits. We say $\mathbf{g}$ is a *bitification* of $\mathbf{f}$ (with respect to L) if

$$\mathbf{f} = L \cdot \mathbf{g}_S,$$

where for a vector $\mathbf{w}$ of integer bits, $\mathbf{w}_S$ denotes its natural projection to a vector with entries the elements $\{0, 1\}_S \subseteq S$. Additionally, we require that¹³

$$L \cdot \mathbf{g}_S \in (S_{\leq B}^{<D_S})^{\ell_0} \quad \text{for all } \mathbf{g} \in \{0, 1\}^\ell.$$

This condition holds for the standard bitification matrix below.

The choice of L and a method for building a bitification of $\mathbf{f}$ can be customized. Below, we describe what we call the *standard bitification* procedure. This can always be used for arbitrary finitely generated rings S , and is suitable when $\mathbf{f}$ has entries of more or less uniform bit size. In some scenarios (see [Remark 4.11](#)) one may choose different more convenient approaches.

For a fixed L and $\mathbf{f}$, bitifications of $\mathbf{f}$ are not unique in general. Nevertheless, abusing the notation, we often denote them as

$$\text{bits}(\mathbf{f}).$$

A *standard bitification* of $\mathbf{f} \in (S_{\leq B}^{<D_S})^{\ell_0}$, is obtained as follows: write $\mathbf{f} = \sum_{j \in [d]} \mathbf{f}_{*j} \cdot \gamma_j$ for some $\mathbf{f}_{*j} \in ([0..2^B - 1]_S)^{\ell_0}$, $j \in [d]$. Then write

$$\mathbf{f}_{*j} = \sum_{k=1}^B \mathbf{f}_{*jk} \cdot \pi_{\text{char}(S)}(2^{k-1})$$

for some vectors

$$\mathbf{f}_{*jk} \in \{0, 1\}_S^{\ell_0} \subseteq S^{\ell_0},$$

for all j, k . We define

$$\begin{aligned} \ell &:= \ell_0 \cdot d \cdot B, \\ \text{bits}(\mathbf{f}) &\in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{aligned}$$

be the vector with integer bit entries obtained by concatenating the vectors $\mathbf{f}_{*jk}$ and lifting and reordering their entries to $\{0, 1\} \subseteq \mathbb{Z}$ in a natural way, i.e.

$$\text{bits}(\mathbf{f}) := (\text{lift}(\mathbf{f}_{ijk}))_{i \in [\ell_0], k \in [B], j \in [d]} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell, \quad (43)$$

where lift is the identity function if $\text{char}(S) = 0$, and $\text{lift} = \pi_{\text{char}(S)}^{-1}$ if $\text{char}(S) > 0$. Note that the vectors $\mathbf{f}_{*j}$ are not uniquely defined, hence the expression "a bitification" instead of "the bitification".

Defining

$$L = \text{Id}_{\ell_0 \times \ell_0} \otimes (1, 2, \dots, 2^{B-1})_S \otimes (\gamma_1, \dots, \gamma_d) \in S^{\ell_0 \times \ell},$$

we have

$$\mathbf{f} = L \cdot \text{bits}(\mathbf{f})_S.$$

¹³This condition is required to guarantee that extractors obtain witnesses in the specific subset $S_{\leq B}^{<D_S}$ of S . In some applications, only proving that the prover knows a witness in S may suffice (even if the relation specifies witnesses with entries $S_{\leq B}^{<D_S \ell_0}$), and in that case the condition can be dropped.

Again, let $\mathcal{C}$ be a linear code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and blocklength m .

Definition 4.10. The *general LinBitsRings* relation is defined as

$$\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L) := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in R \times R^{\ell_0}, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in (S_{\leq B}^{< D_S})^{\ell_0} \end{array} \right) \left| \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})), \\ \langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu \quad \text{in } R, \\ \mathbf{f} = L \cdot (\text{bits}(\mathbf{f}))_S \text{ for some } \text{bits}(\mathbf{f}) \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right. \right\}.$$

When there is no risk of confusion, we denote $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ simply by $\mathcal{R}_{\text{LinBitsRings}}$. The Core LinBitsRings relation is precisely $\mathcal{R}_{\text{LinBitsRings}}$ after taking $S = \mathbb{Z}$, $B = 1$, $\psi = \pi_{\text{char}(R)}$, and $L = \text{Id}_{\ell \times \ell}$ (the standard bitification matrix in this case, so that $\ell_0 = \ell$).

For $\delta \in (0, 1)$, we let $\mathcal{R}_{\text{LinBitsRings}}^\delta$ be the relation $\mathcal{R}_{\text{LinBitsRings}}$ after replacing the constraint $f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f}))$ by $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f}))) < \delta$, as in [Section 4.1](#).

Remark 4.11 (Non-uniform bit-decomposition sizes). In practical settings, it is possible that not all entries of $\mathbf{f}$ have the same bit size. For example, some chunks of $\mathbf{f}$ may be comprised of elements from $\{0, 1\}_S$, while others may be comprised of generic ring elements. However, the standard bitification procedure from [Definition 4.9](#) allocates $d \cdot B$ bits for every entry of $\mathbf{f}$, no matter its size. To avoid this, one can find alternative approaches to defining L and $\text{bits}(\mathbf{f})$.

For efficiency purposes, one needs to make sure that L has enough structure to later support tensor decomposing certain vectors, as discussed in [Remark 4.14](#). In any case, one can always use the matrix L above, which in general does possess enough structure as argued in [Remark 4.14](#).

See [Remark 2.3](#) for comments on $\mathcal{R}_{\text{LinBitsRings}}$ having (generalized) range checks in its definition and about BitZ proving them, in a sense, “for free”.

4.6 The BitZ IOP for the general LinBitsRings relation

Next we describe our IOP for the general relation $\mathcal{R}_{\text{LinBitsRings}}$. We assume the notation of [Section 4.5](#). Let L be a *bitification* matrix as in [Definition 4.9](#), with $L \in S^{\ell_0 \times \ell}$ so that ℓ is the vector length of the vector $\text{bits}(\mathbf{f})$. For the standard bitification matrix $L = \text{Id}_{\ell_0 \times \ell_0} \otimes (1, \dots, 2^{B-1})_S \otimes (\gamma_1, \dots, \gamma_d)$, we have $\ell = \ell_0 \cdot d \cdot B$. Let further ℓ_1, ℓ_2 be tensor decomposition parameters (to be used in [Construction 4.4](#)) such that $\ell = \ell_1 \cdot \ell_2$. We assume that $\mathcal{C}$ is a linear code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν . We will need the following result.

Lemma 4.12. *Let $\mathbf{f} \in (S_{\leq B}^{< D_S})^{\ell_0}$, let $\mathbf{v} \in R^{\ell_0}$, and let $\text{bits}(\mathbf{f})$ be a bitification of $\mathbf{f}$ with respect to L . Then*

$$\begin{aligned} \langle \psi(\mathbf{f}), \mathbf{v} \rangle &= \langle \text{bits}(\mathbf{f})_R, \mathbf{u} \rangle \in R \\ \mathbf{u} &:= \psi(L)^\top \cdot \mathbf{v} \in R^\ell, \end{aligned}$$

where $\top$ denotes transposition. In particular, if $L = \text{Id}_{\ell_0 \times \ell_0} \otimes (1, \dots, 2^{B-1})_S \otimes (\gamma_1, \dots, \gamma_d) \in S^{\ell_0 \times (\ell_0 \cdot B \cdot d)} = S^{\ell_0 \times \ell}$ (cf. [Definition 4.9](#)), then

$$\mathbf{u} = (\mathbf{v} \otimes (1, 2, \dots, 2^{B-1})_R \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d))) \in R^\ell \quad (44)$$

Proof. Rewriting $\langle \psi(\mathbf{f}), \mathbf{v} \rangle$, we obtain by Definition 4.9 and Eq. (29)

$$\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \langle \psi(L \cdot \text{bits}(\mathbf{f})_S), \mathbf{v} \rangle = \langle \psi(\text{bits}(\mathbf{f})_S), \psi(L)^\top \cdot \mathbf{v} \rangle$$

Now $\psi(\text{bits}(\mathbf{f})_S) = \text{bits}(\mathbf{f})_R$ because every entry of $\text{bits}(\mathbf{f})$ consists of bits and ψ is a ring homomorphism, so it sends the 0 and 1 elements of S to the 0 and 1 elements of R . Further,

$$\psi(L) = \text{Id}_{\ell_0 \times \ell_0} \otimes \psi((1, \dots, 2^{B-1})_S) \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d)),$$

and $\psi \circ \pi_{\text{char}(S)}(2^i) = \pi_{\text{char}(R)}(2^i)$ because $\psi \circ \pi_{\text{char}(S)}$ is a ring homomorphism, $\psi \circ \pi_{\text{char}(S)}(2^i) = 2^i \cdot (\psi \circ \pi_{\text{char}(S)}(1))$, hence $\psi \circ \pi_{\text{char}(S)}(2^i)$ is 2^i times the identity element of R , and that is precisely the definition of $\pi_{\text{char}(R)}(2^i)$.

The specific form of $\mathbf{u}$ in (44) is now obtained by direct calculation of $\psi(L)^\top \cdot \mathbf{v}$. $\square$

We assume that the input vector $\mathbf{v}$ for $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$, the matrix L , and the tensor decomposition dimensions are such that $\mathbf{u} = \psi(L)^\top \cdot \mathbf{v}$ admits a tensor decomposition $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ with $\mathbf{u}^{(1)} \in R^{\ell_1}$, $\mathbf{u}^{(2)} \in R^{\ell_2}$, see Remark 4.14 for an explanation that this can be achieved easily in applications, and what to do otherwise.

Construction 4.13 (IOP for $\mathcal{R}_{\text{LinBitsRings}}$). **Parameters.** $(\ell_0, \ell, \ell_1, \ell_2, \Pi_{\text{iopp}}, \Pi_{\text{red}}, \mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}})$ comprised of: a witness vector length ℓ_0 ; a *bitified witness* vector length ℓ ; tensor decomposition dimensions ℓ_1, ℓ_2 for Construction 4.4 satisfying $\ell_1 \cdot \ell_2 = \ell$; IORs and IOPs Π_{iopp} and Π_{red} as in Construction 4.4; and sets of primes and homomorphisms $\mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}}$ again as in Construction 4.4.

Inputs. The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}) \in R \times R^{\ell_0}$ and $\mathbf{y} = f \in \mathbb{F}_2^m$. The prover receives $\mathbf{w} = \mathbf{f} \in (S_{\leq B}^{< D_S})^{\ell_0}$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$, i.e.,

$$f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})), \quad \langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu \quad \text{in } R.$$

for some $\text{bits}(\mathbf{f})$ such that $L \cdot \text{bits}(\mathbf{f})_S = \mathbf{f}$.

1. By Lemma 4.12, the claim $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ is equivalent to the claim

$$(\mathbf{x}' = (\mu, \mathbf{u}), \mathbf{y}' = f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})), \mathbf{w}' = \text{bits}(\mathbf{f})) \in \mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C}). \quad (45)$$

where $\mathbf{u} = \psi(L)^\top \cdot \mathbf{v}$.

By assumption, $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ with $\mathbf{u}^{(1)} \in R^{\ell_1}$, $\mathbf{u}^{(2)} \in R^{\ell_2}$. Then the prover and the verifier use Construction 4.4 to prove (45). They use parameters $(\ell, \ell_1, \ell_2, \Pi_{\text{iopp}}, \Pi_{\text{red}}, \mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}})$. See Remark 4.14 for comments on how to obtain tensor decompositions of $\mathbf{u}$ in Construction 4.4 assuming $\mathbf{v}$ is appropriately tensor-decomposable.

Remark 4.14 (Tensor decompositions of $\mathbf{u}$). Recall that Construction 4.4 considers a (possibly trivial) tensor decomposition of its public input vector $\mathbf{u}$ (denoted $\mathbf{v}$ in Construction 4.4). Moreover, as argued in Sections 2.3 and 6, this tensor decomposition is crucial for performance of BitZ (concretely, we seek $\ell^{0.5} \leq \ell_1 \leq \ell^{0.6}$, cf. Section 6).

In many applications, the input vector $\mathbf{v}$ for $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ is an “eq” vector, i.e. a vector of the form $(\text{eq}(\mathbf{b}; \mathbf{r}))_{\mathbf{b} \in \{0, 1\}^{\log \ell_0}}$ for some $\mathbf{r} \in R^{\log \ell_0}$. In this case $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}$ with the dimensions of $\mathbf{v}^{(1)}, \mathbf{v}^{(2)}$ being arbitrary powers of two as long as $|\mathbf{v}^{(1)}| \cdot |\mathbf{v}^{(2)}| = \ell_0$. In that case, if

$$L = \text{Id}_{\ell_0 \times \ell_0} \otimes (1, \dots, 2^{B-1})_S \otimes (\gamma_1, \dots, \gamma_d) \quad (46)$$

(cf. [Definition 4.9](#)), then $\mathbf{u}$ has the form (44), i.e.

$$\mathbf{u} = (\mathbf{v} \otimes (1, 2, \dots, 2^{B-1})_R \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d)))$$

and we may use a tensor decomposition for $\mathbf{u}$ in [Construction 4.4](#) of the form, say, $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$, where

$$\mathbf{u}^{(1)} := \mathbf{v}^{(1)}, \quad \mathbf{u}^{(2)} := \mathbf{v}^{(2)} \otimes (1, 2, \dots, 2^{B-1})_R \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d)).$$

Other decompositions apply. For example one of the tensor factors in $\mathbf{u}^{(2)}$ above could be placed in $\mathbf{u}^{(1)}$, and the vector $(1, 2, \dots, 2^{B-1})_R$ can admit further tensor decomposition, and some of the factors can be placed in $\mathbf{u}^{(1)}$.

For more general matrices L , one needs to make sure that L has enough structure to allow for a favorable tensor decomposition of $\mathbf{u}$. If that is not possible, one can either:

- Fall back to using the matrix (46), which is always available for arbitrary finitely generated rings S with bit-bound B , or
- Alternatively, if the ring R supports it (e.g. if R is a finite field or a ring with a large exceptional set), one can run a sumcheck to reduce the claim $\langle \text{bits}(\mathbf{f})_R, \mathbf{u} \rangle = \mu$ to an MLE evaluation claim over R , thereby “replacing” the vector $\mathbf{u}$ by an “eq vector” that, as such, admits tensor decompositions of arbitrary dimensions.

Costs of [Construction 4.13](#). We discuss the costs of [Construction 4.13](#). The rounds, oracles and queries of [Construction 4.13](#) are exactly those of [Construction 4.4](#) (see [Table 5](#)), which is run on the bitified witness $\text{bits}(\mathbf{f})$ of length $\ell = \ell_0 \cdot d \cdot B$ (if using the standard bitification matrix of [Definition 4.9](#)), and with the weight vector $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ of [Remark 4.14](#). The only additional costs of [Construction 4.13](#) are those of computing the bitification of $\mathbf{f}$ and of computing $\mathbf{u}^{(1)}$ and $\mathbf{u}^{(2)}$. When using the standard bitification matrix, the verifier does not need to compute $\mathbf{u}$ entirely: it suffices to compute $\mathbf{u}^{(1)}, \mathbf{u}^{(2)}$, avoiding a linear cost. In [Table 6](#) we summarize the costs of [Construction 4.13](#).

	Prover	Verifier	Proof (bits)
Bitification of $\mathbf{f}$ (Definition 4.9)	$O(\ell)$ bit op.	–	–
Instance translation $(\mu, \mathbf{v}) \mapsto (\mu, \mathbf{u}^{(1)}, \mathbf{u}^{(2)})$	$O(\ell_2) \cdot R$	same as prover	–
Construction 4.4 with length $\ell = \ell_0 \cdot d \cdot B$	Table 5	Table 5	Table 5

Table 6: Costs of [Construction 4.13](#) for the standard bitification matrix and the tensor decomposition of [Remark 4.14](#). By $N \cdot R$ we indicate N operations in R .

Theorem 4.15. *Under the same hypotheses and notation as in [Theorem 4.7](#) for the parameters $(\ell, \ell_1, \ell_2, \Pi_{\text{iopp}}, \Pi_{\text{red}}, \mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}})$, the ring R , the code $\mathcal{C}$, and the binary field $\mathbb{F}_{2^\nu}$ with packing map Pack_ν , [Construction 4.13](#) is a perfectly complete $(1+t_{\text{red}}+t_{\text{iopp}})$ -round IOP for $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{LinBitsRings}}^\delta, \perp)$ and errors*

$$\kappa_0, (\kappa_{\text{red } i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp } i})_{i \in [t_{\text{iopp}}]},$$

where $\kappa_0 = \theta(\mathcal{P}, \mathcal{M}, \ell)$ is as in [Theorem 4.7](#).

The error term κ_0 and its round are absent if Round 1 of [Construction 4.4](#) is not executed.

If Round 0 of [Construction 4.4](#) (OOD sampling) is executed, then [Construction 4.13](#) has an additional round and the additional OOD error term κ_{OOD} of [Theorem A.1](#).

Proof. Deferred to [Section A.1](#). In short, the result is a consequence of [Lemma 4.12](#), which establishes a correspondence between triples for $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ and $\mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C})$. [Lemma 4.12](#) further provides a method for efficiently computing a corresponding triple from the other. Using the assumption that $L \cdot \mathbf{g} \in (S_{\leq B}^{< D})^{\ell_0}$ for all $\mathbf{g} \in \{0, 1\}^\ell$, this allows to build a round-by-round knowledge soundness state and an extractor for [Construction 4.13](#) from those of [Construction 4.4](#). $\square$

4.7 Instantiation and achieving 100 or 128 bits of soundness

We instantiate [Constructions 4.4](#) and [4.13](#) by letting Π_{red} be an optimized GKR protocol for multiple grand product claims of certain form (cf. [Section 6](#)). The IOP Π_{iopp} is taken to be the Ligerito [\[NA25\]](#) implementation from Flock [\[BRW26\]](#) with minimal modifications (see [Section 7](#)), with the ring switching step from [\[DP24\]](#). The inner product claims when Π_{iopp} is called are not MLE evaluation claims. Since the ring-switch implementation in [\[BRW26\]](#) does not support arbitrary inner product claims, we run a sumcheck to reduce to those to MLE evaluation claims. We instantiate them solely with the following parameters:

Parameter	Choices
R	A finite field (possibly non-prime)
S	Integers $\mathbb{Z}$
π	Reduction mod $\text{char}(R)$
ψ	Reduction mod $\text{char}(R)$
ν	128
$\mathcal{C}$	Reed-Solomon code over $\mathbb{F}_{2^\nu}$ of rate 1/2 or 1/8
Proximity parameter δ	Close to the Johnson bound ¹⁴
L	The standard bitification matrix (Eq. (46))
$\mathcal{P}$	Set of primes. Discussed below
$\mathcal{M}_q = \{\pi_q\}$	Reduction modulo q
ℓ_1, ℓ_2	Discussed below

Table 7: Parameter choices in our deployment of [Constructions 4.4](#) and [4.13](#)

We next provide example instantiations of [Constructions 4.4](#) and [4.13](#) so as to achieve $\lambda = 128$ bits of round-by-round (rbr) knowledge soundness, while keeping the protocols as efficient as possible, with the main challenge being to use $\mathbb{F}_{2^\nu}$ of size 2^{128} for efficiency purpose. We remark that in [Section 7](#) we provide experimental results for 100 bits of rbr knowledge soundness. Achieving $\lambda = 100$ bits of security is relatively straightforward and we do not discuss it: several of the considerations made below can be used for $\lambda = 100$, which much less constraining parameters. We use a careful selection of parameters in combination with grinding. In [Remark 4.16](#) we discuss other instantiations and techniques that do not require grinding.

The contents of this section should be taken as examples of how to instantiate [Constructions 4.4](#) and [4.13](#), not as strict rules. Other configurations may be perfectly plausible and practical.

In terms of security, we need to ensure the following:

¹⁴Except when targeting > 100 bits of security, in which case we use the unique decoding radius

1. The round-by-round knowledge soundness errors of [Construction 4.4](#) κ_{OOD} (if Round 0 is executed), κ_0 (if Round 1 is executed), $(\kappa_{\text{red}i})_i$ and $(\kappa_{\text{iopp}i})_i$ are all at most $2^{-\lambda}$.

In terms of efficiency, the most important step of [Construction 4.4](#) to be aware of is Phase 3, where one uses an IOR Π_{red} to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2], \quad (47)$$

into a linear claim on $\pi(\mathbf{f})$. As mentioned, in our instantiation, Π_{red} is the GKR protocol for multiple grand product claims, equipped with the optimizations discussed in [Section 6](#). This is the most expensive prover part of [Constructions 4.4](#) and [4.13](#), constituting between 45% ($\ell = 2^{20}$) and 85% ($\ell = 2^{30}$) of prover costs excluding the commitment, cf. [Table 13](#). Because of this, it is important for efficiency that operating in $\mathbb{F}_{2^\nu}$ is as efficient as possible.

With this in mind, we target using $\mathbb{F}_{2^\nu}$ with $\nu \approx 128$. A natural candidate is the 2^{128} sized binary field in GHASH representation

$$\mathbb{F}_{2^\nu} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1).$$

Before anything else, we discuss the constraint $\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. First of all, we set ℓ_1, ℓ_2 with $\ell_1 \cdot \ell_2 = \ell$ and

$$\ell_1 \in [\ell^{0.5}, \ell^{0.6}]. \quad (48)$$

In general, choosing $\ell_1 \approx \ell^{0.6}$ can lead to better prover time and proof size for the GKR grand product step, as observed in our experiments. However choosing $\ell_1 \approx \ell^{0.5}$ allows to use slightly smaller soundness errors, which reduces grinding times if targeting 128 bits of security.

We now discuss how to make the errors $\kappa_{\text{OOD}}, \kappa_0, (\kappa_{\text{red}i})_i, (\kappa_{\text{iopp}i})_i$ be all at most $2^{-\lambda}$.

The errors $\kappa_{i,\text{iopp}}$. The ring switching step adds an error term of the form $7/|\mathbb{F}_{2^\nu}|$. If targeting 128 bits of security, we use grinding ([Section 3.3](#)) of 3 bits in this round. The sumcheck before the ring switch adds similarly small error terms, which are handled with a few bits of grinding. WHIR is configured at any desired $\lambda \leq 128$ bits of security using $|\mathbb{F}_{2^\nu}| = 2^{128}$ following [[Fen24a](#), [Fen24b](#)]. We do not discuss this further since WHIR is a well-known IOP, and it can achieve 128 bits of security (in the Unique Decoding Radius) in 128 bit fields when codewords have dimension approximately 2^{30} assuming an interleaved RS code (to reduce the correlated agreement error) and the use of grinding when needed.

The error κ_{OOD} . As mentioned, we instantiate Π_{iopp} as WHIR with the ring-switching techniques of [[DP24](#)]. For $\lambda = 100$ we use a proximity parameter δ up to the Johnson bound. For $\lambda = 128$ we set δ up to the unique decoding radius, so that the Out Of Domain sampling step (Round 0) is skipped.

The errors $\kappa_{i,\text{red}}$. As mentioned, our Π_{red} IOR is the GKR protocol for multiple grand product claims from [Eq. \(47\)](#), see [Section 6](#). The first round of [Section 6](#) has soundness error $\log \ell_2/|\mathbb{F}_{2^\nu}|$. The rest of rounds are either a round of sumcheck for a polynomial of degree 3, or a unification of two evaluation claims of the same multilinear polynomial. Accordingly we have $\kappa_{\text{red}i} \leq 3/|\mathbb{F}_{2^\nu}|$ for all i , except $\kappa_{\text{red}1} \leq \log \ell_2/|\mathbb{F}_{2^\nu}|$ (for δ within the unique decoding radius; otherwise these bounds

are multiplied by L_δ , cf. [Lemma 6.2](#)). This immediately guarantees $\lambda = 100$ bits of security. To achieve $\lambda = 128$ bits of security, one can do a small number of bits of grinding ([Section 3.3](#)) per round.

The error κ_0 . The error κ_0 is perhaps the most delicate, as it demands at the same time that $|\mathcal{P}|$ is large, while still satisfying $\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$.

Recall that in many scenarios κ_0 does not exist in the IOP (i.e. sometimes our IOP skips Round 1). Concretely, if our initial inner product claim is over $R = \mathbb{F}_q$ a prime field of size $< (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, then κ_0 does not appear as an error term, and we do not need to do anything to adjust for it. The term κ_0 does appear in all other scenarios ($R = \mathbb{Z}$ with large or negative entries in $\mathbf{v}^{(1)}$ [Albert: negative?](#) (cf. [Remark 4.5](#)), or R a prime field of larger size, or R a proper field extension). We divide our approaches by whether this first step is skipped or not.

For the purposes of this discussion, we assume BitZ is used within a bigger proof system, as described in [Section 5](#), so that BitZ is used to prove an inner product claim for a committed witness, as the last phase of a bigger proving system where [Albert: review what the technical overview exactly says](#) 1) first the initial constraint is projected to a finite field $\mathbb{F}$, or, alternatively, this step is skipped if the constraint is already over a finite field; and 2) a PIOP is run over $\mathbb{F}$ for the projected constraint, outputting an inner product claim on the projected witness. Now, this inner product claim is the input of [Construction 4.4](#), and we aim to configure [Construction 4.4](#) at 128 bits of security for this setting.

Strategy 1: the PIOP achieves 128 bits of security through grinding and we avoid the error term κ_0 . If allowing, say, 18 – 19 bits of grinding during each round of the PIOP (which adds 0.8 – 1.7ms per round, cf. [Table 3](#)), then, say, a Spartan-like PIOP will achieve 128 bits of security if run on a field $\mathbb{F}$ of size $\approx 2^{112}$ to 2^{113} (since each one of its rounds is either a sumcheck round for a degree ≤ 3 polynomial, or a step that batches some claims with error $1/|\mathbb{F}|$). Hence, if the initial constraint is over $\mathbb{Z}$, then we may project it to a 112-113 bit random prime field $\mathbb{F}$ and run a PIOP over $\mathbb{F}$. We study this case next. Our analysis also applies if the original constraint was already expressed over a prime field of such size.

Note that the projection of the constraint to $\mathbb{F}$ also requires to be set up to 128 bits of security. We discuss this in [Remark 4.17](#).

Let $q = |\mathbb{F}|$. Then if

$$q < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1,$$

then κ_0 is not present as an error term, and there is nothing to adjust. Since $|\mathbb{F}_{2^\nu}| = 2^{128}$ and $q \approx 2^{113}, 2^{112}$, this occurs as long as $\ell_1 \lesssim 2^{15}, 2^{16}$, and using [\(48\)](#), as long as

$$\ell \lesssim 2^{25}, 2^{26} \quad \text{if } \ell_1 \approx \ell^{0.6}$$

and

$$\ell \lesssim 2^{30}, 2^{32} \quad \text{if } \ell_1 \approx \ell^{0.5}.$$

The specific upper bounds depend on how much grinding one is doing at the PIOP level.

Strategy 2: No or essentially no grinding during the PIOP, and κ_0 is present and grinding is used during the first projection step (Round 1) of [Construction 4.4](#). We now assume $\mathbb{F}$ is either a prime field with $|\mathbb{F}| \geq (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, or a proper extension field. In this case [Construction 4.4](#) goes through an initial projection step with soundness error κ_0 , i.e. Round 1 of [Construction 4.4](#).

Recall ([Theorem 4.7](#)) that $\kappa_0 = \theta(\mathcal{P}, \mathcal{M}, \ell)$. We use $\pi_{\text{canon}} : \mathbb{Z}[X] \rightarrow \mathbb{F}$ as in [Example 4.2](#), so that $\pi_{\text{canon}}(h(X)) = \pi_{\text{char}(\mathbb{F})}(h)(x)$ where $\pi_{\text{char}(\mathbb{F})}(h)$ is the polynomial h after reducing its coefficients modulo $\text{char}(\mathbb{F})$, and x is a root of an irreducible polynomial defining $\mathbb{F}_{q^d}$ (if $d = 1$ and so $\mathbb{F}_{q^d}$ is a prime field, then we can use a morphism $\pi_{\text{canon}} : \mathbb{Z} \rightarrow \mathbb{F}_q$). We have $D = d - 1$, $T \leq d$, and $N_{\mathbb{F}} \leq q - 1$ (see (32) and the text below it for a definition of these quantities). We instantiate the morphism sets $\mathcal{M}_q$ as in [Lemma 4.3](#), i.e. π_{rand} reduces coefficients modulo a sampled prime and evaluates X_1 at a random $\alpha \in \mathbb{F}_q$. In this case, by [Lemma 4.3](#),

$$\kappa_0 \leq \frac{2 \cdot (d - 1)}{q_{\min}} + \frac{\log(2 \cdot d \cdot \ell \cdot (q - 1)^2)}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor},$$

where q is the characteristic of $\mathbb{F}$.

We select $\mathcal{P}$ to be the set of all primes in a range $[2^a, 2^b]$. There are approximately

$$|\mathcal{P}| \approx (2^b/b - 2^a/a) / \log_e(2)$$

such primes. Using $b = 128 - \tau \cdot \log(\ell)$, $\ell_1 = 2^{\tau \cdot \log(\ell)}$, for a parameter $\tau \in [0.5..0.6]$ and plugging these values into κ_0 we obtain

$$\begin{aligned} \kappa_0 &\lesssim \frac{2 \cdot (d - 1)}{2^a} + \frac{\log(2 \cdot d \cdot \ell \cdot (q - 1)^2) \cdot \log_e(2)}{\frac{a}{b} \cdot 2^b - 2^a}, \\ b &:= 128 - r_1, \quad r_1 := \tau \cdot \log(\ell). \end{aligned} \tag{49}$$

We calculate several minima for the right hand side of (49) when $\mathbb{F}$ is a field of size $\approx 2^{128}$ (not necessarily a prime field), for varying values of ℓ and $\tau \in \{0.5, 0.6\}$. We include the corresponding grinding times in order to achieve 128 bits of security for this initial step of [Construction 4.4](#).

$\log \mathbb{F} $	d	$\log q$	$\log \ell$	τ	a^*	$\kappa_0 \leq$	Bit-deficit to 128	Grinding time (ms)
128	1	128	24	0.5	110	$2^{-108.3}$	20	3.4
128	1	128	24	0.6	107	$2^{-105.9}$	23	25.9
128	1	128	28	0.5	108	$2^{-106.3}$	22	11.9
128	1	128	28	0.6	105	$2^{-103.5}$	25	98.1
128	1	128	30	0.5	107	$2^{-105.3}$	23	25.9
128	1	128	32	0.5	106	$2^{-104.3}$	24	54.9
128	4	32	24	0.5	114	$2^{-109.2}$	19	1.8
128	4	32	24	0.6	111	$2^{-106.8}$	22	11.9
128	4	32	28	0.5	112	$2^{-107.1}$	21	6.4
128	4	32	28	0.6	109	$2^{-104.4}$	24	54.9
128	4	32	30	0.5	111	$2^{-106.1}$	22	11.9
128	4	32	30	0.6	108	$2^{-103.1}$	25	98.1
128	4	32	32	0.5	110	$2^{-105.1}$	23	25.9

Table 8: Minima for the upper bound of κ_0 in (49) for different configurations of $\mathbb{F}$ (of characteristic q and degree d over its prime field), witness sizes ℓ , and τ such that $\ell_1 = \ell^\tau$. By choosing primes in the interval $[2^{a^*}, 2^{128-r_1}]$, we have that the first error term κ_0 is bounded by the quantity in the $\kappa_0 \leq$ column. We include the bits left required to achieve $\kappa_0 < 1/2^{128}$ and the cost of grinding such amount of bits in a MacBook Air M5 24 GB multi-threaded, using Blake3 as hash function.

Remark 4.16 (128 bits of security for larger witness sizes ℓ , without or with minimal grinding). Grinding costs scale exponentially in the number of grinding bits. Because of this, we cannot resort to grinding for very large witness sizes ℓ . There are at least two ways to achieve 128 bits of security (or more), in this case.

Method 1: Use a larger binary field $\mathbb{F}_{2^\nu}$, i.e. simply take $\mathbb{F}_{2^\nu}$ with $|\mathbb{F}_{2^\nu}|$ comfortably larger, rather than $|\mathbb{F}_{2^\nu}| = 2^{128}$.

Method 2: We generalize an idea from [Bin26]. In this setting we make the assumption that the entries in γ (cf. Phase 3 of Construction 4.4) are smaller than $|\mathbb{F}_{2^\nu}| - 1$. This means we require $\max \mathcal{P} < |\mathbb{F}_{2^\nu}| - 1$ instead of $\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ (and similarly for the conditions that would allow to skip Round 1 in Construction 4.4). Hence now the bound on the primes in $\mathcal{P}$ does not depend on the witness size.

Note that when we attempt to prove the integer equalities $\langle \gamma, \mathbf{f}_{*j} \rangle = \mu_j$ ($j \in [\ell_2]$) “in the exponent” during Phase 3 of Construction 4.4, what we are really doing is proving the equalities modulo $|\mathbb{F}_{2^\nu}| - 1$. So far, we have used $\mathbb{F}_{2^\nu}$ large enough that this distinction does not matter, in that the total possible magnitude of $\langle \gamma, \mathbf{f}_{*j} \rangle$ does not exceed $|\mathbb{F}_{2^\nu}| - 2$.

If that is not the case, one can, in addition to proving $\langle \gamma, \mathbf{f}_{*j} \rangle = \mu_j$ modulo $|\mathbb{F}_{2^\nu}| - 1$, prove the same equality modulo ℓ_1 . This works because, since ℓ_1 and $|\mathbb{F}_{2^\nu}| - 1$ are coprime (because ℓ_1 is a power of two), by the CRT, if both equalities are true, then $\langle \gamma, \mathbf{f}_{*j} \rangle - \mu_j$ is an integer multiple of $\ell_1 \cdot (|\mathbb{F}_{2^\nu}| - 1)$. However, because the entries of $\mathbf{f}$ are bits and the entries of γ are at most $|\mathbb{F}_{2^\nu}| - 2$, this means that $\langle \gamma, \mathbf{f}_{*j} \rangle = \mu_j$ as integers.

To prove that $\langle \gamma, \mathbf{f}_{*j} \rangle = \mu_j$ modulo ℓ_1 , one can prove the integer equality $\langle \gamma', \mathbf{f}_{*j} \rangle = \mu'_j + a_j \cdot \ell_1$ for some $a_j \in \mathbb{Z}$ (provided by the prover), where γ' contains the entries of γ reduced modulo ℓ_1 , and μ'_j is μ_j reduced modulo ℓ_1 . The verifier checks that a_j is not too large, namely it checks that $|a_j| \cdot \ell_1 \leq (\ell_1 - 1) \cdot \ell_1$, which is the maximum value $|a_j| \cdot \ell_1$ can achieve when $\mathbf{f}$ is filled with bits

(which is guaranteed by our commitment via WHIR). Hence the value $|\langle \gamma', \mathbf{f}_{*j} \rangle - \mu'_j - a_j \cdot \ell_1|$ can be at most $2 \cdot (\ell_1 + 1) \cdot \ell_1$. As long as $2 \cdot (\ell_1 + 1) \cdot \ell_1 < |\mathbb{F}_{2^\nu}| - 1$, this equality can be proved “in the exponent” without causing overflow.

Note that if γ admits a tensor decomposition $\gamma = \gamma^{(1)} \otimes \gamma^{(2)}$, then we can take, instead of γ' , the tensor-product $\gamma^{(1)'} \otimes \gamma^{(2)'}$, where $\gamma^{(i)'}$ denotes $\gamma^{(i)}$ reduced modulo ℓ_1 . This allows to use all of our “low-entropy” grand product optimizations from [Section 6](#), though in this case the required bound becomes $2 \cdot (\ell_1^2 + 1) \cdot \ell_1 < |\mathbb{F}_{2^\nu}| - 1$.

Overall, this method allows to use $\max \mathcal{P} < |\mathbb{F}_{2^\nu}| - 1$, meaning we can choose primes that are $\log(\ell_1)$ bits larger than we otherwise would. This makes soundness errors be smaller by this same amount of bits, which leads to small grinding requirements. The cost is an extra batch of low entropy grand products.

Remark 4.17 (Projecting polynomial constraints to a 112-113 bit prime field with 128 bits of security). The error term of the projection step (also called fingerprinting) in [\[AGK⁺26, GWV⁺25, CHA24\]](#) is (for integer constraints) bounded by (cf. [Lemma 4.8](#) with $d = 0$)

$$\varepsilon_{\text{PIOP-proj}} \leq \frac{\log(N)}{|\mathcal{P}'| \cdot \lfloor \log(q_{\min}) \rfloor}$$

where $\mathcal{P}'$ is a set of primes; $q_{\min}$ is the smallest prime in $\mathcal{P}'$; N is the largest absolute value that an unsatisfying constraint can attain. More precisely, assume integer witness entries $\mathbf{w}$ are guaranteed to have at most certain bit-size B' , and assume the constraints have the form $q_i(\mathbf{x}, \mathbf{w}) = 0$ for all i , for $\mathbf{x}$ a public input vector and q_i constraint polynomials. Then N is the maximum possible value of the form $|q_i(\mathbf{x}, \mathbf{w})|$ among all i , all possible inputs $\mathbf{x}$ and all $\mathbf{w}$ with entries having bit-size at most B' .

Using a similar analysis as in Strategy 2 above, where we let $\mathcal{P}'$ be the set of primes between the interval $[2^{112}, 2^{113}]$, we obtain that, as long as $\log(N) < 1.4 \cdot 2^6 \approx 90$ we have $\varepsilon_{\text{PIOP-proj}} < 2^{-106}$. In that case we could attain 128 bits of security during this projection step by performing 22 bits of grinding (12ms in a MacBook Air M5).

We discuss how to construct end-to-end SNARKs for constraints over different rings, using BitZ as the underlying PCS.

5 An end-to-end BitZ-based SNARK over any finitely generated ring

We present an example SNARK for constraints over finitely generated rings, using BitZ as PCS. In our exposition we make several simplifying assumptions, for the sake of clarity. We discuss the general case in [Remark 5.2](#).

We emphasize that the main novelty is the PCS (Step 4 below). The techniques surrounding the PCS, namely the PIOP, can be traced back to Zaratana and Zinc+ [\[AGK⁺26, CHA24\]](#), though we allow for arbitrary finitely generated rings S by modifying Zinc+’s PIOP slightly, which neither work does, see [Remark 5.2](#). Because of this, we adopt an explanatory and intuitive tone in our presentation, rather than delving into full formality.

Let S be a finitely generated ring, for example S may be the integers $\mathbb{Z}$, a prime field, a lattice ring, a finite field extension, etc. For simplicity, for now let S be either $\mathbb{Z}$ or a finite field $\mathbb{F}$ (possibly

an extension field). We discuss the general case in [Remark 5.2](#). If $S = \mathbb{Z}$, then we assume a public parameter B specifying the maximum bit-size of a witness element. We let $S_{\leq B}$ denote either the finite field $\mathbb{F}$ or the integer range $[0..2^B - 1]$.

In a proof system over a ring S , given a public vector $\mathbf{x} = \mathbf{x} \in S^k$, the prover wants to prove it knows $\mathbf{w} = \mathbf{f} \in S_{\leq B}^{\ell_0}$ such that

$$\mathbf{p}_i(\mathbf{x}, \mathbf{f}) = 0 \quad \text{in } S \quad \text{for all } i \in [m], \quad (50)$$

where $\mathbf{p} = (\mathbf{p}_i)_{i \in [m]}$ is a public collection of polynomials with coefficients in S . For example, in an *R1CS constraint*,

$$\mathbf{p}_i(\mathbf{x}, \mathbf{w}) = (A \cdot (\mathbf{x}, \mathbf{f}))_i \cdot (B \cdot (\mathbf{x}, \mathbf{f}))_i - (C \cdot (\mathbf{x}, \mathbf{f}))_i$$

where A, B, C are $m \times (k + \ell_0)$ matrices over S .

Step 1: Commitment. The prover commits to $\mathbf{f}$ with BitZ , i.e. it sends a commitment to $\text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f}))$ over $\mathbb{F}_{2^\nu}$, as in Step 1 of [Section 2.2](#). This must happen before Step 2, since the soundness of fingerprinting relies on the witness being fixed before the verifier samples the random projection (e.g. reduction modulo a random prime).

Step 2: Projection to a finite field $\mathbb{F}$. If $S = \mathbb{Z}$, we perform the fingerprinting step of [\[CHA24\]](#). Concretely, the verifier samples a random prime q of $\approx \lambda$ bit size. Then the claim (50) is replaced by

$$\pi(\mathbf{p}_i)(\pi(\mathbf{x}), \pi(\mathbf{f})) = 0 \quad \text{in } \mathbb{F} \quad \text{for all } i, \quad (51)$$

where $\pi : \mathbb{Z} \rightarrow \mathbb{F}$ denotes reduction modulo q , and $\pi(\mathbf{p}_i)$ is $\mathbf{p}_i$ with coefficients reduced modulo q .

If $S = \mathbb{F}$ is a finite field, this step is skipped (unless we want to use the approach from [Remark 5.1](#)).

Going forward, we abuse the notation and use the same symbol $\mathbb{F}$ to denote either the field $\mathbb{F}$ above or S if S is a field. In this latter case we also let $\pi = \text{id} : \mathbb{F} \rightarrow \mathbb{F}$ denote the identity homomorphism.

Remark 5.1 (Reducing the size of S , if necessary). If S is a prime field of very large size, one may prefer to lift the initial polynomial constraints to $\mathbb{Z}$ first (between Step 1 and Step 2), and then project onto a random prime field $\mathbb{F}$ of bit-size $\approx \lambda$ during the present Step 2. This can compress the bit-size of the witness in the next steps by a very large factor.

To *lift the constraints to $\mathbb{Z}$* , the prover publishes the integer values $a_i := \text{lift}(\mathbf{p}_i)(\text{lift}(\mathbf{x}), \text{lift}(\mathbf{f}))$ for all i , where lift is the natural lift from a prime field to integers. Then the initial constraints over S are replaced by $\text{lift}(\mathbf{p}_i)(\text{lift}(\mathbf{x}), \text{lift}(\mathbf{f})) - a_i = 0$ in $\mathbb{Z}$.

If S is a proper finite field extension (or any finitely generated ring), one can also lift constraints. This time, one follows an approach similar to Step 3 of [Section 2.2](#).

A scenario where this can be desirable is when proving lattice-type computations, where rings are sometimes very large [\[CKKS17\]](#).

Step 3: PIOP over $\mathbb{F}$. As in [\[CHA24, AGK⁺26, CXNB26\]](#), a PIOP is run over $\mathbb{F}$, reducing (51) to a linear claim over $\mathbb{F}$ or, if $\mathbb{F}$ is too small, to a field extension $\mathbb{F}_{\text{ext}(e)}$ of $\mathbb{F}$. The claim is of the form

$$\langle \mathbf{v}, \pi(\mathbf{f}) \rangle = \mu \quad \text{in } \mathbb{F}_{\text{ext}(e)}, \quad (52)$$

for some $\mu \in \mathbb{F}_{\text{ext}(e)}$, $\mathbf{v} \in (\mathbb{F}_{\text{ext}(e)})^{\ell_0}$, where it is understood that $\mathbb{F}_{\text{ext}(e)} = \mathbb{F}$ if the PIOP did not need to use a field extension.

In general, if $\mathbb{F}$ is a prime field of $\approx \lambda$ bits (e.g. if we started with $R = \mathbb{Z}$), then $\mathbb{F}_{\text{ext}} = \mathbb{F}$. The field $\mathbb{F}_{\text{ext}}$ may be a proper extension if, for example, we started with S being a small finite field, and the PIOP had to use challenges in an extension of $\mathbb{F}$ to guarantee soundness.

We assume $\mathbf{v}$ admits a tensor decomposition $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} \in \mathbb{F}^{n_1 \cdot n_2}$. This is typically the case since $\mathbf{v}$ is usually an “eq” vector, i.e. $\mathbf{v} = (\text{eq}(\mathbf{b}, \mathbf{r}))_{\mathbf{b} \in \{0,1\}^{\log \ell_0}}$ for some $\mathbf{r} \in \mathbb{F}_{\text{ext}}^{\log \ell_0}$. As explained throughout the paper, this is leveraged crucially by BitZ PCS.

Step 4: Opening with the BitZ PCS. The claim (52), together with the commitment to $\text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f}))$, is an instance of the relation $\mathcal{R}_{\text{LinBitsRings}}(S, \mathbb{F}, \pi)$ from Section 2.2 (or of $\mathcal{R}_{\text{LinBitsRings}}(\mathbb{F}_q)$ from Section 2.1 if $S = \mathbb{F} = \mathbb{F}_q$), with the commitment of Step 1 as oracle. Accordingly, prover and verifier prove it with the BitZ PCS.

Remark 5.2 (Handling of arbitrary finitely generated rings). We briefly outline how to handle the case when S is an arbitrary finitely generated ring. The idea is the same as used in BitZ (Step 3 of Section 2.2, or Round 1 of Section 4.3). In these steps, we lift a linear constraint over an arbitrary finitely generated ring, to the free ring $F = \mathbb{Z}[X_1, \dots, X_k]$, and then sample a random morphism from F to a finite field, projecting the lifted constraints onto $\mathbb{F}$. The same method is valid if instead of starting with linear constraints, one starts with arbitrary polynomial constraints. One can use Lemma 4.8 to derive the soundness of this step.

6 A GKR protocol for low entropy batched grand products

We now describe our instantiation of the GKR protocol for proving batched grand products, and our prover optimizations when these have “low entropy”.

The bulk of the work in BitZ (Constructions 4.4 and 4.13) is to reduce the claim over the binary field $\mathbb{F}_{2^\nu}$

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2] \quad (53)$$

to an inner product claim of the form $\langle \mathbf{v}, \pi_2(\mathbf{f}) \rangle = \mu$ over $\mathbb{F}_{2^\nu}$, for some $\mathbf{v} \in \mathbb{F}_{2^\nu}^\ell$, where here $\mathbf{f} \in \{0,1\}^\ell \subseteq \mathbb{Z}^\ell$. Recall that

$$\ell = \ell_1 \cdot \ell_2.$$

To fix intuition, in our instantiation (cf. Section 4.7) we set $\ell_1 \approx \ell^{0.6}, \ell_2 = \ell^{0.4}$.

Cryptographically speaking, our protocol is simply a standard batched version of the GKR protocol applied to grand products from [Tha13]. However, our implementation uses lookup tables (i.e. preprocessed data) to optimize the prover time of such protocol. In particular, our techniques leverage the fact that the factors in (53) have *low entropy*, in the sense that for every $i \in [\ell_1]$ and $j \in [\ell_2]$, the i -th factor in the j -th grand product of (53) can take only two values, depending on whether $\mathbf{f}_{ij} = 1$ or $\mathbf{f}_{ij} = 0$.

The optimizations we introduce apply to any (batched) grand product with low entropy factors, and not only to the grand product in (53). They also work on any field. Because of this, we

describe a general protocol IOR that, given a finite field $\mathbb{K}$; a bivariate polynomial $p \in \mathbb{K}[Y, X]$; a small subset $S \subseteq \mathbb{K}$; and dimensions $\ell = \ell_1 \cdot \ell_2$, reduces triples from

$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{y}, \boldsymbol{\mu}) \in \mathbb{K}^{\ell_1} \times \mathbb{K}^{\ell_2}, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2} \end{array} \right) \left| \prod_{i \in [\ell_1]} p(\mathbf{y}_i, \mathbf{f}_{ij}) = \boldsymbol{\mu}_j \text{ for all } j \in [\ell_2] \right. \right\},$$

to triples in

$$\mathcal{R}_{\text{Lin}}(\mathbb{K}) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{u}, \mu) \in \mathbb{K}^\ell \times \mathbb{K}, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2} \end{array} \right) \left| \langle \mathbf{u}, \mathbf{f} \rangle = \mu \right. \right\}.$$

One can additionally add an encoding $f = \text{Enc}_C(\mathbf{f})$ as an oracle in the above relations (matching the context in which we use the IOR), but our IOR does not make any use of it whatsoever, hence we omit it.

In our instantiation, $\mathbf{y}_i = g^{\gamma_i}$ and $\boldsymbol{\mu}_j = g^{\mu_j}$, where γ_i and μ_j are the integers in (53).

We describe Π_{red} now. We will describe our implementation-level optimizations afterwards. As we mentioned, cryptographically/protocol-wise, our protocol is a standard batched GKR for grand products [Tha22]. We assume ℓ_1, ℓ_2 are powers of two:

$$\ell_1 = 2^{r_1}, \quad \ell_2 = 2^{r_2}.$$

We index $[\ell_1], [\ell_2]$ via elements of the hypercubes $\{0, 1\}^{r_1}, \{0, 1\}^{r_2}$, respectively.

Construction 6.1 (An IOR Π_{red} from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K})$). **Input.** The prover and verifier receive $(\mathbf{y}, \boldsymbol{\mu}) \in \mathbb{K}^{\ell_1} \times \mathbb{K}^{\ell_2}$. The prover receives also $\mathbf{f} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2}$.

Let L_{r_1} be a *multilinear* polynomial over $\mathbb{K}$ on $r_1 + r_2$ variables such that

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = p(\mathbf{y}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2}.$$

(note that p may not be multilinear). For $i = 0, \dots, r_1 - 1$, let L_i be a multilinear polynomial on $r_2 + i$ variables such that

$$L_i(\mathbf{b}, \mathbf{c}) = L_{i+1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{i+1}(\mathbf{b}, 1, \mathbf{c}) \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^i \times \{0, 1\}^{r_2}.$$

1. The verifier sends $\boldsymbol{\alpha}^{(0)} \in \mathbb{K}^{r_2}$. Both prover and verifier compute $C^{(0)} := L_0(\boldsymbol{\alpha}^{(0)})$ (note that L_0 is the multilinear extension of $\boldsymbol{\mu}$ over $\mathbb{K}$).
2. For $i = 0, \dots, r_1 - 1$:

(a) ***i*-th sumcheck** The prover and verifier run the sumcheck protocol for the following claim:

$$C^{(i)} = \sum_{(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^i \times \{0, 1\}^{r_2}} L_{i+1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{i+1}(\mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)}).$$

At the end of the sumcheck protocol, the prover and verifier are left with a claim of the form

$$L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, 0, \mathbf{r}_{\mathbf{c}}^{(i)}) = a_0^{(i)}, \quad L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, 1, \mathbf{r}_{\mathbf{c}}^{(i)}) = a_1^{(i)} \tag{54}$$

for some $\mathbf{r}_{\mathbf{b}}^{(i)} \in \mathbb{K}^i$, $\mathbf{r}_{\mathbf{c}}^{(i)} \in \mathbb{K}^{r_2}$ and $a_0^{(i)}, a_1^{(i)} \in \mathbb{K}$.

(b) **i -th unification step** The prover and verifier unify the two claims from (54) into a single one via the following standard procedure:

- i. The verifier sends a random element $\beta^{(i)} \in \mathbb{K}$.
- ii. The claim (54) is replaced by

$$L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, \beta^{(i)}, \mathbf{r}_{\mathbf{c}}^{(i)}) = C^{(i+1)} := a_0^{(i)} \cdot (1 - \beta^{(i)}) + a_1^{(i)} \cdot \beta^{(i)}.$$

Let

$$\boldsymbol{\alpha}^{(i+1)} := (\mathbf{r}_{\mathbf{b}}^{(i)}, \beta^{(i)}, \mathbf{r}_{\mathbf{c}}^{(i)}).$$

3. At this stage, the prover and verifier are left with the claim

$$L_{r_1}(\boldsymbol{\alpha}^{(r_1)}) = C^{(r_1)}.$$

This claim is equivalent to

$$\sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1} \times \{0,1\}^{r_2}} p(\mathbf{y}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) \cdot \text{eq}((\mathbf{b}, \mathbf{c}), \boldsymbol{\alpha}^{(r_1)}) = C^{(r_1)}, \quad (55)$$

If $p(Y, X)$ is linear on X , then (55) is a linear claim on $\mathbf{f}$. Namely, it is equivalent to

$$\langle \mathbf{u}, \mathbf{f} \rangle = C^{(r_1)} + c$$

for some $\mathbf{u} \in \mathbb{K}^\ell, c \in \mathbb{K}$. In this case, prover and verifier output

$$(\mathbf{x}' = (\mathbf{u}, C^{(r_1)} + c), \perp, \mathbf{w}' = \mathbf{f}) \in \mathcal{R}_{\text{Lin}}(\mathbb{K}).$$

Otherwise, i.e. if p is not linear in X , prover and verifier run a sumcheck for the claim (55), which results in MLE claims for $\mathbf{y}$ and $\mathbf{f}$. The verifier checks the claim for $\mathbf{y}$ itself, and prover and verifier output the MLE claim for $\mathbf{f}$.

Below we discuss what p , $\mathbf{u}$ and $C^{(r_1)} + c$ are specifically in our usage of Π_{red} as a subprotocol of [Constructions 4.4](#) and [4.13](#), and how to keep the verifier succinct.

Using Π_{red} in BitZ and succinctness of the verifier. If $p(\mathbf{y}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) = (\mathbf{y}_{\mathbf{b}} - 1) \cdot \mathbf{f}_{\mathbf{b}, \mathbf{c}} + 1$ as in [Constructions 4.4](#) and [4.13](#), then

$$\mathbf{u} = ((\mathbf{y}_{\mathbf{b}} - 1) \cdot \text{eq}(\mathbf{b}; \boldsymbol{\alpha}^{(r_1, 1)}))_{\mathbf{b} \in \{0,1\}^{r_1}} \otimes (\text{eq}(\mathbf{c}; \boldsymbol{\alpha}^{(r_1, 2)}))_{\mathbf{c} \in \{0,1\}^{r_2}}, \quad C^{(r_1)} + c = C^{(r_1)} - 1,$$

where $\boldsymbol{\alpha}^{(r_1)} = (\boldsymbol{\alpha}^{(r_1, 1)}, \boldsymbol{\alpha}^{(r_1, 2)})$. In this case, to avoid having the verifier pay a linear cost, the verifier constructs $\mathbf{u}^{(1)} := ((\mathbf{y}_{\mathbf{b}} - 1) \cdot \text{eq}(\mathbf{b}; \boldsymbol{\alpha}^{(r_1, 1)}))_{\mathbf{b} \in \{0,1\}^{r_1}}$ and $\mathbf{u}^{(2)} := (\text{eq}(\mathbf{c}; \boldsymbol{\alpha}^{(r_1, 2)}))_{\mathbf{c} \in \{0,1\}^{r_2}}$ separately, and outputs these two vectors instead of $\mathbf{u}$.

When used as a subprotocol of [Constructions 4.4](#) and [4.13](#) (concretely, in Phase 3 of [Construction 4.4](#)), after executing the GKR protocol above ([Construction 6.1](#)), we use an IOP Π_{iopp} to prove the linear claim $\langle \mathbf{u}, \pi_2(\mathbf{f}) \rangle = C^{(r_1)} - 1$ (if using [Construction 4.13](#), $\mathbf{f}$ is replaced with $\text{bits}(\mathbf{f})$), with the verifier having oracle access to $\text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\mathbf{f})$. As we mentioned ([Section 4.7](#)), Π_{iopp} first runs a sumcheck, reducing the claim $\langle \mathbf{u}, \pi_2(\mathbf{f}) \rangle = C^{(r_1)} - 1$ to an MLE evaluation claim of the

vector $\pi_2(\mathbf{f})$ and of $\mathbf{u}$. For the latter, we have, letting $\boldsymbol{\theta} = (\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}) \in \mathbb{K}^{r_1+r_2}$ be the evaluation point,

$$\begin{aligned}\widetilde{\mathbf{u}}(\boldsymbol{\theta}) &= \widetilde{\mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}}(\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}) = \widetilde{\mathbf{u}^{(1)}}(\boldsymbol{\theta}^{(1)}) \cdot \widetilde{\mathbf{u}^{(2)}}(\boldsymbol{\theta}^{(2)}), \\ \widetilde{\mathbf{u}^{(1)}}(\boldsymbol{\theta}^{(1)}) &= \sum_{\mathbf{b} \in \{0,1\}^{r_1}} (\mathbf{y}_{\mathbf{b}} - 1) \cdot \text{eq}(\mathbf{b}; \boldsymbol{\alpha}^{(r_1,1)}) \cdot \text{eq}(\mathbf{b}; \boldsymbol{\theta}^{(1)}), \\ \widetilde{\mathbf{u}^{(2)}}(\boldsymbol{\theta}^{(2)}) &= \text{eq}(\boldsymbol{\theta}^{(2)}; \boldsymbol{\alpha}^{(r_1,2)}),\end{aligned}$$

and so the verifier can compute $\widetilde{\mathbf{u}}(\boldsymbol{\theta})$ by itself in time $O(\ell_1 + \ell_2)$. For the MLE claim on $\pi_2(\mathbf{f})$, we first use ring-switching [DP24] to reduce the claim to a batch of linear claims for $\text{Pack}_\nu \circ \pi_2(\mathbf{f})$, and then prove these with, e.g. WHIR or Ligerito.

Costs. In Table 9 we summarize the number of rounds, the proof size and the verifier work of Construction 6.1 when used within BitZ as described above. These are standard costs corresponding to a standard GKR construction. We discuss an optimized prover, and its costs, in the next subsection. Recall that in our instantiations $\ell_1 \approx \sqrt{\ell}$ and $\ell_2 \approx \sqrt{\ell}$, hence the verifier is succinct.

	Rounds	Proof (elements of $\mathbb{K}$)	Verifier
Step 1 (batching)	1	–	$O(\ell_2) \cdot \mathbb{K}$
Step 2, i -th sumcheck	$i + r_2$	$3 \cdot (i + r_2)$	$O(i + r_2) \cdot \mathbb{K}$
Step 2, i -th unification	1	2	$O(1) \cdot \mathbb{K}$
Computing $\mathbf{u}$	–	–	$O(\ell_1 + \ell_2) \cdot \mathbb{K}$ (only $\mathbf{u}^{(1)}, \mathbf{u}^{(2)}$)
Total	$t_{\text{red}} = 1 + r_1 \cdot r_2 + \binom{r_1+1}{2}$	$3 \cdot t_{\text{red}} - r_1 - 3$	$O(\ell_1 + \ell_2 + \log^2 \ell) \cdot \mathbb{K}$

Table 9: Rounds, proof size and verifier work of Construction 6.1 for the polynomial $p(Y, X) = (Y - 1) \cdot X + 1$ in our deployment of BitZ. By $N \cdot \mathbb{K}$ we indicate N operations in $\mathbb{K}$. The proof size counts 3 elements per sumcheck round. Using [Gru24] we send two 2 elements per sumcheck round in our implementation.

The next lemma records the soundness errors of Construction 6.1.

Lemma 6.2 (Soundness and completeness of Construction 6.1). *Assume $p(Y, X)$ is linear in X ¹⁵. Then Construction 6.1 is a perfectly complete $(1 + r_1 r_2 + r_1(r_1 + 1)/2)$ -round IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K})$, whose verifier makes no oracle queries and whose prover outputs the witness $\mathbf{f}$ it received.*

Construction 6.1 also extends to an IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p, \mathcal{C}, \text{Pack}_\nu)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}_\nu)$ in a natural way (see Section 3.2 for a definition of these relations). For every δ within the unique decoding radius of $\mathcal{C}$, this latter IOR has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p, \mathcal{C}, \text{Pack}_\nu)^\delta, \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}_\nu)^\delta)$ with errors $r_2/|\mathbb{K}|$ in the first round, $3/|\mathbb{K}|$ in every sumcheck round and $1/|\mathbb{K}|$ in every unification step. If δ is beyond the unique decoding radius, the errors must be multiplied by L_δ , where $\mathcal{C}$ is (δ, L_δ) -list-decodable. The same holds for the relations $\mathcal{R}_{\text{PESAT}}^{\delta, \zeta, y}$ and $\mathcal{R}_{\text{Lin}}^{\delta, \zeta, y}$ of Section A.2.

We omit the proof of Lemma 6.2 since Construction 6.1 is a well-known variation of the GKR protocol, see [Tha22]. Round-by-round soundness of GKR is shown in [CCH⁺19]. The knowledge version with relaxation follows as in [BCFW25, proof of Lemma 6.6].

¹⁵We focus on this case since it is the one in our instantiation. A generalization to arbitrary polynomials $p(Y, X)$ would add rounds and error terms corresponding to the sumcheck protocol mentioned at the end of Construction 6.1.

6.1 Prover optimization via lookup tables

We optimize [Construction 6.1](#) as follows. For simplicity we use the subset

$$S = \{0, 1\} \subseteq \mathbb{K},$$

but any small subset of $\mathbb{K}$ is amenable to the techniques discussed here.

Setup We start with the following observation: for every $(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2}$, we have $\mathbf{f}_{\mathbf{b}, \mathbf{c}} \in S = \{0, 1\}$, and so

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = p(\mathbf{y}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) = \begin{cases} \mathbf{y}_{\mathbf{b}}^{(1)} := p(\mathbf{y}_{\mathbf{b}}, 1) & \text{if } \mathbf{f}_{\mathbf{b}, \mathbf{c}} = 1 \\ \mathbf{y}_{\mathbf{b}}^{(0)} := p(\mathbf{y}_{\mathbf{b}}, 0) & \text{if } \mathbf{f}_{\mathbf{b}, \mathbf{c}} = 0. \end{cases}$$

The values $\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)}$ do not depend on $\mathbf{c}$. Hence the prover can set up a lookup table $T_{r_1} \in (\mathbb{K} \times \mathbb{K})^{\{0, 1\}^{r_1}}$ defined as

$$T_{r_1}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)}) \quad \text{for all } \mathbf{b} \in \{0, 1\}^{r_1}$$

at the beginning of the protocol, and, whenever required, use it to obtain the values $L_{r_1}(\mathbf{b}, \mathbf{c})$. Concretely

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = T_{r_1}[\mathbf{b}][\mathbf{f}_{\mathbf{b}, \mathbf{c}}] \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2}$$

In our instantiation, $\mathbf{y}_{\mathbf{b}}^{(1)} = g^{\gamma_{\mathbf{b}}}$ and $\mathbf{y}_{\mathbf{b}}^{(0)} = 1$ (cf. [\(53\)](#)). Hence additionally the prover precomputes the elements $g^{\gamma_{\mathbf{b}}}$ at the beginning of the protocol, before computing T_{r_1} .

As we will explain later, the prover will also precompute similar tables for the values $L_{r_1-j}(\mathbf{b}, \mathbf{c})$ of the levels close to the leaves (i.e. for small j),

$$T_{r_1-j} \in \left(\mathbb{K}^{2^{2^j}} \right)^{\{0, 1\}^{r_1-j}}, \quad (56)$$

as well as other tables relevant for the different sumcheck rounds of [Construction 6.1](#), constructed from the tables T_{r_1-j} themselves. The superindex $\{0, 1\}^{r_1-j}$ in [\(56\)](#) indicates that the table T_{r_1-j} has one entry per element in the hypercube $\{0, 1\}^{r_1-j}$; and this entry is a vector of 2^{2^j} field elements. Concretely, for $\mathbf{b} \in \{0, 1\}^{r_1-j}$, $T_{r_1-j}[\mathbf{b}]$ consists of all possible products of 2^j consecutive factors of the grand product, namely the factors indexed by $(\mathbf{b}, \mathbf{b}')$ with $\mathbf{b}$ fixed and $\mathbf{b}'$ ranging over $\{0, 1\}^j$. Each factor can take two values, depending on whether $\mathbf{f}_{\mathbf{b}, \mathbf{b}', \mathbf{c}}$ is 1 or 0. Hence $T_{r_1-j}[\mathbf{b}]$ consists of 2^{2^j} elements. Hence the size of T_{r_1-j} in [\(56\)](#).

Because $|T_{r_1-j}|$ grows doubly-exponentially in j , these tables are viable only for the bottom few levels of the tree ($j = 0, 1, 2$ in our implementation). In higher levels we use standard sumchecks.

Importantly, all these tables have size independent of ℓ_2 . This (among others, cf. [Section 2.1.1](#)), is one of the reasons why we use the tensor decomposition structure, i.e. the split given by $\ell = \ell_1 \cdot \ell_2$, in [Construction 4.4](#).

Remark 6.3. All precomputed data in this section depends on the input vector $\mathbf{y}$ of $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$, and possibly on further verifier challenges. Since, in a wider context where [Construction 6.1](#) is a sub-process of a larger proof system, $\mathbf{y}$ typically depends on verifier's challenges, these precomputations must be done at *runtime*.

First prover message in the last sumcheck in Step 2 of Construction 6.1 We start by analyzing Step 2 with $i = r_1 - 1$. In this step prover and verifier run a sumcheck for the claim

$$C^{(r_1-1)} = \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-1} \times \{0,1\}^{r_2}} L_{r_1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)})$$

This is the most expensive sumcheck of Construction 6.1 for the prover (if done naively), since it is the sumcheck over the largest number of variables throughout Construction 6.1.

For $1 \leq j \leq r_1 - 1$, the j -th prover message in this sumcheck is, if $\boldsymbol{\rho}^{(j)} = (\rho_1, \dots, \rho_{j-1})$ is the vector of verifier challenges so far in this sumcheck,

$$\begin{aligned} h_j(X) &:= \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-1-j} \times \{0,1\}^{r_2}} L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)}) \\ &= \sum_{\mathbf{c} \in \{0,1\}^{r_2}} \left(\underbrace{\sum_{\mathbf{b} \in \{0,1\}^{r_1-1-j}} L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\boldsymbol{\rho}^{(j)}, \mathbf{b}; \boldsymbol{\alpha}^{(i,1)})}_{:= h_{j,\mathbf{c}}(X)} \right) \cdot \text{eq}(X, \mathbf{c}; \boldsymbol{\alpha}^{(i,2)}) \\ &= \sum_{\mathbf{c} \in \{0,1\}^{r_2}} h_{j,\mathbf{c}}(X) \cdot \text{eq}(X, \mathbf{c}; \boldsymbol{\alpha}^{(i,2)}), \end{aligned}$$

where $\boldsymbol{\alpha}^{(i)} = (\boldsymbol{\alpha}^{(i,1)}, \boldsymbol{\alpha}^{(i,2)}) \in \mathbb{K}^{r_1-2} \times \mathbb{K}^{r_2+1}$. We claim that

$$L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) = \left(L_{r_1}(\boldsymbol{\rho}^{(j)}, 1, \mathbf{b}, 0, \mathbf{c}) - L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 0, \mathbf{c}) \right) \cdot X + L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 0, \mathbf{c}) \quad (57)$$

Indeed, $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c})$ is a linear polynomial, and the right-hand side in the above expression is the line passing through the two evaluations of $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c})$ at $X = 0$ and $X = 1$. An analogous expression holds for $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c})$:

$$L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) = \left(L_{r_1}(\boldsymbol{\rho}^{(j)}, 1, \mathbf{b}, 1, \mathbf{c}) - L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 1, \mathbf{c}) \right) \cdot X + L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 1, \mathbf{c}). \quad (58)$$

Renaming

$$E_{\varepsilon_1 \varepsilon_2}^{(j)}[\mathbf{b}, \mathbf{c}] := L_{r_1}(\boldsymbol{\rho}^{(j)}, \varepsilon_1, \mathbf{b}, \varepsilon_2, \mathbf{c}) \quad \varepsilon_1, \varepsilon_2 \in \{0, 1\}, \quad \omega[\mathbf{b}] := \text{eq}(\boldsymbol{\rho}^{(j)}, \mathbf{b}; \boldsymbol{\alpha}^{(i,1)}),$$

we obtain

$$h_{j,\mathbf{c}}(X) = \sum_{\mathbf{b} \in \{0,1\}^{r_1-1-j}} X^2 \cdot A_2^{(j)}[\mathbf{b}, \mathbf{c}] + X \cdot A_1^{(j)}[\mathbf{b}, \mathbf{c}] + A_0^{(j)}[\mathbf{b}, \mathbf{c}]$$

where,

$$\begin{aligned} A_0^{(j)}[\mathbf{b}, \mathbf{c}] &:= E_{00}^{(j)}[\mathbf{b}, \mathbf{c}] \cdot E_{01}^{(j)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}], \\ A_1^{(j)}[\mathbf{b}, \mathbf{c}] &:= \left((E_{10}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{00}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot E_{01}^{(j)}[\mathbf{b}, \mathbf{c}] + (E_{11}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{01}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot E_{00}^{(j)}[\mathbf{b}, \mathbf{c}] \right) \cdot \omega[\mathbf{b}] \\ A_2^{(j)}[\mathbf{b}, \mathbf{c}] &:= (E_{10}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{00}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot (E_{11}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{01}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot \omega[\mathbf{b}]. \end{aligned}$$

Now, for $j = 1$, we have $\rho^{(j)} = \perp$, and hence each value $E_{\varepsilon_1 \varepsilon_2}^{(1)}[\mathbf{b}, \mathbf{c}]$ can be obtained, whenever needed, by querying $T_{r_1}[\mathbf{b}]$ and selecting $T_{r_1}[\mathbf{b}][1]$ or $T_{r_1}[\mathbf{b}][0]$ depending on whether $\mathbf{f}_{\mathbf{b}, \mathbf{c}} = 1$ or 0. A consequence is that the coefficients themselves $A_k^{(1)}[\mathbf{b}, \mathbf{c}]$ ($k \in [3]$) can be obtained through lookup tables, as follows: we have

$$A_0^{(1)}[\mathbf{b}, \mathbf{c}] = T_{r_1}[0, \mathbf{b}, 0][\mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}}] \cdot T_{r_1}[0, \mathbf{b}, 1][\mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}}] \cdot \omega[\mathbf{b}]$$

Hence

$$A_0^{(1)}[\mathbf{b}, \mathbf{c}] = \begin{cases} \mathbf{y}_{0, \mathbf{b}, 0}^{(1)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(1)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}} = 1, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}} = 1 \\ \mathbf{y}_{0, \mathbf{b}, 0}^{(1)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(0)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}} = 1, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}} = 0 \\ \mathbf{y}_{0, \mathbf{b}, 0}^{(0)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(1)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}} = 0, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}} = 1 \\ \mathbf{y}_{0, \mathbf{b}, 0}^{(0)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(0)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}} = 0, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}} = 0 \end{cases}$$

Thus the prover can precompute the table

$$T_{A_0} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-2}} = \mathbb{K}^{\ell_1}$$

given by

$$T_{A_0}[\mathbf{b}] = (\mathbf{y}_{0, \mathbf{b}, 0}^{(1)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(1)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0, \mathbf{b}, 0}^{(1)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(0)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0, \mathbf{b}, 0}^{(0)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(1)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0, \mathbf{b}, 0}^{(0)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(0)} \cdot \omega[\mathbf{b}])$$

and use it to obtain any value $A_0^{(1)}[\mathbf{b}, \mathbf{c}]$ for $(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-2} \times \{0, 1\}^{r_2}$ via a single read into T_{A_0} . The size of T_{A_0} , again, does not depend on $\ell_2 = 2^{r_2}$. Building T_{A_0} requires at most $8 \cdot 2^{r_1-2} = 2^{r_1+1}$ multiplications and no additions.

We now explain how to obtain the values $A_1^{(1)}[\mathbf{b}, \mathbf{c}]$ and $A_2^{(1)}[\mathbf{b}, \mathbf{c}]$, again via the construction of a lookup table. Note that these depend on the *four* bits $\mathbf{f}_{(\varepsilon_1, \mathbf{b}, \varepsilon_2), \mathbf{c}}$, $\varepsilon_1, \varepsilon_2 \in \{0, 1\}$, so our tables will need $2^4 = 16$ entries (as opposed to 4 as in T_{A_0}) per index $\mathbf{b}$. The prover proceeds as follows:

- It computes a table $T_{S_1} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-2}}$ analogous to T_{A_0} , which allows to obtain the values

$$S_1^{(1)}[\mathbf{b}, \mathbf{c}] := E_{10}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot E_{11}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}],$$

(observe that T_{A_0} allows to obtain the values $E_{00}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot E_{01}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}]$).

- It computes the table $T_{A_2} \in (\mathbb{K}^{16})^{\{0,1\}^{r_1-2}}$ given by

$$T_{A_2}[\mathbf{b}] = \left((\mathbf{y}_{1, \mathbf{b}, 0}^{(x_1)} - \mathbf{y}_{0, \mathbf{b}, 0}^{(x_0)}) \cdot (\mathbf{y}_{1, \mathbf{b}, 1}^{(x'_1)} - \mathbf{y}_{0, \mathbf{b}, 1}^{(x'_0)}) \cdot \omega[\mathbf{b}] \right)_{(x_0, x_1, x'_0, x'_1) \in \{0, 1\}^4}, \quad (59)$$

so that $A_2^{(1)}[\mathbf{b}, \mathbf{c}]$ is the element

$$A_2^{(1)}[\mathbf{b}, \mathbf{c}] = T_{A_2}[\mathbf{b}][\mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}}, \mathbf{f}_{(1, \mathbf{b}, 0), \mathbf{c}}, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}}, \mathbf{f}_{(1, \mathbf{b}, 1), \mathbf{c}}].$$

Building T_{A_2} takes at most 20 multiplications and 8 additions per index $\mathbf{b}$ ¹⁶. I.e. at most $5 \cdot 2^{r_1}$ multiplications and 2^{r_1+1} additions in total.

¹⁶For the multiplications, there are 4 multiplications to compute the 4 elements $t_{x'_0, x'_1, \mathbf{b}} := (\mathbf{y}_{1, \mathbf{b}, 1}^{(x'_1)} - \mathbf{y}_{0, \mathbf{b}, 1}^{(x'_0)}) \cdot \omega[\mathbf{b}]$, and then 16 more multiplications to compute $(\mathbf{y}_{1, \mathbf{b}, 0}^{(x_1)} - \mathbf{y}_{0, \mathbf{b}, 0}^{(x_0)}) \cdot t_{x'_0, x'_1, \mathbf{b}}$

- It obtains the values $A_1^{(1)}[\mathbf{b}, \mathbf{c}]$ via the identity

$$S_1^{(1)}[\mathbf{b}, \mathbf{c}] = A_0^{(1)}[\mathbf{b}, \mathbf{c}] + A_1^{(1)}[\mathbf{b}, \mathbf{c}] + A_2^{(1)}[\mathbf{b}, \mathbf{c}] \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-2} \times \{0, 1\}^{r_2}.$$

Once the tables have been computed, obtaining $h_{1,\mathbf{c}}(X)$ for one $\mathbf{c} \in \{0, 1\}^{r_2}$ costs $3 \cdot 2^{r_1-2}$ table reads (one read per table $T_{A_0}, T_{S_1}, T_{A_2}$ per index $\mathbf{b} \in \{0, 1\}^{r_1-2}$) and $3 \cdot 2^{r_1-2} + 2$ additions: this is the cost of adding together the values read at each table, providing $\sum_{\mathbf{b}} A_0[\mathbf{b}, \mathbf{c}]$, $\sum_{\mathbf{b}} S_1[\mathbf{b}, \mathbf{c}]$, $\sum_{\mathbf{b}} A_2[\mathbf{b}, \mathbf{c}]$, and then doing two additions to obtain $\sum_{\mathbf{b}} A_1[\mathbf{b}, \mathbf{c}] = \sum_{\mathbf{b}} S_1[\mathbf{b}, \mathbf{c}] - \sum_{\mathbf{b}} A_0[\mathbf{b}, \mathbf{c}] - \sum_{\mathbf{b}} A_2[\mathbf{b}, \mathbf{c}]$. In total, this amounts to at most $3\ell/4$ table reads and $\approx 3\ell/4 + 2 \cdot \ell_2$ additions to compute $h_{1,\mathbf{c}}$ for all $\mathbf{c}$, and no multiplications (in our instantiation, additions in $\mathbb{K}$ are 128-bit XORs).

Finally, the prover computes $h_1(X)$ from the polynomials $h_{1,\mathbf{c}}(X)$ with at most $5 \cdot 2^{r_2}$ additions and $3 \cdot 2^{r_2}$ multiplications (plus 2^{r_2} multiplications to precompute the values $\text{eq}(\mathbf{c}; \boldsymbol{\alpha}^{(i,2)})$, shared by all rounds of this sumcheck, and 2^{r_1-2} multiplications for the weights $\omega[\mathbf{b}]$).

In [Table 10](#) we record the total upper bound costs for computing $h_1(X)$. In our application we obtain lower numbers since $\mathbf{y}_{\mathbf{b}}^{(0)} = 1$ and we work in characteristic two, and so in [Eq. \(59\)](#) some elements collapse to 0.

On the other hand, a standard generic approach to compute $h_1(X)$ would require $O(\ell) = O(\ell_1 \cdot \ell_2)$ multiplications and additions, even if using the few sumcheck optimizations that apply in our settings (to our knowledge) [Albert: which ones, exactly?](#). Crucially, we note that our sumcheck features summands of full field bit size $\log |\mathbb{K}|$, which voids some known powerful optimizations when the summands are small, e.g. the univariate skip [\[Gru24\]](#) and some optimizations from [\[DT24, BDDT25\]](#). The prover message degree reduction through factoring the “eq factor” from [\[Gru24\]](#) does apply to our setting, and our implementation uses it: we send 2 elements instead of the 3 in each sumcheck round.

	Our approach	Standard/generic
Multiplications	$\leq 10 \cdot \ell_1 + 4 \cdot \ell_2$	$\Theta(\ell_1 \cdot \ell_2)$
Additions	$\leq 3 \cdot \ell_1 \cdot \ell_2 / 4 + 2 \cdot \ell_1 + 5 \cdot \ell_2$	$\Theta(\ell_1 \cdot \ell_2)$
Table reads	$3 \cdot \ell_1 \cdot \ell_2 / 4$, into tables of $6 \cdot \ell_1$ field elements	–

Table 10: The cost in field operations of the first sumcheck round of the last sumcheck in our GKR protocol for batched grand products, cf. [Construction 6.1](#), where $\ell = \ell_1 \cdot \ell_2 = 2^{r_1} \cdot 2^{r_2}$ is the witness size. In our instantiation $\ell_1 \approx \ell^{0.6} = 2^{r_1}$ and $\ell_2 \approx \ell^{0.4} = 2^{r_2}$ ([Section 4.7](#)).

Subsequent prover messages in the last sumcheck in Step 2 of [Construction 6.1](#) Next we focus on computing $h_2(X)$. Substituting X by the verifier’s first sumcheck challenge ρ_1 in [Eqs. \(57\)](#) and [\(58\)](#), we have, for every $\mathbf{b} \in \{0, 1\}^{r_1-1}$, $\mathbf{c} \in \{0, 1\}^{r_2}$,

$$\begin{aligned} L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c}) &= (L_{r_1}(1, \mathbf{b}, \mathbf{c}) - L_{r_1}(0, \mathbf{b}, \mathbf{c})) \cdot \rho_1 + L_{r_1}(0, \mathbf{b}, \mathbf{c}) \\ &= \begin{cases} \mathbf{y}_{\mathbf{b}}^{(11)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(1)} - \mathbf{y}_{0,\mathbf{b}}^{(1)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(1)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 1, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 1 \\ \mathbf{y}_{\mathbf{b}}^{(10)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(0)} - \mathbf{y}_{0,\mathbf{b}}^{(1)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(1)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 1, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 0 \\ \mathbf{y}_{\mathbf{b}}^{(01)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(1)} - \mathbf{y}_{0,\mathbf{b}}^{(0)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(0)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 0, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 1 \\ \mathbf{y}_{\mathbf{b}}^{(00)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(0)} - \mathbf{y}_{0,\mathbf{b}}^{(0)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(0)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 0, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 0 \end{cases} \end{aligned}$$

The idea now is that the prover can compute $h_2(X)$ in essentially the same way as $h_1(X)$, but this time, instead of starting with the table

$$T_{r_1}^{(1)} := T_{r_1} \in (\mathbb{K}^2)^{\{0,1\}^{r_1}} \quad \text{such that } T_{r_1}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)}) \text{ and } L_{r_1}(\mathbf{b}, \mathbf{c}) = T_{r_1}[\mathbf{b}][\mathbf{f}_{\mathbf{b},\mathbf{c}}], \quad (\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1} \times \{0,1\}^{r_2},$$

it starts with the table

$$T_{r_1}^{(2)} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-1}} \quad \text{such that } T_{r_1}^{(2)}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(11)}, \mathbf{y}_{\mathbf{b}}^{(10)}, \mathbf{y}_{\mathbf{b}}^{(01)}, \mathbf{y}_{\mathbf{b}}^{(00)}) \quad \text{for } \mathbf{b} \in \{0,1\}^{r_1-1}$$

for which we have, indexing the entries of $T_{r_1}^{(2)}[\mathbf{b}]$ with the 4 values $(1,1), (1,0), (0,1), (0,0)$,

$$L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c}) = T_{r_1}^{(2)}[\mathbf{b}][(\mathbf{f}_{(0,\mathbf{b}),\mathbf{c}}, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}})]$$

This allows the prover, when needed, to obtain $L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c})$ for any $(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-1} \times \{0,1\}^{r_2}$ with a simple read in $T_{r_1}^{(2)}$.

We now have, for $(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-3} \times \{0,1\}^{r_2}$,

$$\begin{aligned} A_0^{(2)}[\mathbf{b}, \mathbf{c}] &= L_{r_1}(\rho_1, 0, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\rho_1, 0, \mathbf{b}, 1, \mathbf{c}) \cdot \omega[\mathbf{b}] \\ &= T_{r_1}^{(2)}[0, \mathbf{b}, 0][(\mathbf{f}_{(0,0,\mathbf{b},0),\mathbf{c}}, \mathbf{f}_{(1,0,\mathbf{b},0),\mathbf{c}})] \cdot T_{r_1}^{(2)}[0, \mathbf{b}, 1][(\mathbf{f}_{(0,0,\mathbf{b},1),\mathbf{c}}, \mathbf{f}_{(1,0,\mathbf{b},1),\mathbf{c}})] \cdot \omega[\mathbf{b}] \end{aligned}$$

where $\omega[\mathbf{b}] = \mathbf{eq}(\rho_1, \mathbf{b}; \boldsymbol{\alpha}^{(i,1)})$ is the weight defined above for $j = 2$. Hence the elements $A_0^{(2)}[\mathbf{b}, \mathbf{c}]$ can be obtained by querying a table

$$T_{A_0}^{(2)} \in (\mathbb{K}^{16})^{\{0,1\}^{r_1-3}}$$

with

$$T_{A_0}^{(2)}[\mathbf{b}] = (T_{r_1}^{(2)}[0, \mathbf{b}, 0][\mathbf{x}] \cdot T_{r_1}^{(2)}[0, \mathbf{b}, 1][\mathbf{x}'] \cdot \omega[\mathbf{b}])_{\mathbf{x}, \mathbf{x}' \in \{0,1\}^2}$$

A table $T_{S_1}^{(2)}$ is computed analogously to T_{S_1} . The value $A_2^{(2)}[\mathbf{b}, \mathbf{c}]$, however, now depends on the value taken by *eight* witness bits, this being because for $\varepsilon \in \{0,1\}$, the value $E_{1\varepsilon}^{(2)} - E_{0\varepsilon}^{(2)}$ depends on four bits. Hence computing a table for the values $A_2^{(2)}[\mathbf{b}, \mathbf{c}]$ analogous to T_{A_2} would have $2^8 = 256$ -sized vectors as entries. Instead of computing $T_{A_2}^{(2)}$, we compute two tables for the values $E_{1\varepsilon}^{(2)} - E_{0\varepsilon}^{(2)}$ (one table for each $\varepsilon \in \{0,1\}$, with one of the tables also including multiplication by $\omega[\mathbf{b}]$). Then the value $A_2^{(2)}[\mathbf{b}, \mathbf{c}]$ can be obtained by performing a lookup into each table, followed by a single multiplication.

This can be iterated in further sumcheck rounds, for computing $h_3(X), h_4(X)$, etc. The limitation is that the entries in the tables are vectors whose dimension is squared at each round. More precisely, each further round squares the number of cases per index of every table, but it only halves the number of indices. Hence, in practice we stop using lookup tables after a few rounds of each sumcheck and GKR, and after that we execute the sumcheck protocol normally. Concretely, in our implementation we never compute a table with more than 16 elements per index. This is done so that the tables can be kept in cache, since we observe that, when this is not the case, table lookups become more and more expensive compared to multiplications in $\mathbb{F}_{2^{128}}$. [Table 11](#) displays the different tables built or not built in our simplest implementation, the arithmetic cost of doing so, and their size at $\ell_1 = 2^{r_1} = 2^{16}$ (corresponding to a total witness size of $\approx 2^{26}, 2^{27}$ bits in

our implementation). In our final implementation we do not explicitly compute all of the tables, similarly as done with the table $T_{A_2}^{(2)}$ above. See ?? for details.

Table	Dimension	Adds/mults. in $\mathbb{K}$	Size (MB), $\ell_1 = 2^{16}$	Built?
$T_{r_1}^{(1)}$	$(\mathbb{K}^2)^{2^{r_1}}$	0/0	2.1	yes
$T_{A_0}^{(1)}$	$(\mathbb{K}^{2^2})^{2^{r_1-2}}$	0 / 2^{r_1+1}	1.0	yes
$T_{S_1}^{(1)}$	$(\mathbb{K}^{2^2})^{2^{r_1-2}}$	0 / 2^{r_1+1}	1.0	yes
$T_{A_2}^{(1)}$	$(\mathbb{K}^{2^4})^{2^{r_1-2}}$	2^{r_1+1} / $5 \cdot 2^{r_1}$	4.2	yes
$T_{r_1}^{(2)}$	$(\mathbb{K}^{2^2})^{2^{r_1-1}}$	2^{r_1+2} / 2^{r_1+1}	2.1	yes
$T_{A_0}^{(2)}$	$(\mathbb{K}^{2^4})^{2^{r_1-3}}$	0 / 2^{r_1+2}	2.1	yes
$T_{S_1}^{(2)}$	$(\mathbb{K}^{2^4})^{2^{r_1-3}}$	0 / 2^{r_1+2}	2.1	yes
$T_{A_2}^{(2)}$	$(\mathbb{K}^{2^8})^{2^{r_1-3}}$	—	33.6	no
tables for $E_{1\varepsilon}^{(2)} - E_{0\varepsilon}^{(2)}$, $\varepsilon \in \{0, 1\}$	$(\mathbb{K}^{2^4})^{2^{r_1-3}}$ each	2^{r_1+2} / 2^{r_1+1}	2.1 each	yes
$T_{r_1}^{(3)}$	$(\mathbb{K}^{2^4})^{2^{r_1-2}}$	2^{r_1+3} / 2^{r_1+2}	4.2	yes
$T_{A_0}^{(3)}$	$(\mathbb{K}^{2^8})^{2^{r_1-4}}$	—	16.8	no
$T_{S_1}^{(3)}$	$(\mathbb{K}^{2^8})^{2^{r_1-4}}$	—	16.8	no
$T_{A_2}^{(3)}$	$(\mathbb{K}^{2^{16}})^{2^{r_1-4}}$	—	4295	no

Table 11: Dimensions, build costs, and sizes of the tables for computing $h_1(X), h_2(X), h_3(X)$ in the last sumcheck of [Construction 6.1](#), with $\log |\mathbb{K}| \approx 128$. We use the example size $\ell_1 = 2^{r_1} = 2^{16}$, which, following our instantiation ($\ell_1 \approx \ell^{0.6}$, [Section 4.7](#)), corresponds to a total witness size $\ell \approx 2^{26}, 2^{27}$. See ?? for the tables that our implementation does not build.

The rest of sumchecks in Step 2 of [Construction 6.1](#) So far we have discussed the last sumcheck from Step 2 of [Construction 6.1](#), corresponding to the lower layer of the binary tree of products of our GKR circuit for the grand products.

The sumchecks run for $i = r_1 - 2, r_1 - 3, \dots$ can be optimized in the exact same way as we did for the last sumcheck ($i = r_1 - 1$). In this case, instead of starting with the table $T_{r_1}^{(1)}$, which at each index contains the two values $(\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)})$, one starts with the table

$$T_{r_1-j} \in (\mathbb{K}^{2^{2^j}})^{\{0,1\}^{r_1-j}}$$

described in [Eq. \(56\)](#). Since T_{r_1-j} has, per index, the square of the number of entries of T_{r_1-j+1} , all subsequent tables also have, per index, the square of the number of entries of the analogous tables from the $(i+1)$ -th sumcheck, and they require correspondingly more operations to compute.

In our implementation we want to keep all tables in cache (otherwise we observe that multiplication in $\mathbb{F}_{2^{128}}$ is cheaper than a table lookup). Because of this, the smaller i is (i.e. the further we get from the last sumcheck in the GKR protocol), the fewer number of sumcheck rounds we use tables in. Concretely, for $i = r_1 - 2$ ($j = 1$) we build and use the table $T_{r_1-1}^{(1)}$, the analogues of $T_{A_0}^{(1)}$ and $T_{S_1}^{(1)}$ (now with 16 entries per index), and $T_{r_1-1}^{(2)}$. Similarly as done for $A_2^{(2)}$, the analogue of $T_{A_2}^{(1)}$ (which would have 2^8 entries per index) is replaced by two tables of 16 entries per index and one multiplication per read. For $i = r_1 - 3$ ($j = 2$) we only use the table $T_{r_1-2}^{(1)}$.

[Albert: More optimizations?](#)

7 Experiments

We implement the BitZ PCS [Construction 4.13](#) and an end-to-end proof system, called BitZ-SNARK, for integer R1CS constraints, as described in the introduction ([Section 1.1](#)) and in [Section 5](#) (note that BitZ can be used to prove much more general constraints, such as the ones in Zinc+ [\[AGK⁺26\]](#), cf. [Remark 5.2](#)).

Our repository can be found in <https://github.com/bitzcodes/BitZ.git>.

Our BitZ-PCS is instantiated with Reed-Solomon codes over $\mathbb{F}_{2^{128}}$ of rate $1/2$ or $1/8$. As backend of BitZ we use the Ligerito implementation from Flock [\[BRW26\]](#) (commit `e636760`, with local modifications) with folding arity 4. We set our experiments to 100 bits of round-by-round knowledge soundness¹⁷, except in the Limber [\[CXNB26\]](#) comparison, cf. [Section 7.2](#), where we target 114 bits to match Limber’s implementation.

When targeting 100 bits of round-by-round soundness we use the Johnson bound as proximity parameter, and perform out of domain sampling. When targeting more security (in [Section 7.2](#)) we use the unique decoding radius. Further instantiation details are given in [Section 4.7](#).

The Binius configuration suite. When comparing against Binius64 (commit `c28940a`, with local benchmarking patches), we use four configurations: Binius64 as in its repository, with rates $\rho = 1/2$ and $\rho = 1/8$ and with proximity parameter within the unique decoding radius. We call this “Binius (UDR) ρ ”. The original Binius64 repository uses Basefold [\[ZCF23\]](#) as its PCS. We also experiment with Binius64 using the same Ligerito instantiation as the one we use in BitZ. In this case we use the Johnson bound as the proximity parameter, add out of domain sampling, and call the resulting scheme “Binius (Johnson) ρ ”.

¹⁷This means that the largest round-by-round knowledge soundness of the protocols is at most 2^{-100} .

Bits	N	Scheme	Prover (ms)		Verifier (ms)		Proof (KB)	Peak mem. (GB)
			1 thr	10 thr	1 thr	10 thr		
32	2^{19}	BitZ-SNARK, rate 1/2	380	108	4.35	3.07	189	0.488
		BitZ-SNARK, rate 1/8	484	126	4.22	3.00	116	0.544
		Binius (UDR), rate 1/2	863	277	20.4	9.46	472	3.67
		Binius (UDR), rate 1/8	1052	319	19.9	8.99	330	3.93
		Limber (Brakedown)	4418	1348	192	147	12193	3.66
		Zinc+, rate 1/4	2847	757	31.5	14.4	1791	1.23
64	2^{19}	BitZ-SNARK, rate 1/2	785	206	7.64	4.37	202	0.871
		BitZ-SNARK, rate 1/8	989	249	7.34	4.22	124	0.985
		Binius (UDR), rate 1/2	860	269	15.7	7.20	472	3.17
		Binius (UDR), rate 1/8	1050	308	15.5	7.06	330	3.41
		Limber (Brakedown)	7234	2037	214	173	19135	4.89
128	2^{19}	BitZ-SNARK, rate 1/2	1742	462	14.0	6.61	217	1.56
		BitZ-SNARK, rate 1/8	2182	549	13.7	6.34	133	1.79
		Binius (UDR), rate 1/2	3756	1113	123	35.0	559	11.8
		Binius (UDR), rate 1/8	4650	1559	139	38.6	383	11.8
		Limber (Brakedown)	16419	4561	296	242	37022	9.47

Table 12: Excerpt from Tables 17 to 19. These record the cost of proving N multiplications $x \cdot y = z$ for random 32-, 64- and 128-bit nonnegative integers x and y (with z having twice the bit-size of x and y). 100 bits of round-by-round knowledge soundness. Cells with “—” indicate that the value was not measured due to insufficient memory. Ran on a MacBook Air M5, 24GB, CPU only.

Non-comparisons. The recent paper Flock [BRW26] builds upon the Binius64 scheme, and manages to prove over 300k SHA-256 compressions (and other hashes) per second on ten cores of an Apple M4 Max. Several of the innovations and optimizations of Flock have already been incorporated in the Binius64 repository. Flock was built with the purpose of proving many hash compressions in mind, while Binius64 is designed to prove much wider classes of computations. Because of this, we do not include comparisons with Flock, i.e. because Flock does not immediately support proving the computations we test BitZ on.

We make no claim that BitZ-SNARK is competitive with Flock or Binius64 for the purpose of proving a large amount of hashes (besides offering a simpler arithmetization via the CM relation from Eq. (23)). Indeed if seeking raw performance, Flock or Binius64 is, as of now, better suited for that task.

There is a series of works working on anonymous credentials that prove computations similar to SHA-256 followed by ECDSA signature [Fas24, KS25]. These works, additionally, prove JSON/CBOR document parsing, and are zero-knowledge. Because we do not support the latter at the moment of writing, we do not compare against them.

When proving computations with SHA-256 in them, we do not benchmark against Zinc+[AGK⁺26] because at these amounts of hash compressions it “runs out of memory.”

Usage of AI The scheme was initially implemented with AI. After that we thoroughly reviewed the verifier and, following the approach of Flock [BRW26] and other usages of AI [DHA26] to optimize against verified specs, we used AI extensively to optimize the prover while keeping the verifier unchanged. Subsequently we manually reviewed, refactored and further optimized both the

prover and verifier. The reported benchmark results correspond to this PoC implementation, which is the one provided in the anonymous repository <https://github.com/bitzcodes/BitZ.git>.

Additionally, we are currently developing a parallel production-grade handcrafted implementation, using more conventional methods.

As an experiment, we asked an AI to take the BitZ-PCS prover of the production-grade implementation and optimize it (keeping the verifier unchanged). The resulting optimized prover reproduced or even significantly improved the performance of the BitZ-PCS in the PoC implementation discussed above. This suggests that the published benchmark numbers still have room for improvement.

Additionally, AI was used to prepare the benchmarks in other repositories, and underwent revision by us. It was also used to run the benchmarks and organize the results into the tables in this section.

7.1 Raw performance of the BitZ PCS for the core LinBitsRings relation

In [Table 13](#) we measure the performance of [Construction 4.4](#) for proving the Core LinBitsRing relation $\mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, \mathbb{F}_q, \mathcal{C})$ following the instantiation in [Section 4.7](#). Namely, we prove that $\langle \mathbf{v}, \mathbf{f} \rangle = \mu$ over $\mathbb{F}_q$ for a vector of ℓ bits $\mathbf{f}$, and $\mathbf{v} \in \mathbb{F}_q^\ell, \mu \in \mathbb{F}_q$. We set $\mathbb{F}_{2^\nu} = \mathbb{F}_{2^{128}}$ and $\mathcal{C}$ is a Reed-Solomon code. We choose $|\mathbb{F}_q| < (2^{128} - 1)/\ell_1$ and we use the unique decoding radius as proximity parameter, so that Round 0 (OOD sampling) and Round 1 of [Construction 4.4](#) do not trigger. The goal of this experiments is to measure the performance of the grand product and the Ligerito commit and opening. In later experiments, Round 0 and Round 1 do trigger.

We set $\ell_1 = 2^{\lceil 0.6n \rceil}$ and $\ell_1 \cdot \ell_2 = \ell$ ([Section 4.7](#)).

ℓ	Prover steps, 10 thr (ms)				Prover total		Verifier (ms)		Proof size (KB)			Mem.
	Commit	Grand prod.	Ring s.	Lig.	1 thr	10 thr	1 thr	10 thr	Non-Lig.	Lig.	Total	
2^{20}	0.23	9.49	1.26	2.54	29.8	15.5	2.15	2.28	16.7	81.2	97.9	0.023
2^{21}	0.37	11.4	1.69	2.52	42.1	17.0	2.06	2.85	17.5	91.1	109	0.035
2^{22}	0.59	16.7	2.42	1.86	52.2	25.3	2.58	3.60	18.4	100	119	0.061
2^{23}	1.09	27.8	3.59	3.64	79.0	39.1	1.96	3.15	23.4	117	140	0.103
2^{24}	1.57	35.0	5.82	3.75	120	49.1	2.42	3.23	24.4	129	154	0.205
2^{25}	2.56	48.7	8.19	3.63	197	65.2	2.83	3.10	33.1	141	174	0.380
2^{26}	4.32	83.6	9.76	5.56	338	109	3.76	3.91	34.5	160	195	0.558
2^{27}	8.08	149	15.5	8.57	667	189	6.54	6.33	35.6	173	208	0.963
2^{28}	14.7	279	24.6	10.8	1275	336	6.96	6.99	52.6	186	238	1.87
2^{29}	31.8	596	34.6	18.3	2583	695	12.1	10.1	54.0	207	261	3.52
2^{30}	61.7	1202	56.9	28.6	5156	1363	12.6	10.7	87.4	223	310	6.96

Table 13: Cost of BitZ ([Construction 4.4](#)) for committing to $\ell = 2^n$ bits, $n = 20, \dots, 30$, and proving one claim $\langle \mathbf{v}, \mathbf{f} \rangle = \mu$ over $\mathbb{F}_q$. Configured for 100 bits of round-by-round knowledge soundness. The *grand product* column comprises Phase 3 of [Construction 4.4](#). The *ring switch* and *ligerito* columns comprise Phase 4. The *Non-Ligerito* proof size is comprised of the μ_j values in Round 2 of [Construction 4.4](#), the GKR and sumcheck messages, and the ring-switch messages. The ring switch step includes an initial sumcheck that transforms inner product claims to MLE evaluation claims. “Lig.” stands for Ligerito, “Mem.” stands for peak memory, and “Ring s.” for ring switch. Measured on a MacBook Air M5, 24 GB, CPU only.

7.2 RSA accumulation and Poseidon hashing

We compare Limber [CXNB26] (commit 853c6c4), Zinc+[AGK⁺26] and our BitZ-SNARK on proving the same circuit as in the Limber paper. This consists in verifying four exponentiations modulo a 2048-bit RSA modulus, with 352-bit exponents, plus a Poseidon-based hash-to-prime evaluation (see Section 7.1 of [CXNB26]). We take the corresponding R1CS constraint over the integers from the Limber repository. See Section 1.3 for further details on Limber, Zinc+, etc.

We set BitZ at 114 bits of security, matching Limber¹⁸. For the BitZ opening we use Ligerito in the unique-decoding regime (as opposed to the other benchmarks where we use the Johnson bound) at rate 1/8 with fold arity 4, with grinding to achieve 114 bits of security.

Scheme	Prover (ms)		Verifier (ms)		Proof (KB)
	1 thr	10 thr	1 thr	10 thr	
BitZ-SNARK	287	122	7.9	10.1	268
Zinc+	1876	523	16.1	12.1	1272
Limber (Brakedown)	1048	527	38.3	37.5	5769
Limber (Hyrax)	1104	350	33.8	20.2	175
Arkworks/Garuda [†]	231000	–	25	–	7.2
MultiSwap-x.Jsnark [†]	88500	–	2	–	0.2

Table 14: Proving one MultiSwap [OWWB20] computation (four exponentiations modulo a 2048-bit RSA modulus, with 352-bit exponents, plus a Poseidon-based hash-to-prime evaluation). The first four schemes prove the same R1CS constraint over the integers, taken from [CXNB26], and were run on a MacBook Air M5 24GB, CPU only. “Thr” stands for “threads”. The first four schemes achieve 114 bits of round-by-round knowledge soundness. Zinc+ proof compresses to 847KB with `zstd`. [†] Quoted from [CXNB26]; not measured here.

7.3 SHA-256 hashing followed by ECDSA signature verification

We experiment with proving the SHA-256 hash of a message of a few KB message followed by an ECDSA signature verification on the message digest. This is a common application of proof systems in anonymous credentials. We compare BitZ-SNARK with the Binius64 [Bin26] suite (Section 7). For the latter we compose two circuits in the Binius64 repo, one for SHA-256 and one for ECDSA signature verification.

For BitZ, we arithmetize the computation as an integer R1CS constraint with $\mathbb{F}_2$ -virtualization (Section 2.4). The SHA-256 constraints in particular are an instance of the CM relation from Eq. (23), obtained using the methods from Lemma 2.4.

We emphasize that, as done in the next experiment Section 7.4, when seeking to prove a large amount of hashes, it is preferable to arithmetize the hash over a binary field and follow the approach described in Section 7.4

We do not benchmark against Zinc+[AGK⁺26] because at these amounts of hash compressions it runs out of memory.

¹⁸Limber states in the paper that it achieves 128 bits of security, but its fingerprint step only achieves 114 bits of security in the implementation, because the random prime sampled to perform the fingerprint is of 128 bits, and no other measures (like, say, grinding) are taken.

Message	Scheme	Witgen	Prover (ms)		Verifier (ms)		Proof	Peak mem.
		(ms)	1 thr	10 thr	1 thr	10 thr	(KB)	(GB)
2 ⁴ comp.	960 B BitZ-SNARK, rate 1/2	2.41	46.5	18.9	3.62	3.70	118	0.789
	BitZ-SNARK, rate 1/8	2.40	45.6	19.1	3.38	3.43	74	0.791
	Binius (UDR), rate 1/2	9.50	149	58.1	11.1	5.81	362	0.841
	Binius (UDR), rate 1/8	9.28	174	63.3	11.0	5.61	266	0.886
	Binius (Johnson), rate 1/2	9.30	156	61.4	12.1	6.96	363	0.880
	Binius (Johnson), rate 1/8	9.29	188	67.3	11.7	6.45	194	0.909
2 ⁵ comp.	1984 B BitZ-SNARK, rate 1/2	2.47	49.8	20.3	3.59	3.60	119	0.793
	BitZ-SNARK, rate 1/8	2.48	55.2	21.3	3.35	3.40	76	0.794
	Binius (UDR), rate 1/2	9.53	149	58.1	11.2	5.80	362	0.887
	Binius (UDR), rate 1/8	9.52	177	63.0	11.2	5.64	266	0.930
	Binius (Johnson), rate 1/2	9.37	156	60.6	12.5	6.99	362	0.916
	Binius (Johnson), rate 1/8	9.55	187	67.0	12.0	6.42	193	0.948
2 ⁶ comp.	4032 B BitZ-SNARK, rate 1/2	2.40	53.9	21.8	3.81	3.71	125	0.788
	BitZ-SNARK, rate 1/8	2.37	55.9	22.6	3.68	3.55	83	0.787
	Binius (UDR), rate 1/2	9.49	150	58.1	11.6	5.95	362	0.879
	Binius (UDR), rate 1/8	9.47	177	64.0	11.6	5.87	266	0.928
	Binius (Johnson), rate 1/2	9.46	158	61.8	12.7	7.03	364	0.918
	Binius (Johnson), rate 1/8	9.45	184	66.6	12.2	6.63	194	0.951
2 ⁷ comp.	8128 B BitZ-SNARK, rate 1/2	2.35	68.4	25.7	4.14	3.82	136	0.790
	BitZ-SNARK, rate 1/8	2.40	67.9	26.0	4.02	3.61	92	0.793
	Binius (UDR), rate 1/2	9.96	168	63.0	12.7	6.37	394	0.941
	Binius (UDR), rate 1/8	9.96	219	74.0	12.7	6.29	289	1.03
	Binius (Johnson), rate 1/2	9.73	179	66.9	14.0	7.77	376	1.00
	Binius (Johnson), rate 1/8	9.77	231	77.6	13.6	7.16	200	1.06
	Zinc+	–	4707	970	319	115	2432	2.03

Table 15: Costs of proving a SHA-256 hash of a message m , and then proving correct verification of a P-256 ECDSA signature on the resulting digest. Below message sizes we record the corresponding number of SHA-256 compressions (comp.). Set at $\lambda = 100$ bits of round-by-round knowledge soundness. Ran on a MacBook Air M5, 24 GB. The Zinc+ proof compresses to 915 KB with `zstd`.

7.4 Modular multiplications and bit operations (mod 2^{32} multiplication and SHA-256 hashing)

We prove simultaneously a large chain of SHA-256 compressions, together with a large amount of multiplications modulo 2^{32} . In this experiment, contrary to the previous one, SHA-256 is arithmetized over a binary field, using the R1CS constraint from Binius64 [Bin26]. The modular multiplications are arithmetized as an R1CS constrain over the integers, where each modular multiplication $x \cdot y = z \pmod{2^{32}}$ is expressed as an integer constraint of the form $x \cdot y = z + 2^{32} \cdot u$, where u is a new integer witness.

This is an artificial circuit. For example, the mod 2^{32} integers are not related in any manner to the SHA-256 compressions. We see it as a proxy of computations that require many bit operations and integer/modular multiplications, e.g. RISC-V derived computations. The cost of “gluing” witness elements between one type of computation and another, both in BitZ-SNARK and Binius64, is essentially non-existent, because both schemes commit to all the bits of witness elements separately,

and these can be interpreted as integers or binary field elements freely.

Our end-to-end $\mathbb{B}it\mathbb{Z}$ -SNARK is a hybrid construction combining a the PIOP over $\mathbb{Z}$ from [Section 5](#), together with the Binius64 PIOP over binary fields. The PCS opening proofs are eventually batched together, both of them being over binary fields. To arithmetize modular multiplication in Binius64 we use an approach similar to that for multiplying 32-bit integers, explained in [Section 7.5](#).

N	M	Scheme	Prover (ms)		Verifier (ms)		Proof (KB)	Peak mem. (GB)
			1 thr	10 thr	1 thr	10 thr		
2^{15}	2^7	BitZ-SNARK, rate 1/2	41.8	24.2	3.59	3.87	179	0.189
		BitZ-SNARK, rate 1/8	50.6	25.1	3.24	3.49	107	0.152
		Binius (UDR), rate 1/2	69.0	36.8	3.29	3.20	353	0.410
		Binius (UDR), rate 1/8	85.9	40.4	3.18	3.09	264	0.414
		Binius (Johnson), rate 1/2	75.4	39.2	4.03	4.00	345	0.421
		Binius (Johnson), rate 1/8	90.8	41.8	3.57	3.49	180	0.432
2^{16}	2^8	BitZ-SNARK, rate 1/2	88.1	35.1	5.09	4.54	200	0.368
		BitZ-SNARK, rate 1/8	94.9	36.4	4.79	4.27	118	0.368
		Binius (UDR), rate 1/2	129	55.8	5.81	3.95	388	0.822
		Binius (UDR), rate 1/8	160	62.0	5.65	3.71	282	0.776
		Binius (Johnson), rate 1/2	140	59.1	6.66	4.85	358	0.669
		Binius (Johnson), rate 1/8	161	63.5	6.21	4.31	188	0.692
2^{17}	2^9	BitZ-SNARK, rate 1/2	158	53.8	9.06	5.84	219	0.683
		BitZ-SNARK, rate 1/8	186	60.0	8.85	5.44	127	0.704
		Binius (UDR), rate 1/2	254	94.0	10.9	5.33	423	1.60
		Binius (UDR), rate 1/8	316	106	10.6	5.29	309	1.62
		Binius (Johnson), rate 1/2	256	95.9	11.9	6.71	369	1.74
		Binius (Johnson), rate 1/8	311	106	11.3	5.74	193	1.72
2^{18}	2^{10}	BitZ-SNARK, rate 1/2	303	90.1	14.6	6.61	253	0.988
		BitZ-SNARK, rate 1/8	372	105	14.3	6.34	149	1.11
		Binius (UDR), rate 1/2	509	168	20.9	7.35	471	3.21
		Binius (UDR), rate 1/8	630	197	20.5	7.14	334	3.25
		Binius (Johnson), rate 1/2	507	174	21.4	8.24	391	3.27
		Binius (Johnson), rate 1/8	610	191	21.1	7.64	208	3.47
2^{19}	2^{11}	BitZ-SNARK, rate 1/2	585	156	27.7	10.2	274	2.67
		BitZ-SNARK, rate 1/8	842	204	27.3	9.84	160	2.75
		Binius (UDR), rate 1/2	1004	319	44.3	15.4	516	5.98
		Binius (UDR), rate 1/8	1226	369	43.6	14.8	360	5.94
		Binius (Johnson), rate 1/2	1021	326	45.1	16.1	405	6.14
		Binius (Johnson), rate 1/8	1225	370	44.6	15.6	215	6.38
2^{20}	2^{12}	BitZ-SNARK, rate 1/2	1132	287	51.1	15.7	303	5.30
		BitZ-SNARK, rate 1/8	1456	360	50.6	15.9	176	5.48
		Binius (UDR), rate 1/2	1995	617	87.5	27.3	569	12.7
		Binius (UDR), rate 1/8	2417	710	85.8	26.0	387	12.7
		Binius (Johnson), rate 1/2	2031	628	87.8	27.9	418	12.7
		Binius (Johnson), rate 1/8	2439	717	86.8	26.9	222	12.8

Table 16: Cost of proving N multiplications modulo 2^{32} and $M = N/256$ chained SHA-256 compressions (equal packed witnesses for the two branches). Configured at $\lambda = 100$ bits of round-by-round knowledge soundness. *Prover* includes witness synthesis and the commitments. The choice of N and M is done so that both types of workloads (modular multiplication and SHA-256) amount to roughly the same number of witness bits. Ran on a MacBook Air M5, 24 GB, CPU only.

7.5 Integer multiplication

We measure the cost of proving N integer multiplications of the form $x \cdot y = z$, for x, y random t -bit integers, and z a $2 \cdot t$ -bit integer, for $t \in \{32, 64, 128\}$. This is proved via our implemented end-to-end proof system, called BitZ-SNARK for R1CS over the integers, with BitZ as its PCS, as described in Section 5. For 32-bit integer multiplication, we compare against the Binius suite and Limber. For the sake of brevity, for multiplication of larger bits we compare only against the Binius suite.

Binius64 operates natively with 64 bit words, and has a native constraint for the product of two such words x, y . It's output is two 64-bit words z_1, z_2 such that $x \cdot y = z_1 + 2^{64} \cdot z_2$. Our 64-bit multiplication in Binius64 uses this functionality. For 32 bit products we constrain the low word z_1 to be $x \cdot y$ and the high word z_2 , as well as the upper 32 bits of x and y , to be zero. For $t = 128$ we use the multiplication circuit from the Binius64 repository <https://github.com/binius-zk/binius64/blob/e0ddeb91d3826457322e3b7434a8ca0625f2f56e/crates/circuits/src/bignum/mul.rs#L27-L42>.

For Limber we use the specific integer multiplication benchmark in Limber's repository.

N	Scheme	Witgen	Prover (ms)		Verifier (ms)		Proof	Peak mem.
		(ms)	1 thr	10 thr	1 thr	10 thr	(KB)	(GB)
2^{15}	BitZ-SNARK, rate 1/2	0.01	30.1	15.4	2.06	2.23	120	0.063
	BitZ-SNARK, rate 1/8	0.01	35.5	17.1	1.88	1.95	70.6	0.062
	Binius (UDR), rate 1/2	0.57	61.6	32.8	1.60	1.63	324	0.282
	Binius (UDR), rate 1/8	0.58	75.1	36.5	1.49	1.52	243	0.285
	Binius (Johnson), rate 1/2	0.53	64.5	34.8	2.59	2.75	342	0.291
	Binius (Johnson), rate 1/8	0.55	82.1	37.9	2.14	2.26	180	0.289
	Plonky3 (FRI), rate 1/2	—	—	—	—	—	—	—
	Limber (Brakedown)	0.42	234	138	32.8	32.4	3354	0.344
	Zinc+, rate 1/4	0.63	186	113	28.4	13.1	632	0.098
2^{17}	BitZ-SNARK, rate 1/2	0.04	98.2	36.7	2.76	2.55	152	0.150
	BitZ-SNARK, rate 1/8	0.04	123	40.9	2.55	2.22	91.3	0.165
	Binius (UDR), rate 1/2	2.49	218	82.0	4.81	2.65	388	0.921
	Binius (UDR), rate 1/8	2.41	268	93.5	4.70	2.67	283	1.000
	Binius (Johnson), rate 1/2	2.22	228	86.2	6.19	4.37	371	0.946
	Binius (Johnson), rate 1/8	2.21	282	95.7	5.67	3.70	193	1.00
	Plonky3 (FRI), rate 1/2	—	—	—	—	—	—	—
	Limber (Brakedown)	1.63	1136	415	66.1	58.1	5514	1.05
	Zinc+, rate 1/4	1.89	721	252	29.1	13.6	866	0.343
2^{19}	BitZ-SNARK, rate 1/2	0.17	380	108	4.35	3.07	189	0.488
	BitZ-SNARK, rate 1/8	0.17	484	126	4.22	3.00	116	0.544
	Binius (UDR), rate 1/2	9.81	863	277	20.4	9.46	472	3.67
	Binius (UDR), rate 1/8	9.78	1052	319	19.9	8.99	330	3.93
	Binius (Johnson), rate 1/2	9.20	886	284	21.3	10.3	404	3.76
	Binius (Johnson), rate 1/8	9.16	1081	318	20.9	9.83	215	3.98
	Plonky3 (FRI), rate 1/2	—	—	—	—	—	—	—
	Limber (Brakedown)	6.63	4418	1348	192	147	12193	3.66
	Zinc+, rate 1/4	6.96	2847	757	31.5	14.4	1791	1.23
2^{21}	BitZ-SNARK, rate 1/2	0.67	1537	388	8.00	5.02	225	1.77
	BitZ-SNARK, rate 1/8	0.64	1979	469	7.62	4.47	140	2.06
	Binius (UDR), rate 1/2	—	3443	—	79.9	—	559	—
	Binius (UDR), rate 1/8	—	4211	—	80.5	—	383	—
	Binius (Johnson), rate 1/2	—	3568	—	79.0	—	442	—
	Binius (Johnson), rate 1/8	—	—	—	—	—	237	—
	Plonky3 (FRI), rate 1/2	—	—	—	—	—	—	—
	Limber (Brakedown)	—	—	—	—	—	—	—
	Zinc+, rate 1/4	27.5	11307	2729	34.8	15.5	5552	4.87
2^{23}	BitZ-SNARK, rate 1/2	2.69	7134	1948	14.1	7.36	279	6.69
	BitZ-SNARK, rate 1/8	2.71	9099	2367	13.7	6.95	179	7.87
	Binius (UDR), rate 1/2	—	—	—	—	—	—	—
	Binius (UDR), rate 1/8	—	—	—	—	—	—	—
	Binius (Johnson), rate 1/2	—	—	—	—	—	—	—
	Binius (Johnson), rate 1/8	—	—	—	—	—	—	—
	Plonky3 (FRI), rate 1/2	—	—	—	—	—	—	—
	Limber (Brakedown)	—	—	—	—	—	—	—
	Zinc+, rate 1/4	114	50130	14468	51.5	19.4	20325	15.9

Table 17: BitZ-SNARK cost of proving $N = 2^n$ integer multiplications $x \cdot y = z$ for random 32-bit non-negative integers x, y (with z a 64-bit integer). Configured at 100 bits of round-by-round knowledge soundness (cf. [Section 4.7](#)). Cells with “—” indicate the computer ran out of memory. Ran on a MacBook Air M5, 24 GB.

N	Scheme	Witgen	Prover (ms)		Verifier (ms)		Proof	Peak mem.
		(ms)	1 thr	10 thr	1 thr	10 thr	(KB)	(GB)
2^{15}	BitZ-SNARK, rate 1/2	0.02	56.1	24.9	2.52	2.27	137	0.082
	BitZ-SNARK, rate 1/8	0.02	63.5	26.7	2.43	2.13	84.1	0.098
	Binius (UDR), rate 1/2	0.42	61.7	33.3	1.34	1.30	324	0.224
	Binius (UDR), rate 1/8	0.42	74.6	36.2	1.21	1.15	242	0.242
	Binius (Johnson), rate 1/2	0.38	65.3	35.1	2.28	2.42	345	0.233
	Binius (Johnson), rate 1/8	0.37	83.6	38.7	1.87	1.90	180	0.244
	Limber (Brakedown)	0.41	380	187	34.2	33.8	4053	0.478
2^{17}	BitZ-SNARK, rate 1/2	0.09	194	62.1	4.57	3.46	164	0.251
	BitZ-SNARK, rate 1/8	0.09	245	72.0	4.40	3.11	98.5	0.282
	Binius (UDR), rate 1/2	1.74	218	81.3	3.61	2.08	387	0.840
	Binius (UDR), rate 1/8	1.77	268	92.6	3.53	1.94	283	0.915
	Binius (Johnson), rate 1/2	1.59	225	84.2	5.07	3.75	370	0.872
	Binius (Johnson), rate 1/8	1.60	276	92.9	4.45	3.06	191	0.932
	Limber (Brakedown)	1.61	1825	583	72.0	62.6	7310	1.40
2^{19}	BitZ-SNARK, rate 1/2	0.38	785	206	7.64	4.37	202	0.871
	BitZ-SNARK, rate 1/8	0.38	989	249	7.34	4.22	124	0.985
	Binius (UDR), rate 1/2	6.95	860	269	15.7	7.20	472	3.17
	Binius (UDR), rate 1/8	6.92	1050	308	15.5	7.06	330	3.41
	Binius (Johnson), rate 1/2	6.42	878	278	16.6	8.60	403	3.29
	Binius (Johnson), rate 1/8	6.40	1080	318	15.6	8.19	214	3.51
	Limber (Brakedown)	6.60	7234	2037	214	173	19135	4.89
2^{21}	BitZ-SNARK, rate 1/2	1.43	3019	820	7.79	4.81	264	3.45
	BitZ-SNARK, rate 1/8	1.43	3929	937	7.63	4.61	171	3.86
	Binius (UDR), rate 1/2	27.7	3420	1038	60.4	20.9	559	11.4
	Binius (UDR), rate 1/8	27.8	4185	1233	59.9	21.5	383	11.2
	Binius (Johnson), rate 1/2	25.5	3623	1079	61.8	22.1	438	11.9
	Binius (Johnson), rate 1/8	26.3	4535	1353	61.0	22.9	236	12.3
	Limber (Brakedown)	—	—	—	—	—	—	—

Table 18: Costs of proving $N = 2^n$ multiplications $x \cdot y = z$ for random 64 bit nonnegative integers x and y (with z a 128-bit integer). 100 bits of round-by-round knowledge soundness (Binius64 is at 96 bits). Ran on a MacBook Air M5, 24GB, CPU-only.

N	Scheme	Witgen	Prover (ms)		Verifier (ms)		Proof	Peak mem.
		(ms)	1 thr	10 thr	1 thr	10 thr	(KB)	(GB)
2^{15}	BitZ-SNARK, rate 1/2	0.05	100	38.5	4.22	3.03	149	0.145
	BitZ-SNARK, rate 1/8	0.05	125	42.3	4.06	2.76	90.7	0.158
	Binius (UDR), rate 1/2	1.69	239	86.3	7.52	3.68	388	1.02
	Binius (UDR), rate 1/8	1.67	288	96.7	7.40	3.49	283	1.10
	Binius (Johnson), rate 1/2	1.45	247	90.3	8.92	5.36	371	1.05
	Binius (Johnson), rate 1/8	1.46	295	99.2	8.41	4.71	192	1.11
	Limber (Brakedown)	0.94	1042	366	42.2	38.8	5550	0.791
2^{17}	BitZ-SNARK, rate 1/2	0.19	391	115	7.42	4.13	186	0.452
	BitZ-SNARK, rate 1/8	0.19	492	131	7.18	3.90	112	0.509
	Binius (UDR), rate 1/2	6.46	945	290	31.1	11.4	472	3.95
	Binius (UDR), rate 1/8	6.52	1132	327	30.8	11.3	331	4.19
	Binius (Johnson), rate 1/2	5.82	959	299	32.4	12.9	403	4.06
	Binius (Johnson), rate 1/8	5.88	1166	338	31.8	12.4	215	4.31
	Limber (Brakedown)	3.66	4027	1184	94.9	80.2	12233	2.56
2^{19}	BitZ-SNARK, rate 1/2	0.72	1742	462	14.0	6.61	217	1.56
	BitZ-SNARK, rate 1/8	0.73	2182	549	13.7	6.34	133	1.79
	Binius (UDR), rate 1/2	27.2	3756	1113	123	35.0	559	11.8
	Binius (UDR), rate 1/8	32.8	4650	1559	139	38.6	383	11.8
	Binius (Johnson), rate 1/2	35.8	4018	1623	166	165	442	11.0
	Binius (Johnson), rate 1/8	66.0	6825	2706	372	347	238	10.5
	Limber (Brakedown)	15.8	16419	4561	296	242	37022	9.47
2^{21}	BitZ-SNARK, rate 1/2	2.80	7474	2178	26.2	10.7	263	5.97
	BitZ-SNARK, rate 1/8	2.86	9580	2747	25.9	10.0	163	7.13
	Binius (UDR), rate 1/2	3668	214991	79746	7803	3356	655	—
	Binius (UDR), rate 1/8	3283	236548	88614	8432	4020	437	—
	Binius (Johnson), rate 1/2	3609	222199	85764	8116	3450	472	—
	Binius (Johnson), rate 1/8	—	—	—	—	—	—	—
	Limber (Brakedown)	—	—	—	—	—	—	—

Table 19: Costs of proving $N = 2^n$ multiplications $x \cdot y = z$ for random 128 bit nonnegative integers x and y (with z a 256-bit integer). 100 bits of round-by-round knowledge soundness. Ran on a MacBook Air M5, 24 GB, CPU only.

7.6 Comparison with Fields-witch

We compare experimentally BitZ with Fields-witch [Sou26], a PCS-over the prime field $\mathbb{F}_{2^{127}-1}$ that commits its witness over the binary field $\mathbb{F}_{2^{127}}$. See Section 2.3.1 for a discussion of [Sou26] and a conceptual comparison of both. Fields-witch repository is available in <https://github.com/morgana-proofs/fields-witch> (commit 30cca8c), and it uses Ligerito as its underlying binary-field PCS, configured at the unique decoding radius.

Bits	Scheme	Prover (ms)		Verifier (ms)		Proof (KB)	Peak mem. (GB)
		1 thr	10 thr	1 thr	10 thr		
2^{21}	BitZ-SNARK, rate 1/2	15.3	8.79	1.52	1.64	108	0.04
	BitZ-SNARK, rate 1/8	21.6	10.6	1.35	1.45	66	0.04
	Fields-Witch, rate 1/2	23.5	18.5	6.14	6.83	430	0.08
	Fields-Witch, rate 1/8	25.5	18.9	5.62	6.23	370	0.08
	Fields-Witch + BitZ opener, rate 1/2	28.8	20.8	6.14	6.83	368	0.08
	Fields-Witch + BitZ opener, rate 1/8	29.5	20.4	5.67	6.30	279	0.09
2^{23}	BitZ-SNARK, rate 1/2	51.6	20.0	2.03	2.12	141	0.11
	BitZ-SNARK, rate 1/8	52.1	20.6	1.94	2.02	86	0.13
	Fields-Witch, rate 1/2	72.3	39.7	6.97	7.72	574	0.24
	Fields-Witch, rate 1/8	82.4	41.4	6.62	7.31	482	0.25
	Fields-Witch + BitZ opener, rate 1/2	83.7	43.1	6.98	7.77	472	0.26
	Fields-Witch + BitZ opener, rate 1/8	86.8	44.5	6.58	7.29	345	0.27
2^{25}	BitZ-SNARK, rate 1/2	177	51.8	3.61	3.45	174	0.37
	BitZ-SNARK, rate 1/8	204	58.2	3.39	3.25	108	0.50
	Fields-Witch, rate 1/2	255	100	7.39	8.50	710	0.84
	Fields-Witch, rate 1/8	300	110	6.95	8.31	575	0.91
	Fields-Witch + BitZ opener, rate 1/2	263	105	7.62	8.82	574	0.92
	Fields-Witch + BitZ opener, rate 1/8	295	116	6.96	8.00	408	0.97
2^{27}	BitZ-SNARK, rate 1/2	700	169	9.73	7.01	208	0.98
	BitZ-SNARK, rate 1/8	722	192	9.48	6.73	129	1.17
	Fields-Witch, rate 1/2	860	252	7.11	8.19	852	2.82
	Fields-Witch, rate 1/8	1053	292	6.62	7.89	687	3.06
	Fields-Witch + BitZ opener, rate 1/2	857	263	7.31	8.41	674	3.09
	Fields-Witch + BitZ opener, rate 1/8	968	300	6.59	7.86	473	3.32
2^{29}	BitZ-SNARK, rate 1/2	2953	687	18.4	11.8	259	3.46
	BitZ-SNARK, rate 1/8	2971	793	18.4	11.4	167	4.46
	Fields-Witch, rate 1/2	3264	884	8.32	9.59	1004	10.5
	Fields-Witch, rate 1/8	4099	1049	7.86	9.14	808	11.4
	Fields-Witch + BitZ opener, rate 1/2	3207	912	8.57	9.96	792	11.5
	Fields-Witch + BitZ opener, rate 1/8	3611	1079	7.62	8.93	554	12.5

Table 20: Cost of proving an MLE claim for a vector of 2^{m-7} field elements from $\mathbb{F}_{2^{127-1}}$, for $m \in \{21, 23, 25, 27\}$. 100 bits of security with fields-witch (Soukhanov’s implementation of [Sou26]) and with BitZ. Ran on a MacBook Air M5, 24 GB.

References

- [0xP] 0xPARC. ZK bug tracker. GitHub repository.
- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: reed-solomon proximity testing with fewer queries. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 380–413. Springer, 2024.

- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: reed-solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 214–243. Springer, 2025.
- [AGK⁺26] Alexander Abdugafarov, Albert Garreta, Amit Kumar, Michał Osadnik, Psi Vesely, Ilia Vlasov, and Kai Zhe Zheng. Zinc+: SNARKs for polynomial rings. Cryptology ePrint Archive, Paper 2026/855, 2026.
- [BBMHS22] Carsten Baum, Lennart Braun, Alexander Munch-Hansen, and Peter Scholl. Moz z 2 k arella: efficient vector-ole and zero-knowledge proofs over z 2 k. In *Annual International Cryptology Conference*, pages 329–358. Springer, 2022.
- [BCFW25] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. Linear-time accumulation schemes. In *Theory of Cryptography - 23rd International Conference, TCC 2025*, 2025. Numbering as in the full version, Cryptology ePrint Archive, Paper 2025/753.
- [BDDT25] Suyash Bagad, Quang Dao, Yuval Domb, and Justin Thaler. Speeding up sum-check proving. Cryptology ePrint Archive, Paper 2025/1117, 2025.
- [BFK⁺24] Alexander R. Block, Zhiyong Fang, Jonathan Katz, Justin Thaler, Hendrik Waldner, and Yupeng Zhang. Field-agnostic snarks from expand-accumulate codes. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 276–307. Springer, 2024.
- [BFS19] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. Cryptology ePrint Archive, Paper 2019/1229, 2019. EUROCRYPT 2020.
- [BG23] Jeremy Bruestle and Paul Gafni. Risc zero zkvm: scalable, transparent arguments of risc-v integrity. *Draft. July*, 29, 2023.
- [BHR⁺21] Alexander R. Block, Justin Holmgren, Alon Rosen, Ron D. Rothblum, and Pratik Soni. Time- and space-efficient arguments from groups of unknown order. Cryptology ePrint Archive, Paper 2021/358, 2021. CRYPTO 2021.
- [Bin26] Binius64. Binius64 whitepaper. GitHub repository, 2026.
- [BRW26] Benedikt Bünz, Ron Rothblum, and William Wang. Flock: Fast proving for batch boolean computations. Cryptology ePrint Archive, Paper 2026/1329, 2026.
- [BS23] Ward Beullens and Gregor Seiler. Labrador: compact proofs for r1cs from module-sis. In *Annual International Cryptology Conference*, pages 518–548. Springer, 2023.

- [BSBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17, Dagstuhl, Germany, 2018. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [BSBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.
- [BSCG⁺19] Eli Ben-Sasson, Alessandro Chiesa, Lior Goldberg, Tom Gur, Michael Riabzev, and Nicholas Spooner. Linear-size constant-query IOPs for delegating computation. Cryptology ePrint Archive, Paper 2019/1230, 2019. TCC 2019.
- [BSCKL22] Eli Ben-Sasson, Dan Carmon, Swastik Kopparty, and David Levit. Scalable and transparent proofs over all large fields, via elliptic curves (ECFFT part II). Cryptology ePrint Archive, Paper 2022/1542, 2022.
- [CCH⁺19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat–Shamir: From practice to theory. In *51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 1082–1090, 2019. Full version: Cryptology ePrint Archive, Paper 2018/1004.
- [CET⁺24] Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara, and Benjamin Livshits. SoK: What don’t we know? Understanding security vulnerabilities in SNARKs. USENIX Security 2024; Cryptology ePrint Archive, Paper 2024/514, 2024.
- [CFW26] Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. Zero-knowledge IOPPs for constrained interleaved codes. Cryptology ePrint Archive, Paper 2026/391, 2026.
- [CHA24] Matteo Campanelli and Mathias Hall-Andersen. Fully succinct arguments over the integers from first principles. Cryptology ePrint Archive, Paper 2024/1548, 2024.
- [CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437. Springer, 2017.
- [CXNB26] Jessica Chen, Lucas Xia, Wilson Nguyen, and Benedikt Bünz. Limber: Low overhead SNARKs for integers from any PCS. Cryptology ePrint Archive, Paper 2026/1635, 2026.
- [DHA26] Giorgio Dell’Immagine and Mathias Hall-Andersen. zk.golf: Fearless and collaborative optimization of circuits. ZK/SEC Quarterly (zkSecurity blog), 2026. <https://zk.golf>.

- [DP23] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784, 2023. EUROCRYPT 2025.
- [DP24] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinear over binary towers. Cryptology ePrint Archive, Paper 2024/504, 2024.
- [DT24] Quang Dao and Justin Thaler. More optimizations to sum-check proving. Cryptology ePrint Archive, Paper 2024/1210, 2024.
- [Fas24] Matteo Frigo and abhi shelat. Anonymous credentials from ECDSA. Cryptology ePrint Archive, Paper 2024/2010, 2024.
- [Fen24a] Giacomo Fenzi. STIR: Rust implementation. GitHub repository, 2024.
- [Fen24b] Giacomo Fenzi. WHIR: Rust implementation. GitHub repository, 2024.
- [GNSV23] Chaya Ganesh, Anca Nitulescu, and Eduardo Soria-Vazquez. Rinocchio: Snarks for ring arithmetic. *Journal of Cryptology*, 36(4):41, 2023.
- [Gru24] Angus Gruen. Some improvements for the PIOP for ZeroCheck. Cryptology ePrint Archive, Paper 2024/108, 2024.
- [GWV⁺25] Albert Garreta, Hendrik Waldner, Ilia Vlasov, Katerina Hristova, Luca Dall’Ava, Marko Čupić, and Matthew Klein. Zinc: Succinct arguments with small arithmetization overheads from iops of proximity to the integers. In *Advances in Cryptology – CRYPTO 2025: 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part VII*, pages 259–291, Berlin, Heidelberg, 2025. Springer-Verlag.
- [HLN23] Ulrich Haböck, Daniel Lubarov, and Jacqueline Nabaglo. Reed-Solomon codes over the circle group. Cryptology ePrint Archive, Paper 2023/824, 2023.
- [HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Paper 2024/278, 2024.
- [HMZ25] Mi-Ying Miryam Huang, Xinyu Mao, and Jiapeng Zhang. Sublinear proofs over polynomial rings. Cryptology ePrint Archive, Paper 2025/199, 2025.
- [Imm23] ImmuneFi Editor. Aztec multiple-spend error bugfix review. ImmuneFi blog, 2023.
- [KLOT25] Shuto Kuriyama, Russell W. F. Lai, Michał Osadnik, and Lorenzo Tucci. SALSAA – sumcheck-aided lattice-based succinct arguments and applications. Cryptology ePrint Archive, Paper 2025/2124, 2025.
- [KPS18] Ahmed Kosba, Charalampos Papamanthou, and Elaine Shi. xJsnark: A framework for efficient verifiable computation. IEEE Symposium on Security and Privacy 2018, 2018.
- [KS25] Darya Kaviani and Srinath Setty. Vega: Low-latency zero-knowledge proofs over existing credentials. Cryptology ePrint Archive, Paper 2025/2094, 2025.

- [NA25] Andrija Novakovic and Guillermo Angeris. Ligerito: A small and concretely fast polynomial commitment scheme. Cryptology ePrint Archive, Paper 2025/1187, 2025.
- [NOZ26] Ngoc Khanh Nguyen, George O’Rourke, and Jiapeng Zhang. Hachi: Efficient lattice-based multilinear polynomial commitments over extension fields. Cryptology ePrint Archive, Paper 2026/156, 2026.
- [NS24] Ngoc Khanh Nguyen and Gregor Seiler. Greyhound: Fast polynomial commitments from lattices. Cryptology ePrint Archive, Paper 2024/1293, 2024. CRYPTO 2024.
- [OKMZ24] Michele Orrù, George Kadianakis, Mary Maller, and Greg Zaverucha. Beyond the circuit: How to minimize foreign arithmetic in zkp circuits. *Cryptology ePrint Archive, Paper 2024/265*, 2024.
- [OWWB20] Alex Ozdemir, Riad S. Wahby, Barry Whitehat, and Dan Boneh. Scaling verifiable computation using efficient set accumulators. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2075–2092, 2020.
- [Pol22] Polygon Zero. Plonky2: Fast recursive arguments with PLONK and FRI. GitHub repository, 2022.
- [Set26] Srinath Setty. BinarySpartan: Spartan over binary fields. Cryptology ePrint Archive, Paper 2026/1656, 2026.
- [Sou24] Lev Soukhanov. Hashcaster: batched small-field sumcheck using Frobenius twists. GitHub repository, 2024.
- [Sou25] Lev Soukhanov. Logup*: faster, cheaper logup argument for small-table indexed lookups. Cryptology ePrint Archive, Paper 2025/946, 2025.
- [Sou26] Lev Soukhanov. Char 2 fieldswitch. HackMD post, 2026.
- [SSPP26] Alireza Shirzad, Sriram Sridhar, Dimitrios Papadopoulos, and Charalampos Papanthou. Relaxed modular PCS from arbitrary PCS and applications to SNARKs for integers. Cryptology ePrint Archive, Paper 2026/347, 2026.
- [Sta21] StarkWare. ethSTARK documentation. Online, 2021. Version 1.2.
- [SV22] Eduardo Soria-Vazquez. Doubly efficient interactive proofs over infinite and non-commutative rings. In *Theory of Cryptography Conference*, pages 497–525. Springer, 2022.
- [Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Advances in Cryptology – CRYPTO 2013, Part II*, volume 8043 of *Lecture Notes in Computer Science*, pages 71–89. Springer, 2013.
- [Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. 01 2022.
- [WSC⁺24] Hongbo Wen, Jon Stephens, Yanju Chen, Kostas Ferles, Shankara Pailoor, Kyle Charbonnet, Isil Dillig, and Yu Feng. Practical security analysis of zero-knowledge proof circuits. USENIX Security 2024; Cryptology ePrint Archive, Paper 2023/190, 2024.

- [WZD25] Yuanju Wei, Xinxuan Zhang, and Yi Deng. Transparent SNARKs over galois rings. Cryptology ePrint Archive, Paper 2025/263, 2025.
- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes. Cryptology ePrint Archive, Paper 2023/1705, 2023.

A Deferred proofs

A.1 Completeness and soundness of [Construction 4.13](#)

We prove [Theorem 4.15](#). For readability, we temporarily denote $\pi : \mathbb{Z} \rightarrow R$ for the reduction modulo $\text{char}(R)$ and we let

$$\mathcal{R}_{\text{gen}} := \mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{<D_S}, \psi, R, \mathcal{C}, L), \quad \mathcal{R}_{\text{core}} := \mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C}), \quad \mathbf{x}' := (\mu, \mathbf{u}),$$

where $\mathbf{u} = \psi(L)^\top \cdot \mathbf{v}$ as in [Construction 4.13](#), and $\mathcal{R}_{\text{PESAT}} = \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu)$, $\mathcal{R}_{\text{Lin}} = \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$ (as in [Section 4.4](#)). Recall from [Definition 4.9](#) that a bitification of $\mathbf{f} \in (S_{\leq B}^{<D_S})^{\ell_0}$ is any $\mathbf{g} \in \{0, 1\}^\ell$ with $\mathbf{f} = L \cdot \mathbf{g}_S$. For such $\mathbf{g}$, we say that $((\mu, \mathbf{v}), f, \mathbf{f})$ belongs to $\mathcal{R}_{\text{gen}}^\delta$ *via* $\mathbf{g}$ if $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{g})) < \delta$ and $\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu$. Then, by definition of $\mathcal{R}_{\text{gen}}$ and $\mathcal{R}_{\text{gen}}^\delta$, we have that $((\mu, \mathbf{v}), f, \mathbf{f}) \in \mathcal{R}_{\text{gen}}^\delta$ if and only if $((\mu, \mathbf{v}), f, \mathbf{f})$ belongs to $\mathcal{R}_{\text{gen}}^\delta$ via some bitification $\mathbf{g}$ of $\mathbf{f}$. The same statement holds for $\mathcal{R}_{\text{gen}}$ (with $\delta = 0$). Recall from [Definition 4.9](#) that $L \cdot \mathbf{g}_S \in (S_{\leq B}^{<D_S})^{\ell_0}$ for all $\mathbf{g} \in \{0, 1\}^\ell$.

Given $\mathbf{g} \in \{0, 1\}^\ell$, let $\mathbf{f}_{\mathbf{g}} := L \cdot \mathbf{g}_S \in (S_{\leq B}^{<D_S})^{\ell_0}$. By [Lemma 4.12](#), $\langle \psi(\mathbf{f}_{\mathbf{g}}), \mathbf{v} \rangle = \langle \psi(\mathbf{g}_S), \psi(L)^\top \cdot \mathbf{v} \rangle = \langle \pi(\mathbf{g}), \mathbf{u} \rangle$, and hence

$$(\mathbf{x}', f, \mathbf{g}) \in \mathcal{R}_{\text{core}}^\delta \iff ((\mu, \mathbf{v}), f, \mathbf{f}_{\mathbf{g}}) \text{ belongs to } \mathcal{R}_{\text{gen}}^\delta \text{ via } \mathbf{g}, \quad (60)$$

and likewise for $\mathcal{R}_{\text{core}}$ and $\mathcal{R}_{\text{gen}}$. Completeness then follows from (60) and [Theorem 4.7](#).

Round-by-round knowledge soundness. We decompose [Construction 4.13](#) as in the proof of [Theorem 4.7](#): the first IOR Π'_1 consists in computing $\mathbf{x}'$ given the input $(\mu, \mathbf{v})$, and then applying the IOR Π_1 of that proof (together with OOD sampling if Round 0 is executed, cf. [Section A.2](#)). Π'_1 is perfectly complete by (60). The remaining IORs are Π_{red} and Π_{ioff} . We assume first that OOD (Round 0) is not executed. By the IOR composition result [Lemma 3.7](#), it suffices to show that Π'_1 has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{gen}}^\delta, \mathcal{R}_{\text{PESAT}}^\delta)$ and with the same errors as Π_1 , i.e. $\kappa_0 = \theta(\mathcal{P}, \mathcal{M}, \ell)$. We assume Round 1 triggers, so that in Π'_1 the verifier sends one single challenge $\rho = (q, \pi_{\text{rand}})$ for $q \in \mathcal{P}$ and $\pi_{\text{rand}} \in \mathcal{M}_q$. If Round 1 does not trigger, then the verifier does not send any challenge in Π_1 , and the analysis below applies after replacing ρ with an empty message and the error κ_0 by 0.

Note that Π_1 and Π'_1 are IORs with two rounds, where the last verifier message is empty.

Let KState_1 be the knowledge state function of Π_1 . Recall we defined its extractor as the identity map. We define a state function KState for Π'_1 by setting $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{gen}}^\delta$ when tr contains no verifier message, and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) := \text{KState}_1(\mathbf{x}', \mathbf{y}, \text{tr}, \mathbf{w})$ otherwise. We set $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) := \mathbf{f}_{\mathbf{w}}$ if tr ends with the first verifier message (q, π_{rand}) and $\mathbf{w} \in \{0, 1\}^\ell$. Otherwise we set $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) := \mathbf{w}$. The conditions of [Definition 3.5](#) hold: the first

one (empty transcript) holds by construction; the second one (prover moves) holds because KState is constant before the verifier message, and coincides with KState_1 afterwards (and KState_1 satisfies the condition); the third one (full transcript) holds because Π'_1 and Π_1 have the same output.

Let $\rho = (q, \pi_{\text{rand}})$ be the first verifier message and $\text{tr} = (\mu')$ the first prover message. Suppose that $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho, \mathbf{w}) = 1$ and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho, \mathbf{w})) = 0$. We claim that then $(\mathbf{x}', \mathbf{y}, \mathbf{w})$ is not in $\mathcal{R}_{\text{core}}^\delta$. Indeed, if it was, then $\mathbf{w} \in \{0, 1\}^\ell$, hence $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho, \mathbf{w})$ outputs $\mathbf{f}_{\mathbf{w}}$. Since $(\mathbf{x}', \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{core}}^\delta$, we have by (60) that $(\mathbf{x}, \mathbf{y}, \mathbf{f}_{\mathbf{w}}) \in \mathcal{R}_{\text{gen}}^\delta$. By definition of KState we then obtain $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{f}_{\mathbf{w}}) = 1$, contradicting the assumption $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho, \mathbf{w})) = 0$. Hence $(\mathbf{x}', \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{core}}^\delta$. By definition of KState_1 , $\text{KState}_1(\mathbf{x}', \mathbf{y}, \text{tr}, \mathbf{w}) = 0$. On the other hand, $1 = \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho, \mathbf{w}) = \text{KState}_1(\mathbf{x}', \mathbf{y}, \text{tr} \parallel \rho, \mathbf{w})$. The latter two conditions hold with probability at most κ_0 by the proof of Theorem 4.7. Hence the round-by-round knowledge error of Π'_1 at ρ is at most κ_0 .

Albert: review If Round 0 is executed, we must analyse the round-by-round knowledge soundness of Π'_1 with relaxation $(\mathcal{R}_{\text{gen}}^\delta, \mathcal{R}_{\text{PESAT}}^{\delta, \zeta, y})$ (see Section A.2 for a definition). The arguments above apply almost verbatim: it suffices to let the extractor output $\mathbf{f}_{\mathbf{w}}$ when tr ends with the first verifier message in this case (the OOD sample) and $\mathbf{w} \in \{0, 1\}^\ell$, and otherwise the extractor outputs $\mathbf{w}$. The errors in this case are $(\kappa_{\text{OOD}}, \kappa_0, 0)$ (cf. Theorem A.1), with κ_0 and 0 following from the same arguments as in the proof of Theorem A.1, since after Round 0 Π'_1 proceeds exactly as Π_1 , with $\text{KState} = \text{KState}_1$.

A.2 Soundness of Out Of Domain Sampling (OOD) in Construction 4.4

When δ exceeds the unique decoding radius of $\mathcal{C}$, Round 0 of Construction 4.4 is executed. We modify Theorem 4.7 to accommodate this. Following the approach of [ACFY25] (see also [ACFY24]), we extend the relations we work with, adding the OOD evaluation claim to them as a constraint. Concretely, we define $\mathcal{R}_{\text{PESAT}}^{\zeta, y}$ as the set of the triples $((\mathbf{x}, \zeta, y), \mathbf{y}, \mathbf{w})$ with $\zeta, y \in \mathbb{F}_{2^\nu}$, $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}$ and $\widetilde{\text{Pack}}_\nu(\mathbf{w})(\zeta) = y$, where $\zeta = (\zeta^{2^{i-1}})_{i \in [\log(\ell/\nu)]}$ is the evaluation point of Round 0; $\mathcal{R}_{\text{Lin}}^{\zeta, y}$ is defined analogously, and $\mathcal{R}_{\text{PESAT}}^{\delta, \zeta, y}, \mathcal{R}_{\text{Lin}}^{\delta, \zeta, y}$ are obtained by replacing $\mathcal{R}_{\text{PESAT}}, \mathcal{R}_{\text{Lin}}$ with $\mathcal{R}_{\text{PESAT}}^\delta, \mathcal{R}_{\text{Lin}}^\delta$.

Theorem A.1 (Theorem 4.7 when Round 0 is executed). *Let $\delta \in (0, 1)$ and $L_\delta \geq 1$ be such that $\mathcal{C}$ is (δ, L_δ) -list-decodable, let ℓ/ν be a power of two, and assume that Round 0 of Construction 4.4 is executed and that the rest of hypotheses of Theorem 4.7 hold. Assume the hypotheses from Theorem 4.7 placed on Π_{red} and Π_{iopp} hold when replacing the relations by $\mathcal{R}_{\text{PESAT}}^{\zeta, y}, \mathcal{R}_{\text{Lin}}^{\zeta, y}, \mathcal{R}_{\text{PESAT}}^{\delta, \zeta, y}, \mathcal{R}_{\text{Lin}}^{\delta, \zeta, y}$ (where Π_{red} copies (ζ, y) from its input instance to its output instance). Then the conclusion of Theorem 4.7 holds with one more round and the additional first error term $\kappa_{\text{OOD}} \leq \binom{L_\delta}{2} \cdot (\ell/\nu - 1)/|\mathbb{F}_{2^\nu}|$.*

Lemma A.2 (OOD collision probability). *Let $f \in \mathbb{F}_{2^\nu}^m$, let $\text{List}(f, \delta)$ be the set of $\mathbf{g} \in \mathbb{F}_2^\ell$ with $\Delta(f, \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu(\mathbf{g})) < \delta$, and let $\text{Coll}(f, \zeta)$ be the event that two distinct $\mathbf{g}, \mathbf{g}' \in \text{List}(f, \delta)$ satisfy $\text{Pack}_\nu(\mathbf{g})(\zeta) = \text{Pack}_\nu(\mathbf{g}')(\zeta)$. Then*

$$\Pr_{\zeta \leftarrow \mathbb{F}_{2^\nu}} [\text{Coll}(f, \zeta)] \leq \binom{L_\delta}{2} \cdot \frac{\ell/\nu - 1}{|\mathbb{F}_{2^\nu}|}.$$

Proof. Since $\text{Enc}_\mathcal{C} \circ \text{Pack}_\nu$ is injective, $|\text{List}(f, \delta)| \leq L_\delta$. For distinct $\mathbf{g}, \mathbf{g}'$ in the list, $P := \widetilde{\text{Pack}}_\nu(\mathbf{g}) - \widetilde{\text{Pack}}_\nu(\mathbf{g}')$ is a nonzero multilinear polynomial in $r := \log(\ell/\nu)$ variables, and the substitution $X_i \leftarrow \zeta^{2^{i-1}}$ maps its monomials $\prod_{i \in I} X_i$ injectively to the powers $\zeta^{\sum_{i \in I} 2^{i-1}}$, with exponents in $[0.. \ell/\nu - 1]$. Hence $P(\zeta)$ is a nonzero univariate polynomial in ζ of degree at most $\ell/\nu - 1$, and it vanishes for at most that many ζ . A union bound over the $\binom{L_\delta}{2}$ pairs concludes. $\square$

Proof of Theorem A.1. We explain how to modify the proof of Theorem 4.7. Define the IOR Π_1 as in the proof of Theorem 4.7, except that now it consists of Rounds 0, 1 and 2, and outputs $(\mathbf{x}' = (g^\gamma, g^\mu, \zeta, y), \mathbf{y}' = f, \mathbf{w}' = \pi_2(\mathbf{f}))$. Π_1 is now a 3-round IOR from $\mathcal{R}_{\text{LinBitsRings}}$ to $\mathcal{R}_{\text{PESAT}}^{\zeta, y}$, and it is perfectly complete because the original Π_1 is perfectly complete, and the honest value y equals $\text{Pack}_\nu \circ \pi_2(\mathbf{f})(\zeta)$. The IORs Π_2 and Π_3 are the variants of Π_{red} and Π_{iopp} for $\mathcal{R}_{\text{PESAT}}^{\zeta, y}$ and $\mathcal{R}_{\text{Lin}}^{\zeta, y}$ from the hypotheses. By the composition of IORs result Lemma 3.7, it is enough to show that Π_1 has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{LinBitsRings}}^\delta, \mathcal{R}_{\text{PESAT}}^{\delta, \zeta, y})$ and errors $(\kappa_{\text{OOD}}, \kappa_0, 0)$ (the last verifier message is empty and is merged with the first round of Π_2 , as in the proof of Theorem 4.7).

Let KState be the knowledge state function of the proof of Theorem 4.7, and for a transcript tr of Π_1 let $\bar{\text{tr}}$ denote tr with the messages of Round 0 removed. Write Ev for the condition $\text{Pack}_\nu \circ \pi_2(\mathbf{w})(\zeta) = y$ on $\mathbf{w}$, and Coll for the event $\text{Coll}(f, \zeta)$ of Lemma A.2. We keep the identity extractor, i.e. $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = \mathbf{w}$.

We define $\text{KState}'(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w})$ as follows (we omit the empty first prover message from transcripts): if tr contains no verifier message, it is defined as $\text{KState}(\mathbf{x}, \mathbf{y}, \bar{\text{tr}}, \mathbf{w})$; if $\text{tr} = (\zeta)$, as $\text{KState}(\mathbf{x}, \mathbf{y}, \bar{\text{tr}}, \mathbf{w}) \vee \text{Coll}$; if $\text{tr} = (\zeta, (y, \mu'))$ or $\text{tr} = (\zeta, (y, \mu'), (q, \pi_{\text{rand}}))$, as $(\text{KState}(\mathbf{x}, \mathbf{y}, \bar{\text{tr}}, \mathbf{w}) \wedge \text{Ev}) \vee \text{Coll}$; if $\text{tr} = (\zeta, (y, \mu'), (q, \pi_{\text{rand}}), \mu)$, possibly followed by the empty last verifier message, as $\text{KState}(\mathbf{x}, \mathbf{y}, \bar{\text{tr}}, \mathbf{w}) \wedge \text{Ev}$, which is 1 if and only if the verifier accepts its checks and $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^{\delta, \zeta, y}$, where $(\mathbf{x}', \mathbf{y}') = ((g^\gamma, g^\mu, \zeta, y), f)$ is the verifier's output (if Round 1 is skipped, the same definition applies after removing μ' and (q, π_{rand}) from the transcripts).

We prove that the conditions of Definition 3.5, which hold for KState , also hold for KState' : the first one (empty transcript) holds because $\bar{\text{tr}}$ is empty. The third one (full transcript) holds because $\mathcal{R}_{\text{PESAT}}^{\delta, \zeta, y}$ is $\mathcal{R}_{\text{PESAT}}^\delta$ together with Ev . We now argue that the second one (prover moves) holds: indeed, KState satisfies the second condition, and so KState never changes from 0 to 1 after a prover sends a message. Hence adding the condition Ev cannot make a 0 state become 1. Moreover, Coll depends only on (f, ζ) , so it is unaffected by prover messages, and dropping the disjunctive Coll in the definition of KState' for a full transcript cannot make a 0 state become a 1 state.

At the first verifier message ζ , KState' can only change from 0 to 1 if Coll holds. Hence the first round knowledge soundness error is at most $\Pr_\zeta[\text{Coll}] \leq \kappa_{\text{OOD}}$ by Lemma A.2. For the error for the second round where the verifier sends (q, π_{rand}) (assuming the round is executed), a witness $\mathbf{w}$ such that $\text{KState}'(\mathbf{x}, \mathbf{y}, (\zeta, (y, \mu')), \mathbf{w}) = 0$ and $\text{KState}'(\mathbf{x}, \mathbf{y}, (\zeta, (y, \mu'), (q, \pi_{\text{rand}})), \mathbf{w}) = 1$ satisfies $\neg \text{Coll}$, Ev , $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^\delta$ and $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1$. In particular $\pi_2(\mathbf{w}) \in \text{List}(f, \delta)$.

Since Coll fails, $\pi_2(\mathbf{w})$ is the unique element $\mathbf{g}$ of the list with $\text{Pack}_\nu(\mathbf{g})(\zeta) = y$. Hence $\mathbf{w}$, and with it the polynomial h of the proof of Theorem 4.7, are determined by (f, ζ, y, μ') before (q, π_{rand}) is drawn. Finally, for this fixed $\mathbf{w}$, the proof of Theorem 4.7 bounds by κ_0 the probability over (q, π_{rand}) that $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1$ and $\text{KState}(\mathbf{x}, \mathbf{y}, \mu', \mathbf{w}) = 0$ (the latter occurs if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^\delta$). $\square$